UNIVERSITI POLY-TECH MALAYSIA
UPTM

Name: **MUHAMMAD ANAS BIN HADDRI** **ISHAK BIN ISMAIL** **AHMAD NAUFAL BIN AHMAD FUAD** **MUHAMMAD SHIHABUDDIN AL-FIKRI BIN ANUAR**		
ID Number: **AM2512021976** **AM2512021928** **AM2512021850** **AM2512021974**		
Lecturer: **NURAIN NABILA BINTI BAHAROM**		Section: **SECTION 04**
Course and Course Code: **SWC3633/SWC4443 WEB API DEVELOPMENT**		Submission Date: **14/10/2026**
Assignment No. / Title: **Development of a RESTful Web API for an Event Ticketing and Venue Booking System**		Extension & Late submission: ALLO WED / **DISALLOWED**
Assignment type: **GROUP PROJECT**	% of Assignment Mark: **50%**	Returning Date:

Penalties:

1. 10% of the original mark will be deducted for every one-week period after the submission date
2. No work will be accepted after two weeks of the deadline
3. If you were unable to submit the coursework on time due to extenuating circumstances you may be eligible for an extension
4. Extension will not exceed one week

Declaration: I the undersigned confirm that I have read and agreed to abide by these regulations on plagiarism and cheating. I confirm that this piece of work is my own. I consent to appropriate storage of my work for checking to ensure that there is no plagiarism/ academic cheating.

Signature(s):

This section may be used for feedback or other information:

Table of Contents

1.0 System Overview **5**
1.1 Project Objectives 5
1.1.1 Provide a RESTful JSON API for The Four Required Resources 5
1.1.2 Correct Ticket Numbers 5
1.1.3 Access Limited by Role 6
1.1.4 One Backend for Every Member's Frontend 6
1.2 Intended Users 6
1.3 Main Features 7
1.3.1 CRUD Operations Guarded by Business Rules 7
1.3.2 Authentications and Access Control 7
1.3.3 Validation and Error Handling 7
1.3.4 QR Code Tickets 7
1.3.5 API Management 8
1.4 Technology 8
1.5 Architecture 9
2.0 Database Design **10**
2.1 Entity Relationship Diagram 11
2.2 Relationships 13
2.3 Data Integrity Rules 14
2.4 DDL Script 16
2.5 DML Script 18
3.0 RESTful API Documentation **19**
3.1 Conventions 20
3.2 Status Codes 21
3.3 Authentication 22
3.3.1 Register: POST /api/auth/register 22
3.3.2 Login: POST /api/auth/login 23
3.3.3 My Profile: GET /api/auth/me 24
3.3.4 Refresh Token: POST /api/auth/refresh 25
3.3.5 Logout: POST /api/auth/logout 26
3.4 Users 27
3.4.1 List Users: GET /api/users 27
3.4.2 Create User: POST /api/users 28
3.4.3 Get User: GET /api/users/{id} 29
3.4.4 Update User: PUT /api/users/{id} 30
3.4.5 Delete User: DELETE /api/users/{id} 31
3.5 Venues 32
3.5.1 List Venues: GET /api/venues 32
3.5.2 Get Venue: GET /api/venues/{id} 33

3.5.3 Create Venue: POST /api/venues 34
3.5.4 Update Venue: PUT /api/venues/{id} 35
3.5.5 Delete Venue: DELETE /api/venues/{id} 36
3.6 Events 37
3.6.1 List Events: GET /api/events 37
3.6.2 Get Event: GET /api/events/{id} 38
3.6.3 Create Event: POST /api/events 39
3.6.4 Update Event: PUT /api/events/{id} 40
3.6.5 Delete Event: DELETE /api/events/{id} 41
3.7 Bookings 42
3.7.1 List Bookings: GET /api/bookings 42
3.7.2 Create Booking: POST /api/bookings 43
3.7.3 Get Booking: GET /api/bookings/{id} 44
3.7.4 Update Booking: PUT /api/bookings/{id} 45
3.7.5 Delete Booking: DELETE /api/bookings/{id} 46
3.7.6 Get Booking QR Code: GET /api/bookings/{id}/qr 47
3.8 Tickets 48
3.8.1 Verify Ticket: GET /api/tickets/{reference} 48
4.0 API Testing 49
4.1 The Postman Collection 49
4.2 Representative Requests 50
5.0 System Implementation 60
5.1 Shared Backend 60
5.2 Frontends 62
5.2.1 Member 1: Ishak Bin Ismail (frontend-ishak/) 62
5.2.2 Member 2: Ahmad Naufal bin Ahmad Fuad 65
5.2.3 Member 3: Muhammad Anas Bin Haddri 69
5.2.4 Member 4: Muhammad Shihabuddin Al-Fikri Bin Anuar 73
5.3 Frontend and API Communication 79
5.3.1 Member 1: Ishak Bin Ismail (frontend-ishak/) 79
5.3.2 Member 2: Ahmad Naufal bin Ahmad Fuad (frontend-naufal/) 82
5.3.3 Member 3 : Muhammad Anas Bin Haddri (frontend-anas/) 87
5.3.4 Member 4: Muhammad Shihabuddin Al-Fikri Bin Anuar 93
6.0 System Demonstration 98
6.1 Member 1: Ishak Bin Ismail (frontend-ishak/) 98
6.1.1 Guest 98
6.1.2 Customer 101
6.1.3 Staff 105
6.1.4 Organizer 106
6.1.5 Admin 109
6.1.6 Frontend and Backend Interaction 110

6.2 Member: Ahmad Naufal bin Ahmad Fuad (frontend-naufal) 111
6.2.1 Guest 111
6.2.2 Customer 112
6.2.3 Staff 113
6.2.4 Organiser 113
6.2.5 Admin and access control 116
6.3 Member: Muhammad Anas Bin Haddri (frontend-anas/) 118
6.3.1 Guest 118
6.4 Member: Muhammad Shihabuddin Al-Fikri Bin Anuar (frontend-Al) 119
6.4.1 Main Page and Event Listing 119
7.0 Project Repository 125
7.1 Setup in Brief 126
7.2 Collaboration 127
8.0 Additional and Advanced Features 129
8.1 Security 129
8.1.1 JWT Authentication 129
8.1.2 Password and Roles 130
8.1.3 Role-Based Access Control 130
8.1.4 API Key 131
8.2 Middleware and Error Handling 133
8.2.1 Validation 133
8.2.2 Central Error Handler 133
8.2.3 Logging 134
8.3 API Management 135
8.3.1 Rate Limiting 135
8.3.2 Pagination, Search, Filtering and Sorting 136
8.4 Third-Party API: goQR.me 136
9.0 Debugging and Performance Optimization 138
9.1 Exception Handling 138
9.2 Performance Improvements 139
9.2.1 Fewer Database Queries by Design 140
9.2.2 Indexes by Design 141
9.2.3 Bounded Responses by Design 141
10.0 Individual Reflection and Contribution Record 142
10.1 Member 1: Ishak Bin Ismail (AM2512021928) 142
10.2 Member 2: Ahmad Naufal bin Ahmad Fuad(AM2512021850) 143
10.3 Member 3: Muhammad Shihabuddin Al-Fikri Bin Anuar (AM2512021974) 144
References 145

1.0 System Overview

EventHub lets organisers publish events at venues and lets customers book tickets, with every rule enforced by one shared REST API. Admins and organisers are responsible for managing the venues, the events, and the number of tickets that are available. Customers can browse the events that have been published, book or cancel tickets, and open each booking to produce a ticket complete with a QR code. When checking customers in at the door, staff use the same system to locate a booking and verify the customer's attendance. The API ensures that the number of tickets is kept accurate, prevents two events from using the same venue at the same time, and restricts each action to those roles which are authorised to perform it. Since each group member develops their own frontend interface based on this one backend, they all work with the same data and the same rules.

1.1 Project Objectives

1.1.1 Provide a RESTful JSON API for The Four Required Resources

A RESTful JSON API should be provided for the four required resources which are users, venues, events and bookings. Each of these resources should have its own URL and must support the create, read, update and delete operations that are permitted by its business rules. Thus, any client that is capable of sending HTTP requests and of reading JSON can make use of the system.

1.1.2 Correct Ticket Numbers

Make sure that the ticket numbers are kept accurate so that an event is never oversold or booked twice at a venue. Each booking and cancellation updates the number of tickets still available for an event as part of a database transaction. Also, the API will reject any new event that overlaps with another event at the same venue.

1.1.3 Access Limited by Role

Restrict each action to the roles which are authorised to perform it. When a user logs in they are given a token, and for every protected request the API verifies both the token and the user's role. It also checks to ensure ownership, so that an organiser may alter only their own events and a customer can see only their own bookings.

1.1.4 One Backend for Every Member's Frontend

Each group member should develop their own frontend using the same backend; the backend contains all the data and rules and provides only JSON. It follows that each frontend can freely select its own design and technology while displaying the same information.

1.2 Intended Users

Role	Responsibilities
Guest	Browse published events and venues, register and log in
Customer	Book 1 to 10 tickets, view own bookings and QR code ticket and can cancel before the events start
Staff	See every booking, check customers in and can cancel bookings
Organizer	Create and edit venues, create, publish, cancel and delete their own events and manage the bookings for those events
Admin	Everything above for all records, plus manage users, delete venues and delete cancelled bookings

Table 1.1: Intended Users for The System

1.3 Main Features

1.3.1 CRUD Operations Guarded by Business Rules

You can create, read, update and delete all four resources but every of these operations is subject to business rules. For example, a customer is only allowed to book a published event which still has tickets available and a venue that has events cannot be deleted. If a request violates a rule then it returns a 409 response indicating the reason.

1.3.2 Authentications and Access Control

The system uses a JSON Web Token for logging in, provides role-based access control and includes an API key for machine access. A user has to log in once and then sends the token with all subsequent requests and each route will only accept the roles that have been specified for it. The API key safeguards the ticket check which is called by an external system like a door scanner.

1.3.3 Validation and Error Handling

Validate the request and include a single central error handler which returns the same JSON format in all cases. When the input is invalid, a 422 response is returned together with a message for each field in all other situations a message is returned alongside the appropriate status code allowing the frontend to deal with all errors uniformly.

1.3.4 QR Code Tickets

Each booking has a QR code produced by the goQR.me web API. The backend retrieves the image, stores it for one day and then returns it ready for display. The code contains the booking reference, which can be verified at the venue door.

1.3.5 API Management

The API includes features such as rate limiting, request logging, pagination, search, filtering and sorting. Rate limiting is in place in order to prevent the API from being used too much and to reduce the speed of password guessing, and each request is recorded in a log file. The four list endpoints deliver the results one page at a time and allow the client to search, filter and sort them.

1.4 Technology

Layer	Technology
Backend	PHP 8.5 and Laravel 13
Database	MySQL 8
Authentication	JSON Web Tokens
Third-party API	goQR.me QR code API
Frontend	HTML, CSS, JavaScript and Bootstrap 5.3
Testing	Postman and Newman
Version control	Git and Github

1.5 Architecture

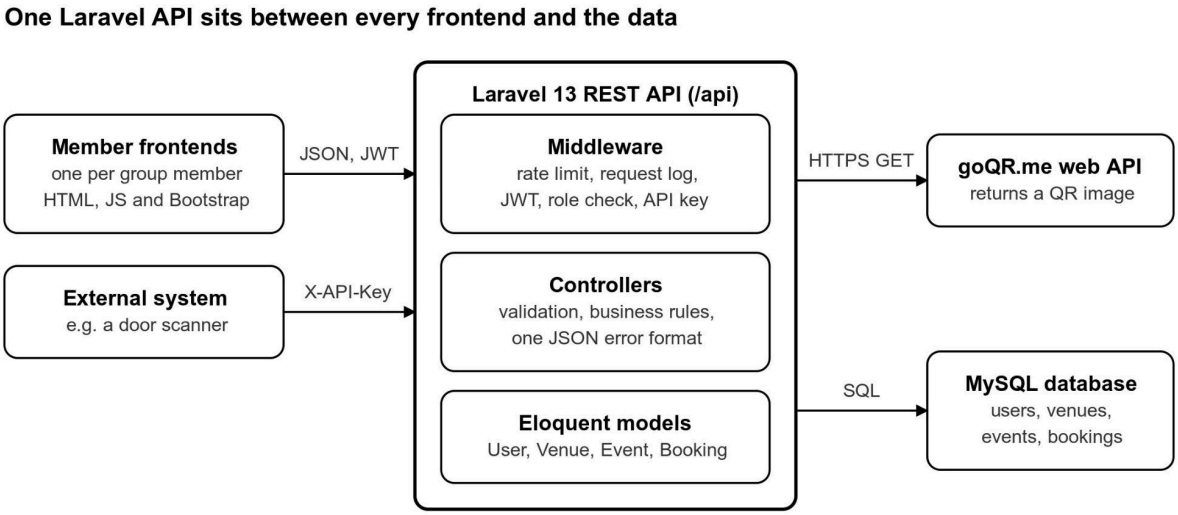


Figure 1.1: System Architecture

The frontend sends JSON over HTTP to the same Laravel API and each request goes through the middleware after which the controller validates it and enforces the business rules, the models then carrying out MySQL reads or writes and only the backend communicates with goQR.me so the frontends have no need to know anything about the external service.

2.0 Database Design

The database consists of four tables that are related to one another and it rejects invalid data by means of keys, unique indexes and CHECK constraints.

The system uses MySQL with the InnoDB storage engine, which supports foreign keys, and texts are stored using utf8mb4 so that names in all languages are preserved. The four tables are called users, venues, events and bookings. An event is associated with one venue and one organiser, and a booking connects one customer to one event. Each table has an auto-incrementing bigint primary key as well as created_at and updated_at timestamps.

The database structure is set up using the DDL script database/01_ddl.sql and the DML script database/02_dml.sql is then used to insert 26 sample records consisting of 7 users, 5 venues, 6 events and 8 bookings; this satisfies the requirement for a minimum of five records per table. The sample users include all four different roles and the bookings contain examples of confirmed, cancelled and checked-in ones, which means that all the business rules can be tested on a fresh database.

2.1 Entity Relationship Diagram

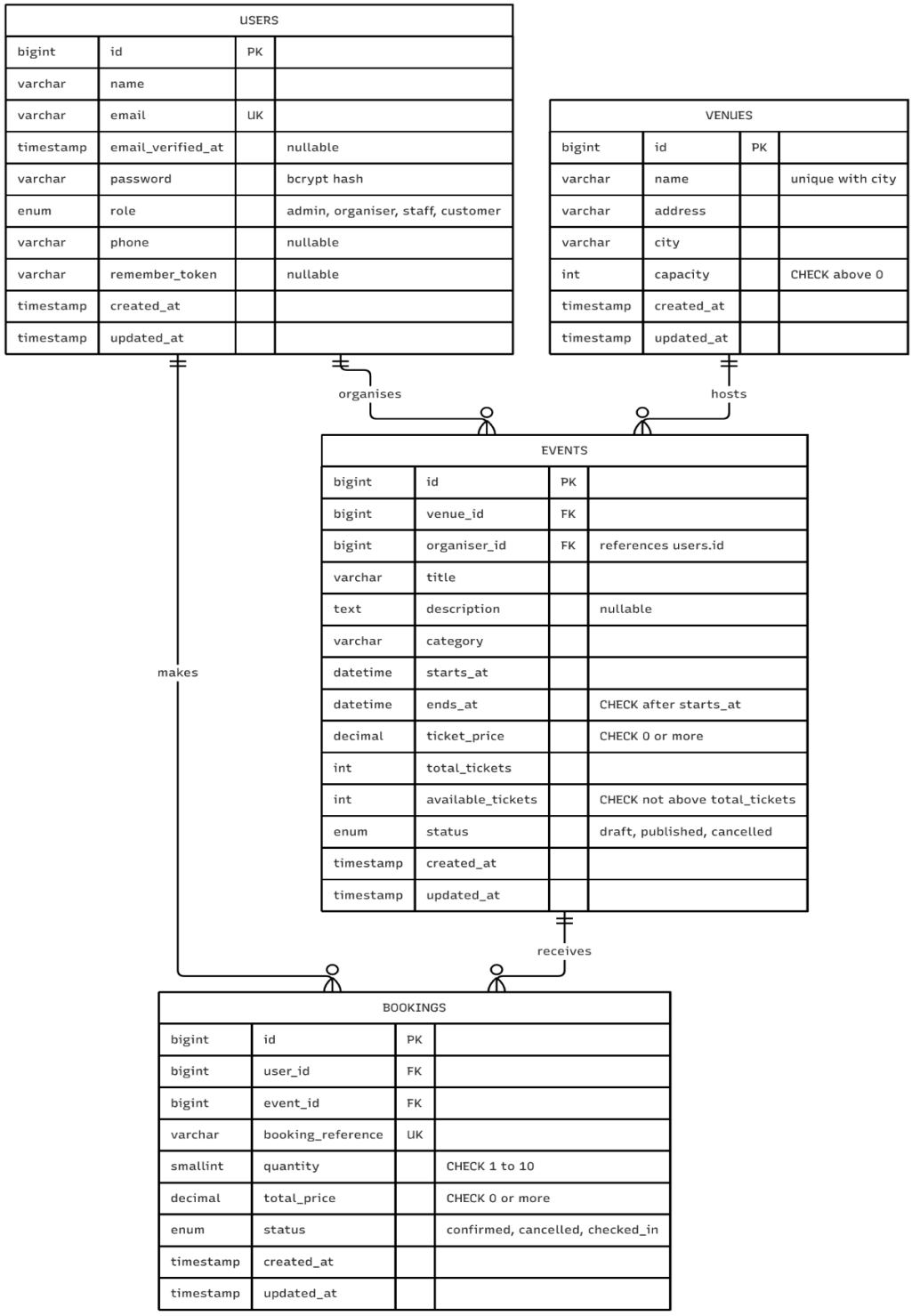


Figure 2.1: ERD Diagram

Every box is a table and each row in it represents a column having a type, a name, a key and any constraints. The PK stands for the primary key, the FK for a foreign key and the UK for a unique key which means that values such as users.email and bookings.booking_reference will never be repeated.

The lines are indicated by means of crow's foot notation. The double bar at one end stands for "exactly one" and the circle together with the three prongs at the other end means "zero or many". When the four lines are read:

- **organizes**: a user who acts as an organizer may organise either no events or a number of events, and each event has exactly one organizer as determined by the events.organizer_id.

- **hosts**: a venue may host either no events or a number of events, and each event occurs at exactly one venue as specified by events.venue_id.

- **makes**: a customer can make either zero or multiple bookings, and each booking is associated with exactly one user via the bookings.user_id.

- **receives**: An event may have zero or multiple bookings, and each booking is for just one event as determined by the event_id in bookings.

The two child tables are events and bookings. Bookings also function as the link between users and events since a customer can book several events and an event can have many customers booking it. The notes column displays the rules that the database enforces on its own, for example that the capacity must be above 0, that ends_at comes after starts_at, that available_tickets is not above total_tickets and that quantity is between 1 and 10. The diagram's source is database/erd.md and the image is database/erd.png in the repository.

2.2 Relationships

Relationship	Type	Foreign key	On delete
A user organizes events	One to many	events.organizer_id to users.id	RESTRICT
A venue hosts events	One to many	event.venue_id to venues.id	RESTRICT
A user makes bookings	One to many	bookings.user_id to users.id	RESTRICT
An event receive bookings	One to many	bookings.event_id to events.id	RESTRICT

Since all foreign keys are set to RESTRICT, it is not possible to delete a record that is depended upon by other records. First, the API checks this and then responds with a 409 message, thus ensuring that the user does not see the original database error.

RESTRICT was deliberately preferred to CASCADE. In the case of CASCADE, deleting a venue would silently remove its events, along with all the bookings for those events; customers would thus lose the tickets they had paid for and their booking history would be deleted. With RESTRICT, however, the database refuses to carry out the deletion and the record must be addressed first.

The same rules are applied to the API before it even gets to the database. If a venue has events, it returns a 409 error with the message "This venue has events and cannot be deleted". In a similar manner, a user who has events or bookings is also refused. An event which has bookings cannot be deleted either; instead the organiser is asked to cancel it. A booking can only be deleted after it has been cancelled. Records that are important are never lost: their status is changed to cancelled and the history remains complete.

2.3 Data Integrity Rules

Table	Rule	Enforced by
users	Email is unique	UNIQUE index
users	Role is admin, organizer, staff or customer	ENUM, default customer
venues	Name is unique within a city	UNIQUE index on (name, city)
venues	Capacity is above 0	CHECK
events	End time is after start time	CHECK
events	Ticket price is 0 or more	CHECK
events	Available tickets never exceed total tickets	CHECK, unsigned integers
events	Status is draft, published or cancelled	ENUM, default draft
bookings	Booking reference is unique	UNIQUE index
bookings	Quantity is 1 to 10	CHECK
bookings	Total price is 0 or more	CHECK

The data types are chosen according to the type of data they will hold: DECIMAL for money, DATETIME for event times, unsigned integers for counts and ENUM for fixed lists. Two additional indexes speed up common queries: one on users.role and another on events by (status, starts_at) for the public event list.

Each type has been selected for a specific reason. Money uses DECIMAL(8,2) for ticket prices and DECIMAL(10,2) for booking totals rather than FLOAT because floating-point numbers cannot represent values such as 0.10 exactly and rounding errors in prices are

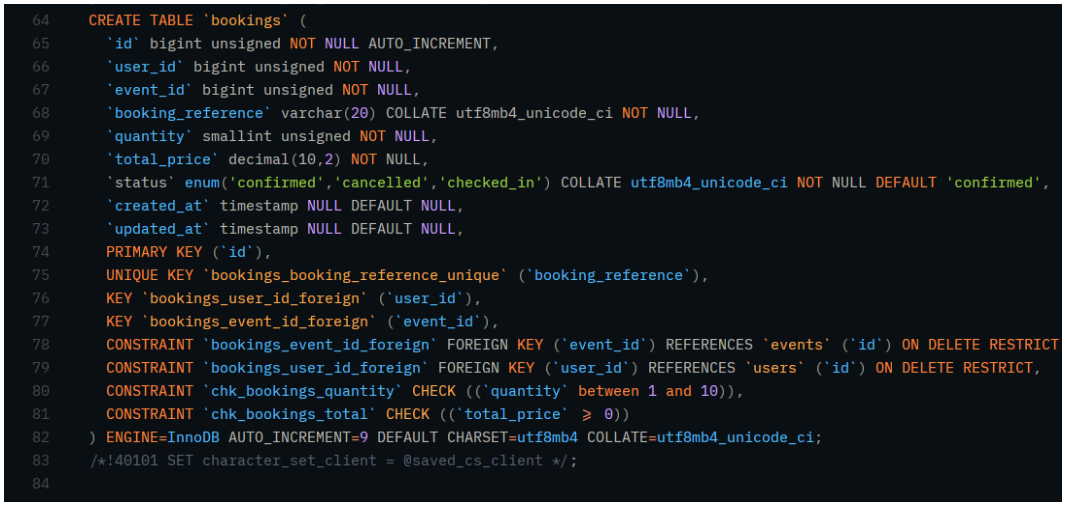
not acceptable. Counts like capacity, total_tickets and available_tickets are unsigned integers and therefore cannot have a negative number stored. quantity is of type SMALLINT since this is sufficient for the maximum of 10 tickets permitted per booking. For fixed lists such as user roles and the event and booking statuses, ENUM columns are used so that any value outside the list or one resulting from a typing mistake is rejected by the database, and each of these has an appropriate default value: customer, draft and confirmed. All primary and foreign keys are unsigned BIGINTs so that each foreign key exactly matches the key it refers to.

MySQL automatically creates an index for every foreign key column, which is why joins between the bookings, events, users and venues remain quick. The composite index on the events table for the columns status and starts_at matches the way the public event list is queried—which always requests published events in date order—so that MySQL can read the relevant rows directly rather than having to scan the entire table.

2.4 DDL Script

```sql
CREATE TABLE `users` (
  `id` bigint unsigned NOT NULL AUTO_INCREMENT,
  `name` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL,
  `email` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL,
  `email_verified_at` timestamp NULL DEFAULT NULL,
  `password` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL,
  `role` enum('admin','organiser','staff','customer') COLLATE utf8mb4_unicode_ci NOT NULL DEFAULT 'customer',
  `phone` varchar(20) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `remember_token` varchar(100) COLLATE utf8mb4_unicode_ci DEFAULT NULL,
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `users_email_unique` (`email`),
  KEY `users_role_index` (`role`)
) ENGINE=InnoDB AUTO_INCREMENT=8 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
/*!40101 SET character_set_client = @saved_cs_client */;
/*!40101 SET @saved_cs_client     = @@character_set_client */;
/*!50503 SET character_set_client = utf8mb4 */;
CREATE TABLE `venues` (
  `id` bigint unsigned NOT NULL AUTO_INCREMENT,
  `name` varchar(150) COLLATE utf8mb4_unicode_ci NOT NULL,
  `address` varchar(255) COLLATE utf8mb4_unicode_ci NOT NULL,
  `city` varchar(100) COLLATE utf8mb4_unicode_ci NOT NULL,
  `capacity` int unsigned NOT NULL,
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `venues_name_city_unique` (`name`,`city`),
  CONSTRAINT `chk_venues_capacity` CHECK ((`capacity` > 0))
) ENGINE=InnoDB AUTO_INCREMENT=6 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
/*!40101 SET character_set_client = @saved_cs_client */;
/*!40101 SET @saved_cs_client     = @@character_set_client */;
/*!50503 SET character_set_client = utf8mb4 */;
```

```sql
CREATE TABLE `events` (
  `id` bigint unsigned NOT NULL AUTO_INCREMENT,
  `venue_id` bigint unsigned NOT NULL,
  `organiser_id` bigint unsigned NOT NULL,
  `title` varchar(150) COLLATE utf8mb4_unicode_ci NOT NULL,
  `description` text COLLATE utf8mb4_unicode_ci,
  `category` varchar(50) COLLATE utf8mb4_unicode_ci NOT NULL,
  `starts_at` datetime NOT NULL,
  `ends_at` datetime NOT NULL,
  `ticket_price` decimal(8,2) NOT NULL,
  `total_tickets` int unsigned NOT NULL,
  `available_tickets` int unsigned NOT NULL,
  `status` enum('draft','published','cancelled') COLLATE utf8mb4_unicode_ci NOT NULL DEFAULT 'draft',
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `events_venue_id_foreign` (`venue_id`),
  KEY `events_organiser_id_foreign` (`organiser_id`),
  KEY `events_status_starts_at_index` (`status`,`starts_at`),
  CONSTRAINT `events_organiser_id_foreign` FOREIGN KEY (`organiser_id`) REFERENCES `users` (`id`) ON DELETE RE
  CONSTRAINT `events_venue_id_foreign` FOREIGN KEY (`venue_id`) REFERENCES `venues` (`id`) ON DELETE RESTRICT,
  CONSTRAINT `chk_events_dates` CHECK ((`ends_at` > `starts_at`)),
  CONSTRAINT `chk_events_price` CHECK ((`ticket_price` >= 0)),
  CONSTRAINT `chk_events_tickets` CHECK ((`available_tickets` <= `total_tickets`))
) ENGINE=InnoDB AUTO_INCREMENT=7 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
/*!40101 SET character_set_client = @saved_cs_client */;
/*!40101 SET @saved_cs_client     = @@character_set_client */;
/*!50503 SET character_set_client = utf8mb4 */;
```



```
CREATE TABLE `bookings` (
  `id` bigint unsigned NOT NULL AUTO_INCREMENT,
  `user_id` bigint unsigned NOT NULL,
  `event_id` bigint unsigned NOT NULL,
  `booking_reference` varchar(20) COLLATE utf8mb4_unicode_ci NOT NULL,
  `quantity` smallint unsigned NOT NULL,
  `total_price` decimal(10,2) NOT NULL,
  `status` enum('confirmed','cancelled','checked_in') COLLATE utf8mb4_unicode_ci NOT NULL DEFAULT 'confirmed',
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `bookings_booking_reference_unique` (`booking_reference`),
  KEY `bookings_user_id_foreign` (`user_id`),
  KEY `bookings_event_id_foreign` (`event_id`),
  CONSTRAINT `bookings_event_id_foreign` FOREIGN KEY (`event_id`) REFERENCES `events` (`id`) ON DELETE RESTRICT
  CONSTRAINT `bookings_user_id_foreign` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE RESTRICT,
  CONSTRAINT `chk_bookings_quantity` CHECK ((`quantity` between 1 and 10)),
  CONSTRAINT `chk_bookings_total` CHECK ((`total_price` >= 0))
) ENGINE=InnoDB AUTO_INCREMENT=9 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
/*!40101 SET character_set_client = @saved_cs_client */;
```

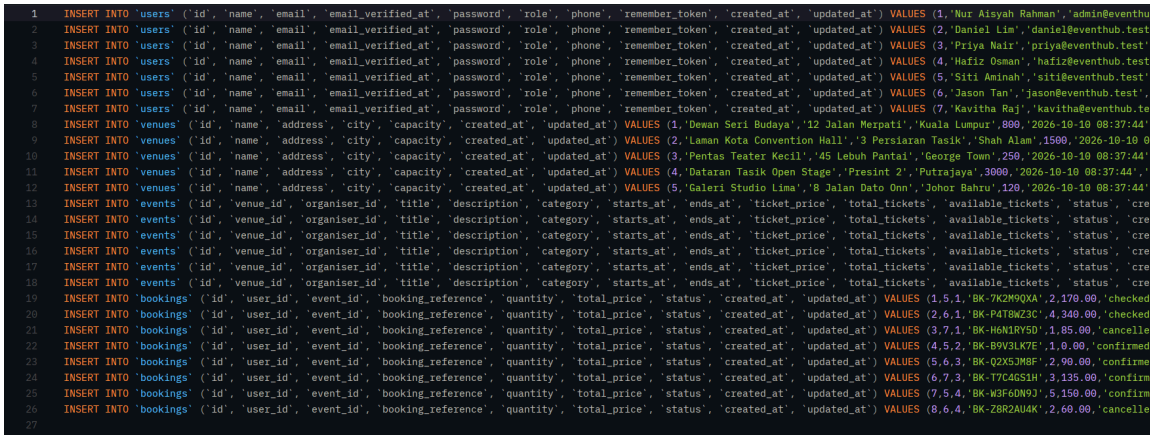
Figure 2.2: DDL Script

The following Data Definition Language (DDL) script is used to create the database structure and is stored in the repository under the path database/01_ddl.sql. Since the COLLATE utf8mb4_unicode_ci clause is repeated on all of the text columns in the file, this clause has been omitted in the listing for the sake of keeping it readable. This makes no difference, as each table already has that collation as its default.

The script was taken from the working database and therefore agrees with the tables that the API uses. These tables are set up by the Laravel migrations in the backend/database/migrations directory. The script creates the four tables in the order of their dependencies: it first creates the users and venues tables, then the events table—which refers to both of them—and finally the bookings table, which refers to both the users and the events. It is important to maintain this order since a foreign key can only refer to a table that has already been created. Each CREATE TABLE statement assigns the primary key, the unique keys, the indexes, the foreign keys with ON DELETE RESTRICT, and the CHECK constraints mentioned above. The statements also specify the use of the InnoDB engine and the utf8mb4 character set.

The script can run by itself without the need for Laravel. The README document illustrates how to load it, along with the DML script, into a blank database in order to test it.

2.5 DML Script



```
INSERT INTO `users` (`id`, `name`, `email`, `email_verified_at`, `password`, `role`, `phone`, `remember_token`, `created_at`, `updated_at`) VALUES (1,'Nur Aisyah Rahman','admin@eventhu
INSERT INTO `users` (`id`, `name`, `email`, `email_verified_at`, `password`, `role`, `phone`, `remember_token`, `created_at`, `updated_at`) VALUES (2,'Daniel Lim','daniel@eventhub.test
INSERT INTO `users` (`id`, `name`, `email`, `email_verified_at`, `password`, `role`, `phone`, `remember_token`, `created_at`, `updated_at`) VALUES (3,'Priya Nair','priya@eventhub.test'
INSERT INTO `users` (`id`, `name`, `email`, `email_verified_at`, `password`, `role`, `phone`, `remember_token`, `created_at`, `updated_at`) VALUES (4,'Hafiz Osman','hafiz@eventhub.test
INSERT INTO `users` (`id`, `name`, `email`, `email_verified_at`, `password`, `role`, `phone`, `remember_token`, `created_at`, `updated_at`) VALUES (5,'Siti Aminah','siti@eventhub.test'
INSERT INTO `users` (`id`, `name`, `email`, `email_verified_at`, `password`, `role`, `phone`, `remember_token`, `created_at`, `updated_at`) VALUES (6,'Jason Tan','jason@eventhub.test',
INSERT INTO `users` (`id`, `name`, `email`, `email_verified_at`, `password`, `role`, `phone`, `remember_token`, `created_at`, `updated_at`) VALUES (7,'Kavitha Raj','kavitha@eventhub.te
INSERT INTO `venues` (`id`, `name`, `address`, `city`, `capacity`, `created_at`, `updated_at`) VALUES (1,'Dewan Seri Budaya','12 Jalan Merpati','Kuala Lumpur',800,'2026-10-10 08:37:44'
INSERT INTO `venues` (`id`, `name`, `address`, `city`, `capacity`, `created_at`, `updated_at`) VALUES (2,'Laman Kota Convention Hall','3 Persiaran Tasik','Shah Alam',1500,'2026-10-10 0
INSERT INTO `venues` (`id`, `name`, `address`, `city`, `capacity`, `created_at`, `updated_at`) VALUES (3,'Pentas Teater Kecil','45 Lebuh Pantai','George Town',250,'2026-10-10 08:37:44'
INSERT INTO `venues` (`id`, `name`, `address`, `city`, `capacity`, `created_at`, `updated_at`) VALUES (4,'Dataran Tasik Open Stage','Presint 2','Putrajaya',3000,'2026-10-10 08:37:44',
INSERT INTO `venues` (`id`, `name`, `address`, `city`, `capacity`, `created_at`, `updated_at`) VALUES (5,'Galeri Studio Lima','8 Jalan Dato Onn','Johor Bahru',120,'2026-10-10 08:37:44'
INSERT INTO `events` (`id`, `venue_id`, `organiser_id`, `title`, `description`, `category`, `starts_at`, `ends_at`, `ticket_price`, `total_tickets`, `available_tickets`, `status`, `cre
INSERT INTO `events` (`id`, `venue_id`, `organiser_id`, `title`, `description`, `category`, `starts_at`, `ends_at`, `ticket_price`, `total_tickets`, `available_tickets`, `status`, `cre
INSERT INTO `events` (`id`, `venue_id`, `organiser_id`, `title`, `description`, `category`, `starts_at`, `ends_at`, `ticket_price`, `total_tickets`, `available_tickets`, `status`, `cre
INSERT INTO `events` (`id`, `venue_id`, `organiser_id`, `title`, `description`, `category`, `starts_at`, `ends_at`, `ticket_price`, `total_tickets`, `available_tickets`, `status`, `cre
INSERT INTO `events` (`id`, `venue_id`, `organiser_id`, `title`, `description`, `category`, `starts_at`, `ends_at`, `ticket_price`, `total_tickets`, `available_tickets`, `status`, `cre
INSERT INTO `events` (`id`, `venue_id`, `organiser_id`, `title`, `description`, `category`, `starts_at`, `ends_at`, `ticket_price`, `total_tickets`, `available_tickets`, `status`, `cre
INSERT INTO `bookings` (`id`, `user_id`, `event_id`, `booking_reference`, `quantity`, `total_price`, `status`, `created_at`, `updated_at`) VALUES (1,5,1,'BK-7K2M9QXA',2,170.00,'checked
INSERT INTO `bookings` (`id`, `user_id`, `event_id`, `booking_reference`, `quantity`, `total_price`, `status`, `created_at`, `updated_at`) VALUES (2,6,1,'BK-P4T8WZ3C',4,340.00,'checked
INSERT INTO `bookings` (`id`, `user_id`, `event_id`, `booking_reference`, `quantity`, `total_price`, `status`, `created_at`, `updated_at`) VALUES (3,7,1,'BK-H6N1RY5D',1,85.00,'cancelle
INSERT INTO `bookings` (`id`, `user_id`, `event_id`, `booking_reference`, `quantity`, `total_price`, `status`, `created_at`, `updated_at`) VALUES (4,5,2,'BK-B9V3LK7F',1,0.00,'confirmed
INSERT INTO `bookings` (`id`, `user_id`, `event_id`, `booking_reference`, `quantity`, `total_price`, `status`, `created_at`, `updated_at`) VALUES (5,6,3,'BK-Q2X5JM8F',2,90.00,'confirme
INSERT INTO `bookings` (`id`, `user_id`, `event_id`, `booking_reference`, `quantity`, `total_price`, `status`, `created_at`, `updated_at`) VALUES (6,7,3,'BK-T7C4GS1H',3,135.00,'confirm
INSERT INTO `bookings` (`id`, `user_id`, `event_id`, `booking_reference`, `quantity`, `total_price`, `status`, `created_at`, `updated_at`) VALUES (7,5,4,'BK-W3F6DN9J',5,150.00,'confirm
INSERT INTO `bookings` (`id`, `user_id`, `event_id`, `booking_reference`, `quantity`, `total_price`, `status`, `created_at`, `updated_at`) VALUES (8,6,4,'BK-Z8R2AU4K',2,60.00,'cancelle
```

Figure 2.3: DML Script

The DML script listed above is used to enter sample data into the tables; it is stored in the repository under the name database/02_dml.sql and includes 7 users, 5 venues, 6 events and 8 bookings. Since the aim is to keep the display concise, several rows are shown in each INSERT statement here, the password hashes have been shortened and the timestamp columns omitted. The file in the repository, by contrast, has one INSERT statement for each row with all the columns given in full.

The rows are inserted in the order in which the tables were created: first the users, then the venues, followed by the events and finally the bookings. In this way, every foreign key value already exists when the row referring to it is inserted. The sample data has been selected so that each rule in the system can be displayed right away. Of the seven users all four roles are represented: one admin, two organisers, one staff member and three customers. The six events consist of four that are published, one that is a draft and one that has been cancelled, and one of the published events has already taken place. The eight bookings include four confirmed ones, two cancelled ones and two that have been checked in. The number of available_tickets for each event has already taken into account the bookings made against it, which ensures that the figures remain consistent from the first request.

3.0 RESTful API Documentation

There are 27 endpoints available via http://127.0.0.1:8000/api, and all requests and response bodies are in JSON format. Sample error responses for each endpoint are included in the same documentation, which can be found in docs/API.md within the repository.

The resources are grouped according to their type: authentication (5), users (5), venues (5), events (5), bookings (6), and one endpoint for checking tickets which is used by the door scanner. The URLs adhere to REST naming conventions with each resource expressed as a plural noun, for example /api/events, and a single item is referenced by its id, as in /api/events/{id}. The HTTP method indicates the action to be performed: GET is used for reading, POST for creating, PUT or PATCH for updating and DELETE for deletion. Since actions like logging in are located under /api/auth, none of the URLs include a verb that would normally be used for creating, reading, updating or deleting an item.

The section is made up of nine parts. Part 3.1, Conventions, states the rules that apply to all requests, including authentication, roles, rate limits, the date format and the structure of the responses. Part 3.2, Status codes, gives a list of all the HTTP status codes that the API returns and the meaning of each one in this API. Parts 3.3 through 3.8 then provide a full description of each endpoint, grouped by resource, specifying its purpose, URL, HTTP method, who may call it, the request parameters or body, an example of the JSON response and the relevant status codes.

3.1 Conventions

Item	Value
User authentication	JWT. Log in, then send Authorization: Bearer <token>
Machine authentication	X-API-Key header, used only by /api/tickets/{reference}
Roles	admin, organizer, staff, customer
Rate limits	120 requests a minute per user or per IP address for guests. Login, 5 attempts a minute per email and IP address
Dates	YYYY-MM-DD HH:MM:SS, Malaysian time (Asia/Kuala_Lumpur)
Updates	PATCH is accepted wherever PUT is listed
One item	{ "message": "...", "data": { ... } }. message is present on create, update and delete
Lists	Paginated. data holds the items, next to current_page, per_page, total, last_page and page links. The samples below leave out the page links
Errors	{ "message": "..." }. Validation errors add an errors object with one entry per field

3.2 Status Codes

Code	Meaning
200	Success
201	Created
401	Not logged in, or the token or API key is missing or invalid
403	Logged in, but the role or ownership does not allow the action
404	Resource or URL not found
405	HTTP method not supported for the URL
409	The request breaks a business rule
422	Validation failed
429	Rate limit reached
500	Unexpected server error
502	The third-party QR code service failed

Status codes in the 2xx range indicate that the request has been successful, those in the 4xx range show that the client needs to make a change, and the 5xx range means that the issue lies with the server. Some of the codes may appear similar but have different meanings. A 401 indicates that the caller is not logged in, whereas a 403 means that they are logged in but their role does not permit the action. A 422 means that the data is invalid, and a 409 means that the data is valid but violates a business rule, for example by booking more tickets than are available. All errors follow the same JSON structure, consisting of a message and an errors object in the case of a 422, so the frontend can deal with all errors in the same way.

3.3 Authentication

3.3.1 Register: POST /api/auth/register


```
{
  "message": "Registration successful.",
  "data": {
    "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...(shortened)",
    "token_type": "bearer",
    "expires_in": 3600,
    "user": {
      "name": "Postman Tester",
      "email": "postman.1791654092940@eventhub.test",
      "phone": "012-555 0200",
      "role": "customer",
      "updated_at": "2026-10-11 01:41:33",
      "created_at": "2026-10-11 01:41:33",
      "id": 8
    }
  }
}
```


Figure 3.1: POST /api/auth/register Sample Response

This public endpoint is used to set up a customer account and provides a login token as a result, meaning that it is not necessary to be signed in order to use it. All new accounts are customer accounts, and any role value included in the body is disregarded. The body has to contain a name of 255 characters or fewer, a unique email address, a password which is at least 8 characters long and a matching password_confirmation. A phone number of up to 20 characters is optional. If the request is successful, the response will be 201 together with the account and its token; if the input is invalid, a 422 response code is returned, and if there are too many requests, a 429 response code is returned.

3.3.2 Login: POST /api/auth/login


```
{
  "message": "Login successful.",
  "data": {
    "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...(shortened)",
    "token_type": "bearer",
    "expires_in": 3600,
    "user": {
      "id": 5,
      "name": "Siti Aminah",
      "email": "siti@eventhub.test",
      "email_verified_at": null,
      "role": "customer",
      "phone": "012-555 0105",
      "created_at": "2026-10-11 01:41:31",
      "updated_at": "2026-10-11 01:41:31"
    }
  }
}
```


Figure 3.2: POST /api/auth/login Sample Response

A public endpoint allows an email and password to be exchanged for a JSON Web Token, requiring that both the email and the password be included in the body. The token is valid for 60 minutes and is sent out on protected requests in the form of Authorization: Bearer <token>. If the login is successful, the response will be 200 together with the token and the user details; if the email or password is incorrect, a 401 response is returned, and if the required fields are missing, a 422 response is given. When there have been more than five attempts in a minute, a 429 response is returned, which helps to prevent password guessing.

3.3.3 My Profile: GET /api/auth/me

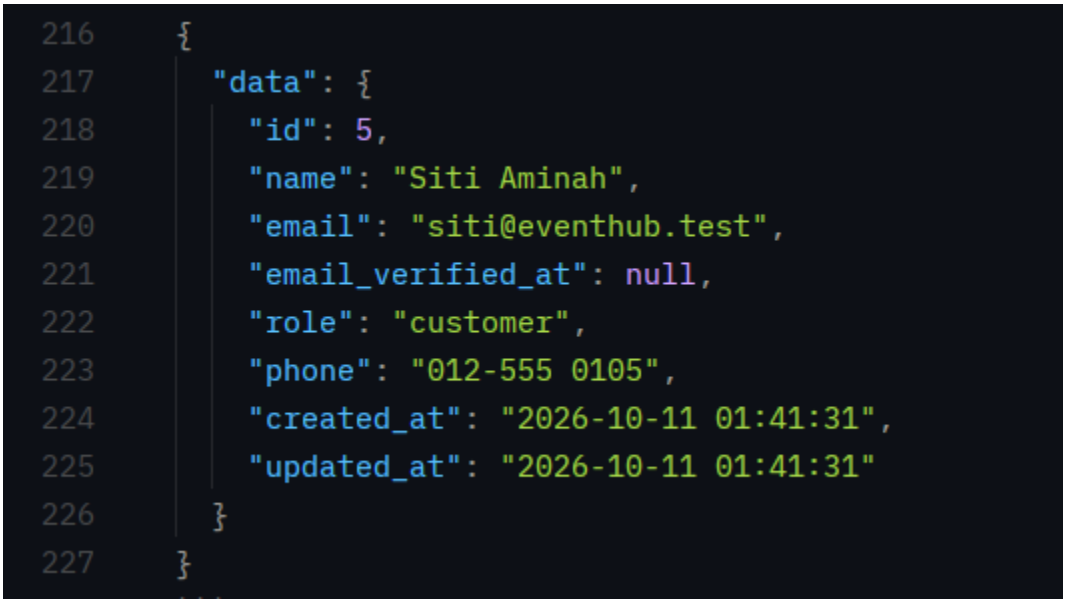


Figure 3.3: GET /api/auth/me Sample Response

The endpoint provides the details of the user who is currently logged in. Since the token identifies the person making the request, any user who is logged in is able to call this endpoint and it requires no parameters or a body. When a frontend loads a page it uses this endpoint to verify that the saved token is still valid and to obtain the user's name and role. If the token is valid the system returns status 200 together with the user details, while if the token is missing, invalid or has expired a 401 response is returned.

3.3.4 Refresh Token: POST /api/auth/refresh



Figure 3.4: POST /api/auth/refresh Sample Response

This endpoint allows the current token to be replaced with a new one, after which the old token immediately becomes invalid. Any user who is logged in is able to use this endpoint, and it requires no parameters or a body since the token to be replaced is included in the Authorization header. A frontend application can make use of it in order to keep a user signed in without having to ask them for their password once more. If the token is valid, the response will be a 200 status code together with the new token, whereas if the token is missing, invalid or has expired, a 401 status code is returned.

3.3.5 Logout: POST /api/auth/logout

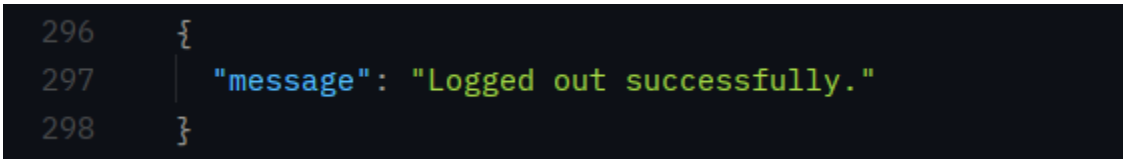


Figure 3.5: POST /api/auth/logout Sample Response

This endpoint logs the user out by adding their current token to a blacklist, which means that the token can no longer be used even if it has been copied. Since any user who is logged in can use this endpoint, it does not require any parameters or a body since the token to be cancelled is the one included in the Authorization header. If the token is valid, the response is a 200 status code together with a brief confirmation message, while if the token is missing, invalid or expired a 401 response is returned.

3.4 Users

3.4.1 List Users: GET /api/users


```
{
  "current_page": 1,
  "data": [
    {
      "id": 1,
      "name": "Nur Aisyah Rahman",
      "email": "admin@eventhub.test",
      "email_verified_at": null,
      "role": "admin",
      "phone": "012-555 0101",
      "created_at": "2026-10-11 01:41:31",
      "updated_at": "2026-10-11 01:41:31"
    }
  ],
  "first_page_url": "http://127.0.0.1:8000/api/users?page=1",
  "from": 1,
  "last_page": 2,
  "last_page_url": "http://127.0.0.1:8000/api/users?page=2",
  "links": [ "...page links..." ],
  "next_page_url": "http://127.0.0.1:8000/api/users?page=2",
  "path": "http://127.0.0.1:8000/api/users",
  "per_page": 5,
  "prev_page_url": null,
  "to": 5,
  "total": 8
}
```


Figure 3.6: GET /api/users Sample Response

The endpoint available only to administrators shows a list of user accounts. The list can be filtered using optional query parameters: the search option matches either a name or an email, while the role parameter restricts the results to a specific role. The results can be ordered by means of sort (using id, name, email, role or created_at) and direction (asc or desc). They are delivered one page at a time via the page and per_page parameters (the number ranging from 1 to 100, with a default of 10). A response code of 200 is given when the request is successful, 401 is returned if the user is not logged in, 403 is given if the user is not an administrator, and 422 is sent when the filter is invalid.

3.4.2 Create User: POST /api/users


```
{
  "message": "User created.",
  "data": {
    "name": "Aiman Hakim",
    "email": "aiman.1791654092940@eventhub.test",
    "phone": "012-555 0199",
    "role": "staff",
    "updated_at": "2026-10-11 01:41:37",
    "created_at": "2026-10-11 01:41:37",
    "id": 9
  }
}
```


Figure 3.7: POST /api/users Sample Response

The endpoint which is restricted to administrators can be used to create a user with any role; it is by means of this endpoint that staff, organiser and admin accounts are created. The data sent must include a name, a unique email address, a password of at least eight characters and a role (which can be admin, organiser, staff or customer), together with an optional phone number. It sends back a response code 201 when successful, 401 if the user is not logged in, 403 if the person is not an admin, and 422 when validation fails.

3.4.3 Get User: GET /api/users/{id}


```
{
  "data": {
    "id": 9,
    "name": "Aiman Hakim",
    "email": "aiman.1791654092940@eventhub.test",
    "email_verified_at": null,
    "role": "staff",
    "phone": "012-555 0199",
    "created_at": "2026-10-11 01:41:37",
    "updated_at": "2026-10-11 01:41:37"
  }
}
```


Figure 3.8: GET api/users/{id} Sample Response

The endpoint which is available only to administrators gives one user, the one specified by the {id} in the URL, and it has no other parameters or request body. It sends back a 200 status code together with the user, a 401 if the person is not logged in, a 403 if the person is not an administrator, and a 404 if there is no user with that id.

3.4.4 Update User: PUT /api/users/{id}

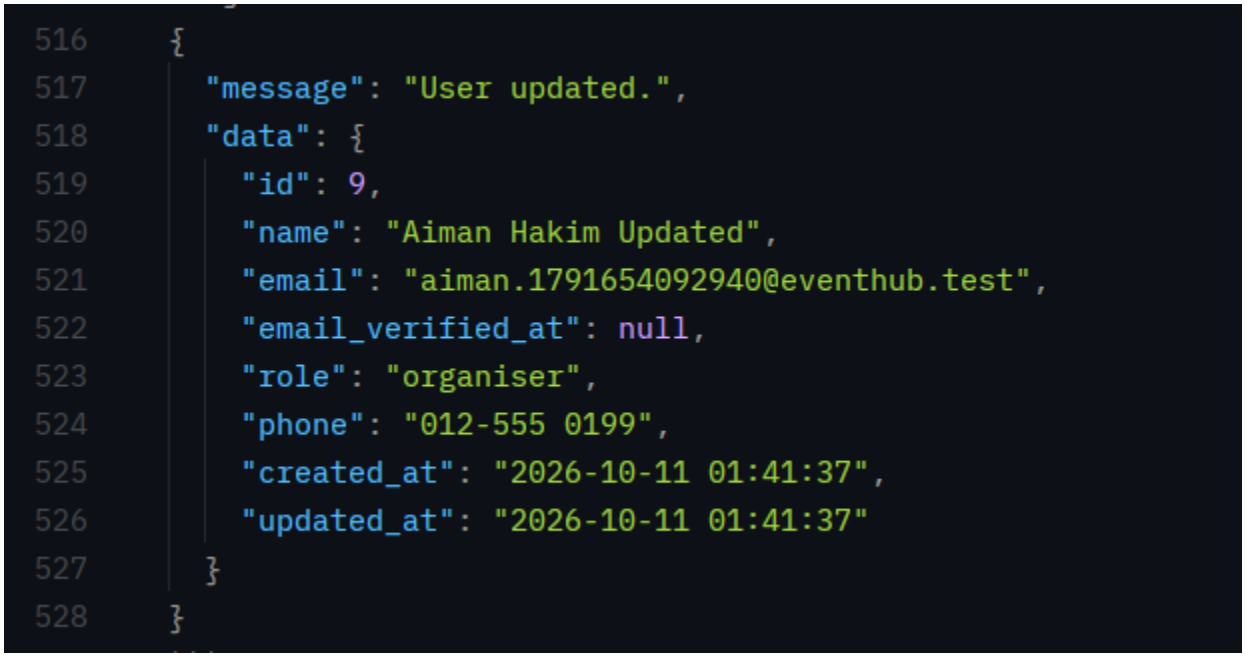


Figure 3.9: PUT /api/users/{id} Sample Response

The endpoint which is available only to administrators is used for updating a user, with only the fields included in the request being modified. The data in the body may contain the name, a unique email address, a password consisting of at least eight characters, the role and the phone number. It sends back a response code 200 when successful, 401 if the user is not logged in, 403 if the person is not an administrator, 404 when the user does not exist and 422 when validation fails. It also returns a 409 code if an administrator attempts to change their own role, thus ensuring that the system is never left without an administrator.

3.4.5 Delete User: DELETE /api/users/{id}

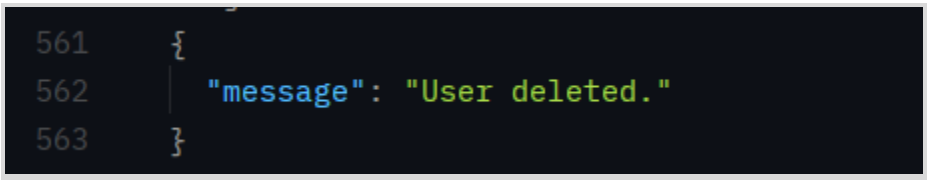


Figure 3.10: DELETE /api/users/{id} Sample Response

The endpoint which is available only to administrators deletes the user indicated by the {id} in the URL and requires no other parameters or data in the body. It sends back a response code of 200 when the operation is successful, 401 if the user is not logged in, 403 if the person is not an administrator, and 404 if the user does not exist. A 409 response is given if the user still has events or bookings, to ensure that no references are left pointing to a user who has been deleted. Similarly, a 409 response is returned if an administrator attempts to delete their own account.

3.5 Venues

3.5.1 List Venues: GET /api/venues


```
{
  "current_page": 1,
  "data": [
    {
      "id": 4,
      "name": "Dataran Tasik Open Stage",
      "address": "Presint 2",
      "city": "Putrajaya",
      "capacity": 3000,
      "created_at": "2026-10-11 01:41:31",
      "updated_at": "2026-10-11 01:41:31"
    }
  ],
  "first_page_url": "http://127.0.0.1:8000/api/venues?page=1",
  "from": 1,
  "last_page": 1,
  "last_page_url": "http://127.0.0.1:8000/api/venues?page=1",
  "links": [ "...page links..." ],
  "next_page_url": null,
  "path": "http://127.0.0.1:8000/api/venues",
  "per_page": 10,
  "prev_page_url": null,
  "to": 2,
  "total": 2
}
```


Figure 3.11: GET /api/venues Sample Response

This public endpoint provides a list of venues, which means that anybody can access it without having to log in. The list can be filtered using optional query parameters: the search option matches a name, an address or a city, the city parameter matches one city exactly, and min_capacity causes smaller venues to be excluded. The results can be ordered using sort (which can be id, name, city, capacity or created_at, with name being the default) and direction (which can be asc or desc). They are returned one page at a time by means of the page and per_page parameters (the number ranging from 1 to 100, with 10 as the default). A 200 status code is returned if the operation is successful and 422 is returned if the filter is invalid.

3.5.2 Get Venue: GET /api/venues/{id}

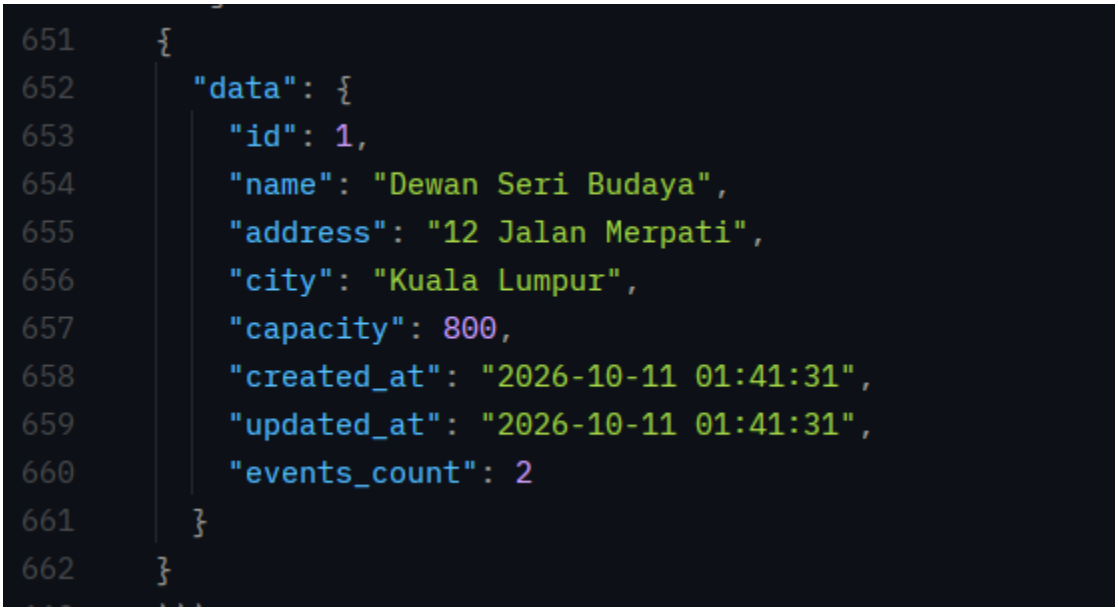


Figure 3.12: GET /api/venues/{id} Sample Response

The public endpoint gives back one venue, the one selected by the {id} in the URL, along with the number of events that have taken place at that venue. It does not require any other parameters or a body of data. If there is a venue with that id it returns 200 together with the venue, but if no venue has that id it returns 404.

3.5.3 Create Venue: POST /api/venues


```
{
  "message": "Venue created.",
  "data": {
    "name": "Postman Hall 1791654092940",
    "address": "1 Jalan Ujian",
    "city": "Shah Alam",
    "capacity": 300,
    "updated_at": "2026-10-11 01:41:37",
    "created_at": "2026-10-11 01:41:37",
    "id": 6
  }
}
```


Figure 3.13: POST /api/venues Sample Response

This endpoint is used to create a venue, but it may only be called by an admin or an organiser. The data submitted must include a name of no more than 150 characters which is unique within the city, an address, a city and a capacity that is at least 1. A response with status 201 is given if the operation is successful, 401 if the user is not logged in and 403 in all other cases. A 422 response is returned if the validation fails or if a venue with the same name already exists in that city.

3.5.4 Update Venue: PUT /api/venues/{id}

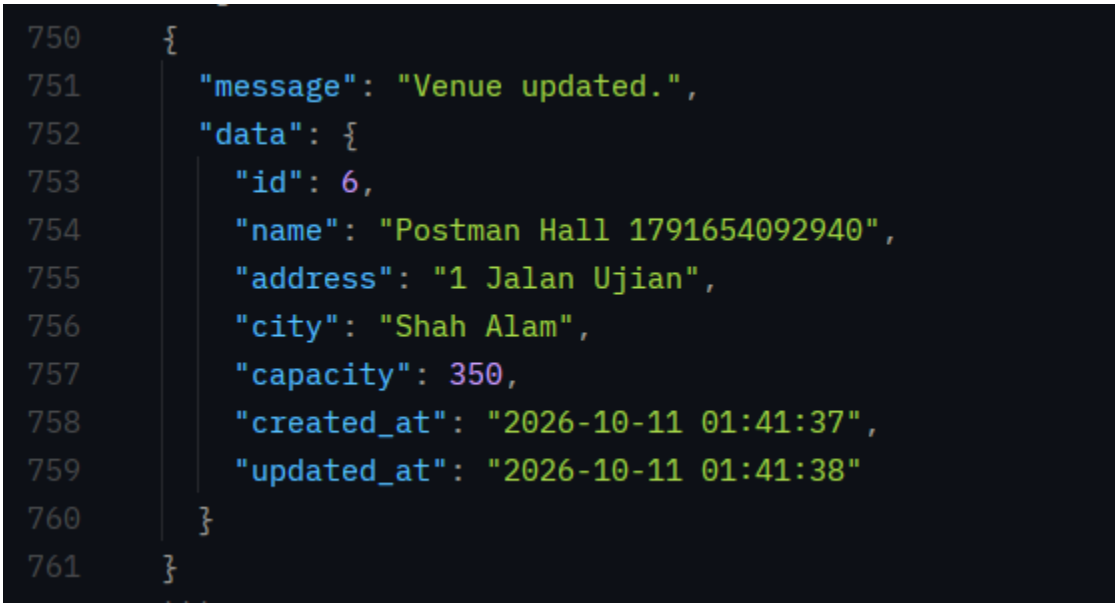


Figure 3.14: PUT /api/venues/{id} Sample Response

This endpoint is used for updating a venue, but it may only be accessed by an admin or an organiser. Only the fields which are included in the request will be altered, and the body may contain the name, address, city and capacity. A response with 200 is given if the operation is successful, 401 is returned if the user is not logged in, 403 is given for any other role, 404 is returned if the venue does not exist and 422 is sent if the validation fails. The endpoint returns a 409 if the new capacity is less than the total number of tickets for an active event at that venue, thus ensuring that an event can never have more tickets than the venue can hold.

3.5.5 Delete Venue: DELETE /api/venues/{id}



Figure 3.15: DELETE /api/venues/{id} Sample Response

The endpoint which is available only to administrators deletes the venue specified by the {id} in the URL and requires no other parameters or a body message; it sends back a response code of 200 when successful, 401 if the user is not logged in, 403 if the user is not an administrator, and 404 if the venue does not exist. It also returns a 409 code if the venue still has events, thus ensuring that no event is left without a venue.

3.6 Events

3.6.1 List Events: GET /api/events


```
{
  "current_page": 1,
  "data": [
    {
      "id": 3,
      "venue_id": 3,
      "organiser_id": 3,
      "title": "Teater Bangsawan Puteri Saadong",
      "description": "Teater Bangsawan Puteri Saadong at a venue near you. Sample event for testing.",
      "category": "Theatre",
      "starts_at": "2026-12-05 20:30:00",
      "ends_at": "2026-12-05 22:30:00",
      "ticket_price": "45.00",
      "total_tickets": 250,
      "available_tickets": 245,
      "status": "published",
      "created_at": "2026-10-11 01:41:31",
      "updated_at": "2026-10-11 01:41:31",
      "venue": {
        "id": 3,
        "name": "Pentas Teater Kecil",
        "city": "George Town"
      },
      "organiser": {
        "id": 3,
        "name": "Priya Nair"
      }
    }
  ],
  "first_page_url": "http://127.0.0.1:8000/api/events?page=1",
  "from": 1,
  "last_page": 1,
  "last_page_url": "http://127.0.0.1:8000/api/events?page=1",
  "links": [ "...page links..." ],
  "next_page_url": null,
  "path": "http://127.0.0.1:8000/api/events",
  "per_page": 5,
  "prev_page_url": null,
  "to": 3,
  "total": 3
}
```


Figure 3.16: GET /api/events Sample Response

The public endpoint provides a list of events; guests and customers are shown only those events which have been published, whereas staff, organisers and administrators can see events of all statuses. The list may be filtered by means of a search, by category, by venue, by date range and by the number of tickets remaining, and then sorted and paginated (with 10 items per page as the default). A response code of 200 is given upon success and 422 is returned when the filter is invalid.

3.6.2 Get Event: GET /api/events/{id}


```
{
  "data": {
    "id": 2,
    "venue_id": 2,
    "organiser_id": 2,
    "title": "Tech Career Fair 2026",
    "description": "Tech Career Fair 2026 at a venue near you. Sample event for testing.",
    "category": "Conference",
    "starts_at": "2026-11-21 09:00:00",
    "ends_at": "2026-11-21 17:00:00",
    "ticket_price": "0.00",
    "total_tickets": 1500,
    "available_tickets": 1499,
    "status": "published",
    "created_at": "2026-10-11 01:41:31",
    "updated_at": "2026-10-11 01:41:31",
    "venue": {
      "id": 2,
      "name": "Laman Kota Convention Hall",
      "address": "3 Persiaran Tasik",
      "city": "Shah Alam",
      "capacity": 1500,
      "created_at": "2026-10-11 01:41:31",
      "updated_at": "2026-10-11 01:41:31"
    },
    "organiser": {
      "id": 2,
      "name": "Daniel Lim"
    }
  }
}
```


Figure 3.17: GET /api/events/{id} Sample Response

The public endpoint provides one event, the event being selected by the {id} in the URL, together with the name of the venue and that of the organiser. It does not require any other parameters or a body and returns a 200 status code together with the event or a 404 status code if the event is not found. Even a guest or customer receives a 404 if the event is not published, so that draft and cancelled events remain hidden from them.

3.6.3 Create Event: POST /api/events


```
{
  "message": "Event created.",
  "data": {
    "venue_id": 6,
    "title": "Postman Launch Night",
    "description": "Created by the Postman collection.",
    "category": "Music",
    "starts_at": "2027-06-15 20:00:00",
    "ends_at": "2027-06-15 23:00:00",
    "ticket_price": "25.50",
    "total_tickets": 5,
    "organiser_id": 2,
    "available_tickets": 5,
    "status": "draft",
    "updated_at": "2026-10-11 01:41:38",
    "created_at": "2026-10-11 01:41:38",
    "id": 7,
    "venue": {
      "id": 6,
      "name": "Postman Hall 1791654092940",
      "city": "Shah Alam"
    },
    "organiser": {
      "id": 2,
      "name": "Daniel Lim"
    }
  }
}
```


Figure 3.18: POST /api/events Sample Response

This endpoint is used to create an event, but it may only be accessed by an admin or an organiser. The data sent must include information about the venue, the title, the category, the start and end times (both of which must be in the future with the end time occurring after the start time), a ticket price that is zero or higher, and a ticket total that does not exceed the venue's capacity. The description and the status are optional, the status being set at draft by default unless it is specified as published. All tickets are initially made available. The endpoint returns a 201 status code when it is successful, a 401 if the user is not logged in, a 403 for any other role, and a 422 if the validation fails. It also returns a 409 error code if there is already an active event at that venue which overlaps in time.

3.6.4 Update Event: PUT /api/events/{id}


```
{
  "message": "Event updated.",
  "data": {
    "id": 7,
    "venue_id": 6,
    "organiser_id": 2,
    "title": "Postman Launch Night",
    "description": "Created by the Postman collection.",
    "category": "Music",
    "starts_at": "2027-06-15 20:00:00",
    "ends_at": "2027-06-15 23:00:00",
    "ticket_price": "25.50",
    "total_tickets": 5,
    "available_tickets": 5,
    "status": "published",
    "created_at": "2026-10-11 01:41:38",
    "updated_at": "2026-10-11 01:41:39",
    "venue": {
      "id": 6,
      "name": "Postman Hall 1791654092940",
      "city": "Shah Alam"
    },
    "organiser": {
      "id": 2,
      "name": "Daniel Lim"
    }
  }
}
```


Figure 3.19: PUT /api/events/{id} Sample Response

The endpoint allows an administrator or the organiser of the event to update, publish or cancel an event, modifying only the fields that are provided. When an event is cancelled, its confirmed bookings are also cancelled and the tickets associated with them are returned, after which the event becomes locked. A response with status 200 is given if the operation is successful, 401 if the user is not logged in, 403 if the person making the request is not the owner, 404 if the event does not exist and 422 if validation fails. It returns a 409 error code if the event has already been cancelled, if the new times conflict with another event, or if the new total is less than the number of tickets that have already been booked.

3.6.5 Delete Event: DELETE /api/events/{id}

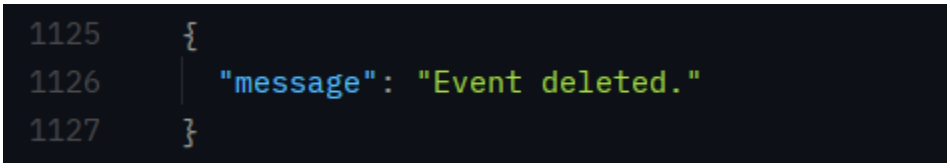


Figure 3.20: DELETE /api/events/{id}

The endpoint removes an event which has no bookings; only an administrator or the organiser of the event can use it, the only input being the event ID in the URL. It gives a response code of 200 when successful, 401 if the user is not logged in, 403 if the person making the request is not the owner, and 404 if the event does not exist. A code of 409 is returned if the event already has bookings, thus ensuring that booking records are never lost.

3.7 Bookings

3.7.1 List Bookings: GET /api/bookings


```
{
  "current_page": 1,
  "data": [
    {
      "id": 7,
      "user_id": 5,
      "event_id": 4,
      "booking_reference": "BK-W3F6DN9J",
      "quantity": 5,
      "total_price": "150.00",
      "status": "confirmed",
      "created_at": "2026-10-11 01:41:31",
      "updated_at": "2026-10-11 01:41:31",
      "event": {
        "id": 4,
        "venue_id": 4,
        "title": "Putrajaya Food and Music Festival",
        "starts_at": "2026-12-19 16:00:00",
        "ends_at": "2026-12-19 23:00:00",
        "ticket_price": "30.00",
        "status": "published",
        "venue": {
          "id": 4,
          "name": "Dataran Tasik Open Stage",
          "city": "Putrajaya"
        }
      },
      "user": {
        "id": 5,
        "name": "Siti Aminah",
        "email": "siti@eventhub.test"
      }
    }
  ],
  "first_page_url": "http://127.0.0.1:8000/api/bookings?page=1",
  "from": 1,
  "last_page": 2,
  "last_page_url": "http://127.0.0.1:8000/api/bookings?page=2",
  "links": [ "...page links..." ],
  "next_page_url": "http://127.0.0.1:8000/api/bookings?page=2",
  "path": "http://127.0.0.1:8000/api/bookings",
  "per_page": 5,
  "prev_page_url": null,
  "to": 5,
  "total": 6
}
```


Figure 3.21: GET /api/bookings Sample Response

The endpoint removes an event which has no bookings; only an administrator or the organiser of the event can use it, the only input being the event ID in the URL. It gives a response code of 200 when successful, 401 if the user is not logged in, 403 if the person making the request is not the owner, and 404 if the event does not exist. A code of 409 is returned if the event already has bookings, thus ensuring that booking records are never lost.

3.7.2 Create Booking: POST /api/bookings


```
{
  "message": "Booking confirmed.",
  "data": {
    "event_id": 7,
    "quantity": 2,
    "user_id": 5,
    "booking_reference": "BK-XNPZSRA7",
    "total_price": "51.00",
    "status": "confirmed",
    "updated_at": "2026-10-11 01:41:39",
    "created_at": "2026-10-11 01:41:39",
    "id": 9,
    "event": {
      "id": 7,
      "venue_id": 6,
      "title": "Postman Launch Night",
      "starts_at": "2027-06-15 20:00:00",
      "ends_at": "2027-06-15 23:00:00",
      "ticket_price": "25.50",
      "status": "published",
      "venue": {
        "id": 6,
        "name": "Postman Hall 1791654092940",
        "city": "Shah Alam"
      }
    },
    "user": {
      "id": 5,
      "name": "Siti Aminah",
      "email": "siti@eventhub.test"
    }
  }
}
```


Figure 3.22: POST /api/bookings Sample Response

The endpoint available only to customers is used for booking tickets for an event that has been published. The data sent must include the event_id of an event that already exists and a quantity ranging from 1 to 10. The server calculates the total_price by multiplying the quantity by the price of the ticket and generates the booking reference on its own. It sends a response with status 201 when the booking is confirmed, gives a 401 status if the user is not logged in, returns 403 if the person making the request is not a customer, and provides a 422 status if the validation fails. It returns a 409 status if the event is not open for booking, has already started, or does not have enough tickets remaining.

3.7.3 Get Booking: GET /api/bookings/{id}


```
{
  "data": {
    "id": 9,
    "user_id": 5,
    "event_id": 7,
    "booking_reference": "BK-XNPZSRA7",
    "quantity": 2,
    "total_price": "51.00",
    "status": "confirmed",
    "created_at": "2026-10-11 01:41:39",
    "updated_at": "2026-10-11 01:41:39",
    "event": {
      "id": 7,
      "venue_id": 6,
      "title": "Postman Launch Night",
      "starts_at": "2027-06-15 20:00:00",
      "ends_at": "2027-06-15 23:00:00",
      "ticket_price": "25.50",
      "status": "published",
      "venue": {
        "id": 6,
        "name": "Postman Hall 1791654092940",
        "city": "Shah Alam"
      }
    },
    "user": {
      "id": 5,
      "name": "Siti Aminah",
      "email": "siti@eventhub.test"
    }
  }
}
```


Figure 3.23: Get /api/bookings/{id} Sample Response

The endpoint provides a single booking along with the event, venue, and customer associated with it. A logged-in user who has the right to view that booking can use this endpoint, the only input required being the booking ID in the URL. It sends back a response code 200 on success, 401 if the user is not logged in, 403 if the caller is not authorized to see the booking, and 404 if the booking does not exist. An example of the response is given below.

3.7.4 Update Booking: PUT /api/bookings/{id}


```
{
  "message": "Customer checked in.",
  "data": {
    "id": 10,
    "user_id": 6,
    "event_id": 7,
    "booking_reference": "BK-ZUEM0RUY",
    "quantity": 1,
    "total_price": "25.50",
    "status": "checked_in",
    "created_at": "2026-10-11 01:41:39",
    "updated_at": "2026-10-11 01:41:40",
    "event": {
      "id": 7,
      "venue_id": 6,
      "title": "Postman Launch Night",
      "starts_at": "2027-06-15 20:00:00",
      "ends_at": "2027-06-15 23:00:00",
      "ticket_price": "25.50",
      "status": "published",
      "venue": {
        "id": 6,
        "name": "Postman Hall 1791654092940",
        "city": "Shah Alam"
      }
    },
    "user": {
      "id": 6,
      "name": "Jason Tan",
      "email": "jason@eventhub.test"
    }
  }
}
```


Figure 3.24: PUT /api/bookings/{id} Sample Response

The endpoint allows a booking to have its status changed to cancelled or checked_in. Both staff members, administrators and the event's organiser can carry out this action, but a customer can cancel only their own booking prior to the event taking place, and in such a case the tickets are returned. A response with 200 is given when the operation is successful, 401 is returned if the user is not logged in, 403 is returned if they are not authorised (this includes customers attempting to check in), 404 is given if the booking does not exist and 422 is returned when an invalid status is provided. It also returns a 409 error if the booking has already been cancelled or checked in, or if the event has already started. An example of the response is given below.

3.7.5 Delete Booking: DELETE /api/bookings/{id}

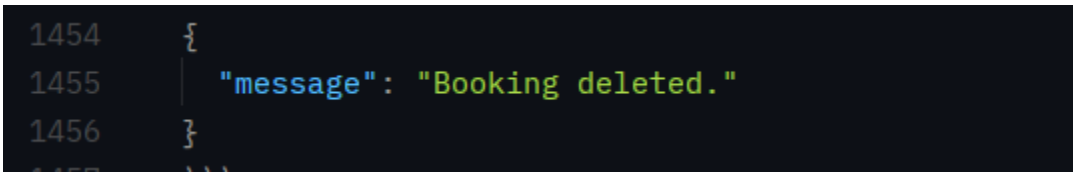


Figure 3.25: DELETE /api/bookings/{id} Sample Response

The endpoint which is available only to administrators is used for deleting a cancelled booking, the only input being the booking ID contained in the URL. It sends back a response code of 200 when the operation is successful, 401 if the user is not logged in, 403 if the person making the request is not an admin, and 404 if the booking does not exist. It gives a 409 response if the booking has not been cancelled, thus preventing accidental deletion of active bookings. An example of the response is given below.

3.7.6 Get Booking QR Code: GET /api/bookings/{id}/qr


```
{
  "data": {
    "booking_id": 9,
    "booking_reference": "BK-XNPZSRA7",
    "status": "confirmed",
    "qr_code": " ... (shortened)",
    "provider": "goQR.me (api.qrserver.com)",
    "from_cache": false
  }
}
```


Figure 3.26: GET /api/bookings/{id}/qr Sample Response

The endpoint provides a booking's QR code and allows any logged-in user who has access to the booking to access it by including the booking ID in the URL as the only required input. The backend retrieves the code from the goQR.me web API, the code containing the booking reference. The qr_code field is given as a PNG in base64 format as a data URI so that it can be used directly in an <img> tag. The image is cached for one day and the from_cache field indicates whether the external API was used this time. It sends back a 200 status code on success, a 401 if the user is not logged in, a 403 if the caller does not have access to the booking, a 404 if the booking does not exist and a 502 if the QR code service does not respond. An example of the response is given below.

3.8 Tickets

3.8.1 Verify Ticket: GET /api/tickets/{reference}


```
{
  "data": {
    "valid": true,
    "booking_reference": "BK-B9V3LK7E",
    "status": "confirmed",
    "quantity": 1,
    "holder": "Siti Aminah",
    "event": "Tech Career Fair 2026",
    "venue": "Laman Kota Convention Hall",
    "starts_at": "2026-11-21 09:00:00"
  }
}
```


Figure 3.27: GET /api/tickets/{reference} Sample Response

The endpoint allows a booking reference to be checked, for instance one obtained from a scanned QR code, and it is intended for use by external systems such as a door scanner. Rather than requiring a user login, it demands an API key in the X-API-Key header, the reference being provided in the URL as the sole input. It sends back a 200 status code when the ticket is located, but only sets valid to true in the case of a confirmed booking. If the API key is missing or incorrect, it returns a 401, and if there is no booking with that reference, it returns a 404. An example of a response is given below.

4.0 API Testing

The endpoint tests were carried out using Postman, with both successful requests and error cases being included. The test collection contains 96 requests distributed among seven folders, one folder for each section of the API together with a separate folder for general errors, and each request includes a test script which verifies the status code and the key fields in the response. When run as a whole on a database that has just been seeded, it successfully passes all 134 checks with no failures. It was just as important to test the failure cases as it was to test the successful ones, since a client needs the API to reject bad input, missing logins and the wrong role, giving back the appropriate status code and a clear message. This section first explains the collection and how to run it, then provides screenshots of some typical requests.

4.1 The Postman Collection

The postman/EventHub.postman_collection.json file is where the collection is stored in the repository, and each request includes a test script which verifies the status code and the important fields in the response.

The entire collection can be run from the command line using Newman, as indicated in the README. When carried out on a database that has just been seeded, the run results in 134 checks being passed and 0 failed.

	executed	failed
iterations	1	0
requests	101	0
test-scripts	96	0
prerequest-scripts	2	0
assertions	134	0
total run duration: 6.3s		
total data received: 39.2kB (approx)		
average response time: 52ms [min: 7ms, max: 879ms, s.d.: 108ms]		

Figure 4.1: Newman Collection

4.2 Representative Requests

The screenshots shown below were obtained using Postman on 11 when the database had just been seeded, meaning that each test started with the same initial data. From the 96 requests in the collection, 14 were selected so that every aspect of the API was included at least once, ranging from logging in and managing users to creating events, booking tickets, obtaining a QR code and verifying a ticket with an API key. These requests include both successful ones and the main errors that a client is likely to encounter: 401, 403, 404, 409, 422 and 429. Each screenshot displays the request, the status code and the JSON body returned, thus allowing the reader to see that errors follow the same response format as successful requests.

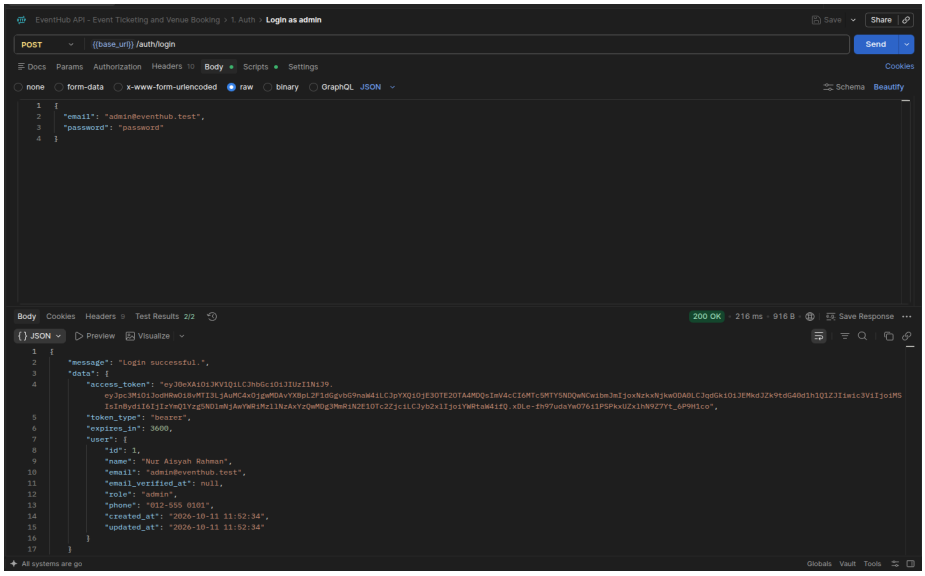


Figure 4.2: Successful Login

The request includes the seeded admin's email and password in JSON format and is directed to the POST /auth/login endpoint, after which the API responds with 200 OK within 216 ms. The response adheres to the API's standard format consisting of a message and a data object; in the data object there is the JWT access_token, the token_type (which is bearer), the expires_in value (3600 seconds, meaning the token is valid for one hour), together with the details of the logged-in user, among which is the role admin. The password is not returned. The Test Results tab displays 2/2, indicating that both of the checks in the test script for this request have passed

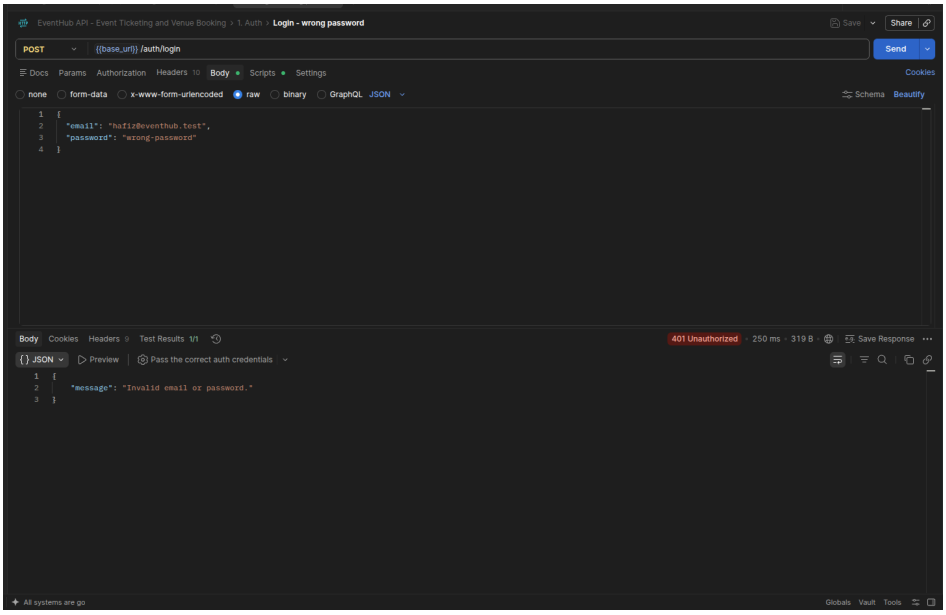


Figure 4.3: Login with Wrong Password

In this case, the email of a seeded user is submitted together with the incorrect password, and the API responds with an error code 401 Unauthorized and the message "Invalid email or password". The message is deliberately silent as to whether the problem lies with the email or the password, so that an attacker cannot use the login form to determine which emails are registered. No token is returned and the one check in the test script passes (1/1).

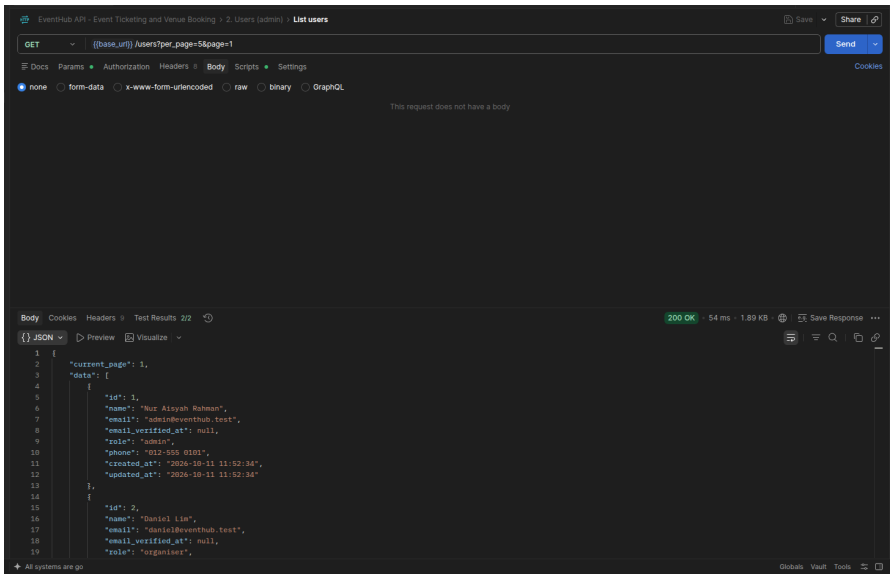


Figure 4.4: Admin Lists Users

The admin makes a GET request to /users?per_page=5&page=1 without including a request body, and the API responds with 200 OK after 54 ms. The response is a paginated list: the current_page field indicates which page is being displayed and the data field contains the user records, with the admin coming first followed by the organiser. The per_page and page query parameters enable the client to retrieve the list in small portions rather than all at once. Both of the checks in the test script are passed (2/2).

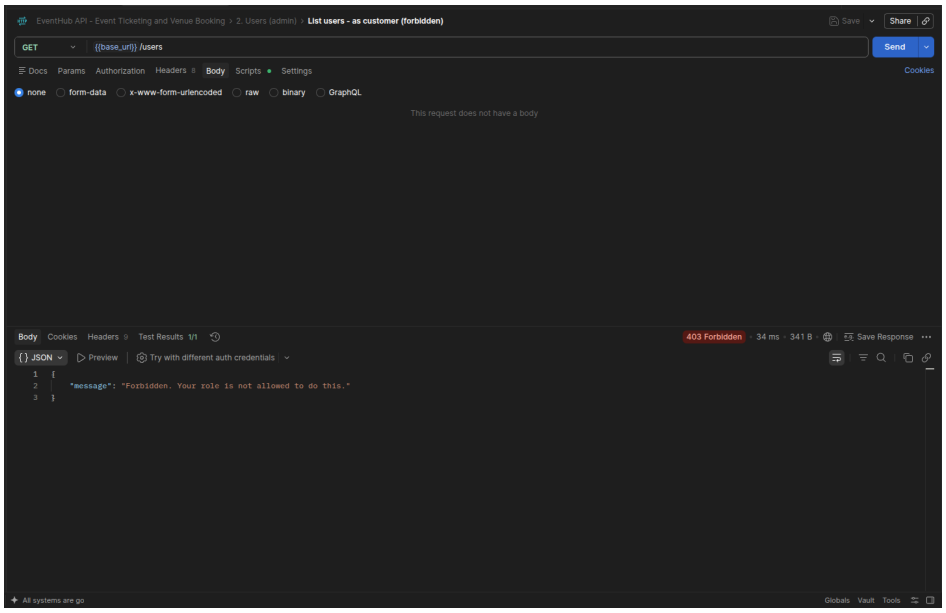


Figure 4.5: List Users as Customer (Forbidden)

It's the identical GET /users request, but this time it's made using the customer's token. Since the customer is logged in, authentication succeeds, but the role check denies the request with a 403 Forbidden response and the message "Forbidden. Your role is not allowed to do this." This illustrates the difference between 401 (who are you?) and 403 (you are known, but not allowed).

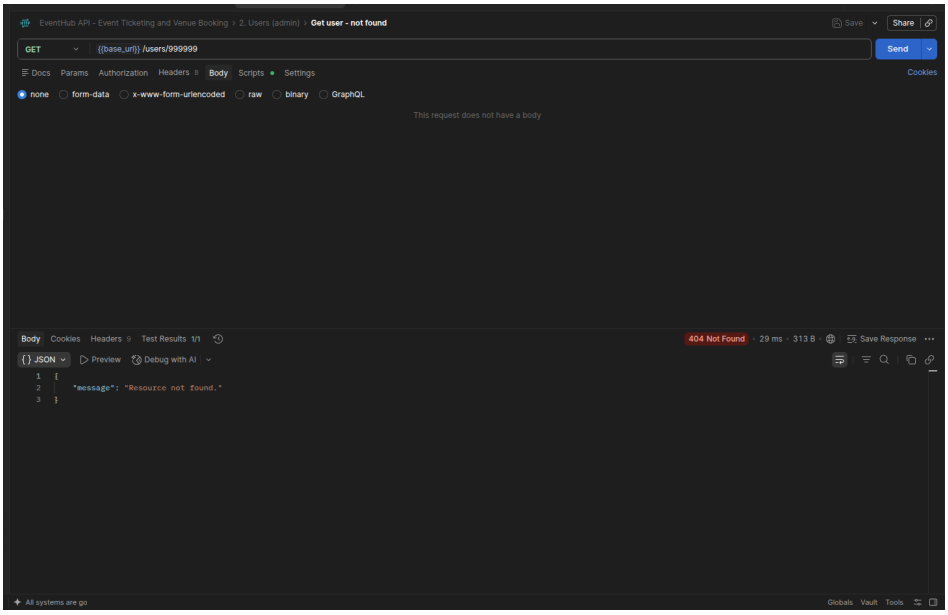


Figure 4.6: Get User - Not Found

The administrator requests GET /users/999999, which is an ID that does not exist; rather than displaying an HTML error page or a stack trace, the main error handler sends back a 404 Not Found response together with a brief JSON message stating "Resource not found"; each missing record in the API—whether it be a user, a venue, an event or a booking—is dealt with in the same manner.

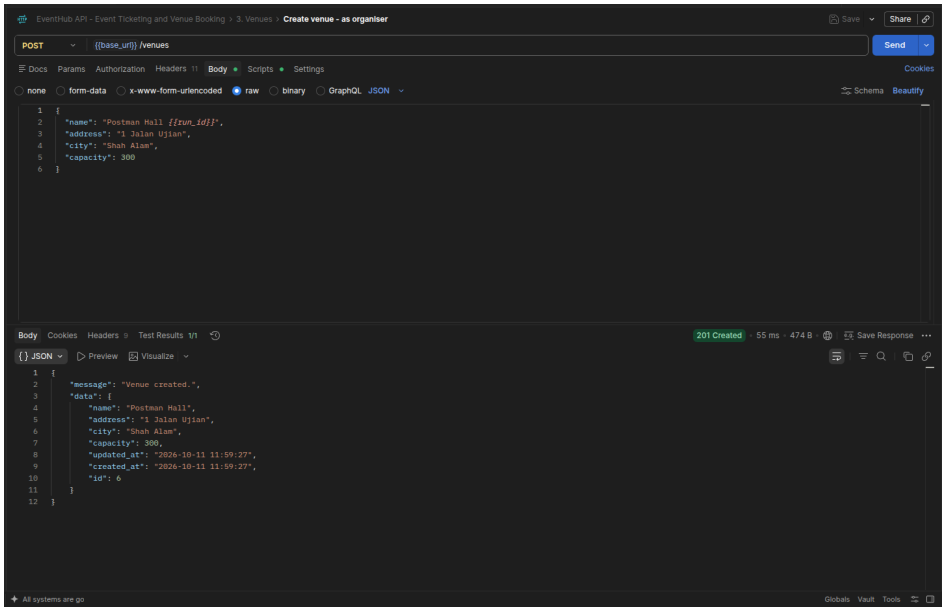


Figure 4.7 - Create Venue as Organizer

The organiser sends the name, address, city and capacity in JSON format to the POST /venues endpoint. The API responds with a status code 201 Created, which is the appropriate code for a new resource, and returns the saved venue with its new id and the created_at and updated_at times filled in by the server.

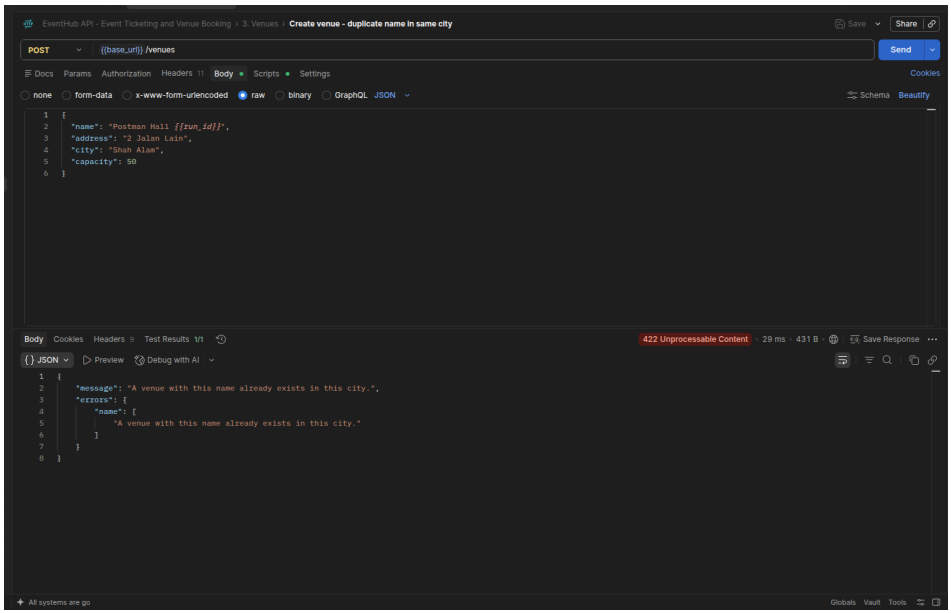


Figure 4.8: Create Venue - Duplicate Name in Same City

The request is attempting to set up a second venue in Shah Alam with the same name, but the API rejects it with a 422 Unprocessable Content response. In addition to the main message, the response includes an errors object with the field name as the key, which allows a frontend to display the error directly below the input that caused it.

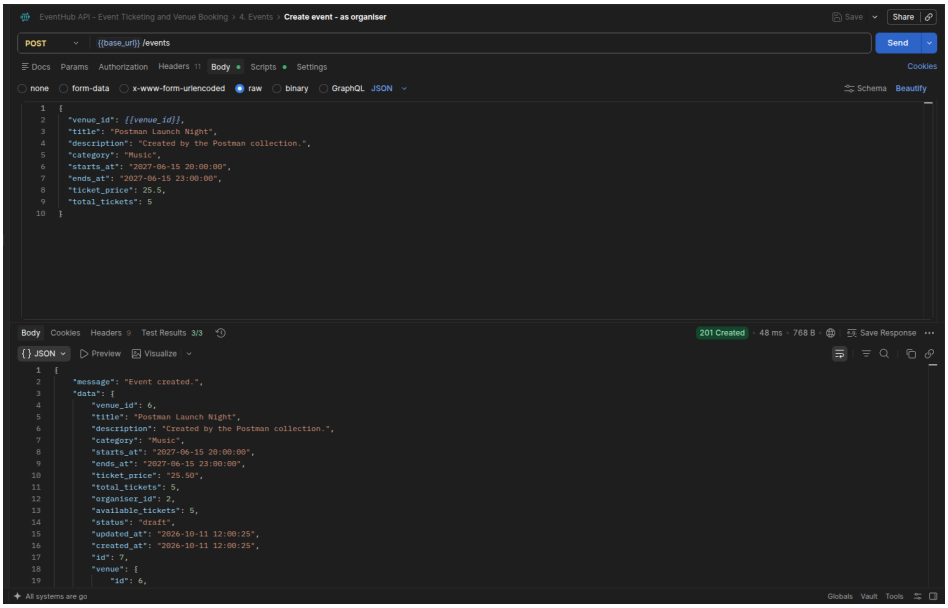


Figure 4.9: Create Event as Organizer

The organiser sets up an event at the new venue with a ticket price of 25.5 and 5 tickets; the API responds with a status of 201 Created and fills in a number of fields by itself, the organiser_id being taken from the logged-in user, the available_tickets starting at the same value as the total_tickets, and the status remaining draft since no invitation has been sent. Together with the related venue, the response includes all three checks, which therefore all pass (3/3).

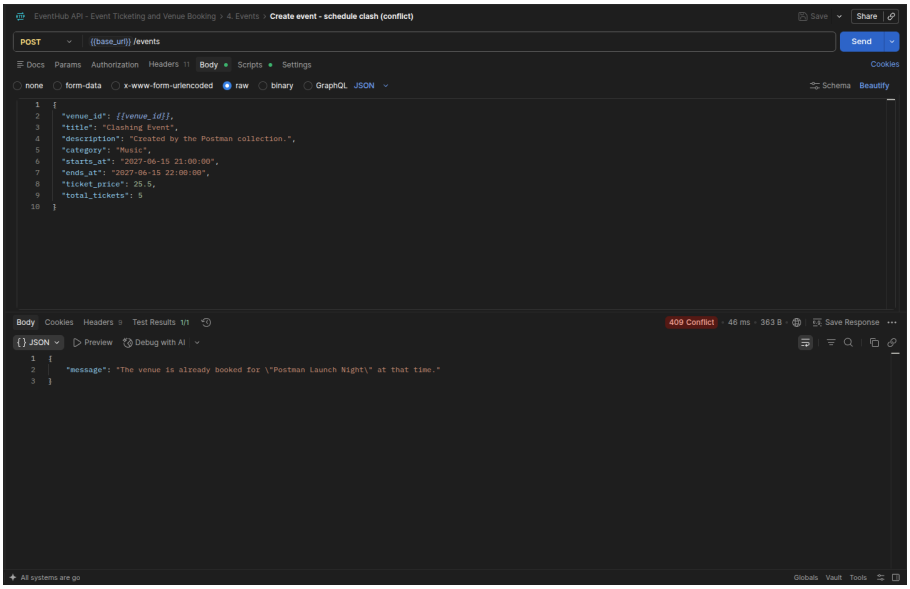


Figure 4.10: Create Event - Schedule Clash (Conflict)

The event will take place from 21:00 to 22:00 on the same day at the same venue and thus lies within the 20:00 to 23:00 time slot of the event. The request is valid by itself, so it is not a 422; the API sends back a 409 Conflict since it overlaps with data that is already stored. The message identifies the event which occupies that time slot, thereby informing the organiser precisely what needs to be altered.

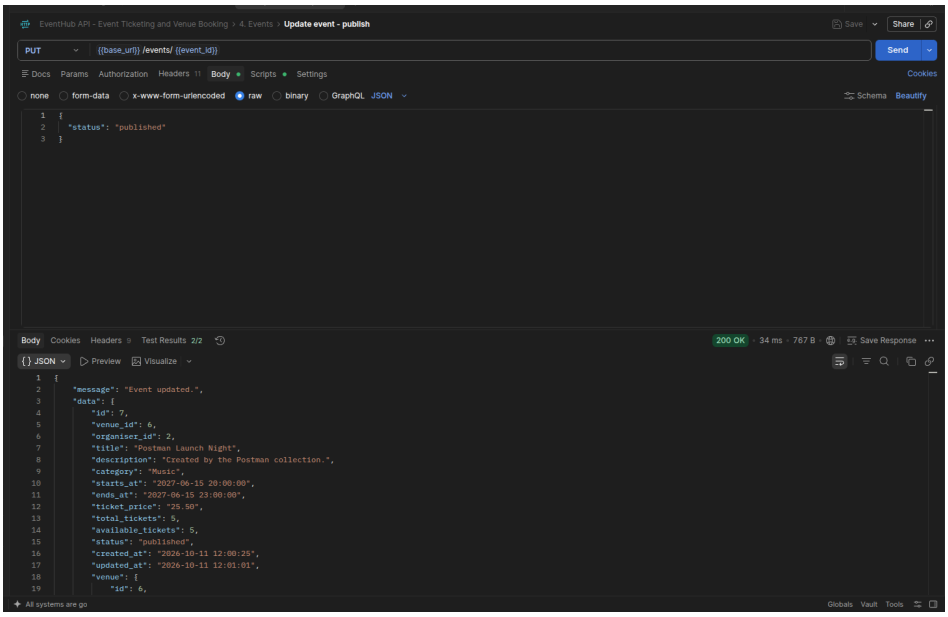


Figure 4.11: Update Event - Publish

The organiser sends a PUT request to /events/{id} containing only the following in the body: {"status": "published"}. Just this field is altered; all the other fields retain their current values, with updated_at being updated while created_at remains unchanged. After it has been published the event shows up in the public listings and customers are able to book it.

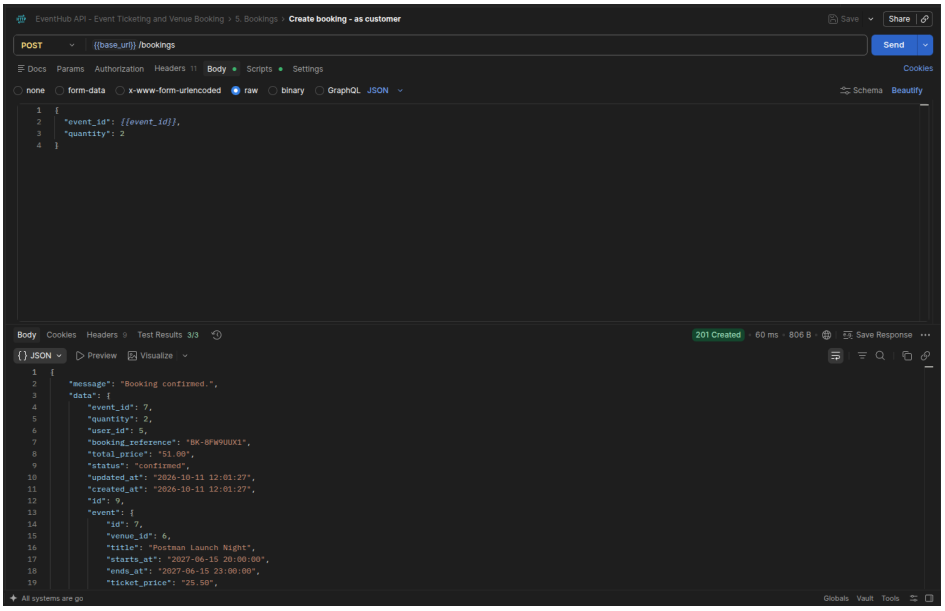


Figure 4.12: Create Booking as Customer

The only information that the customer sends is the event_id together with a quantity of 2; the server is responsible for the rest: it obtains the user_id from the token, calculates the total_price using the stored ticket price, generates the booking_reference, and sets the status to confirm. Since the client does not send a price, it cannot alter the amount it pays. All three of the checks are successful (3/3).

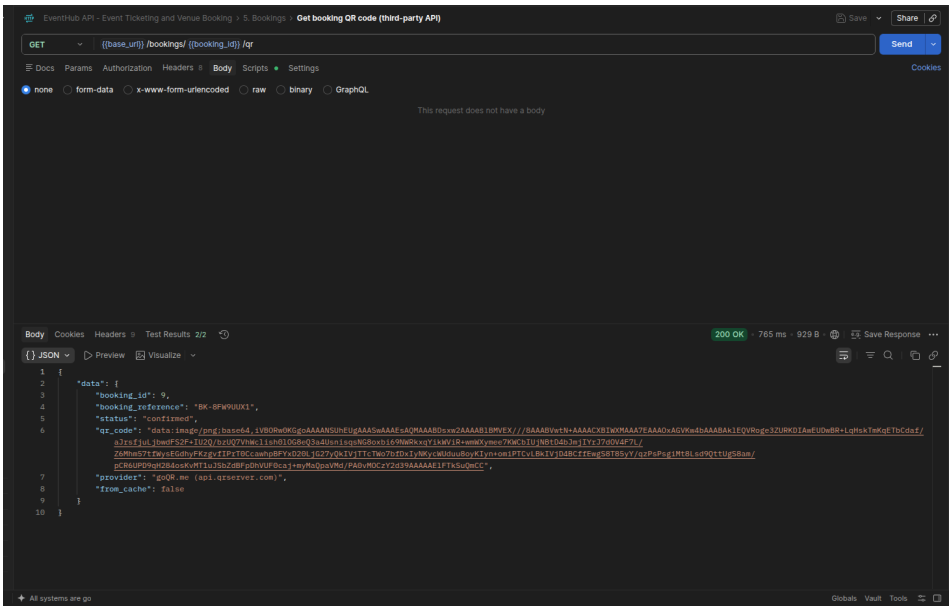


Figure 4.13: Get Booking QR code (Third-Party API)

The GET request for /bookings/{id}/qr provides the booking's ID, reference and status, the QR image in the form of a base64-encoded PNG within the field qr_code, and the provider who created it. Since this was the first request for the booking, the backend had to get in touch with goQR.me, which is the reason why the request took 765 ms rather than the under 100 ms seen with most other requests. If the request is made again within a day it is served from the cache and does not require another contact with goQR.me.

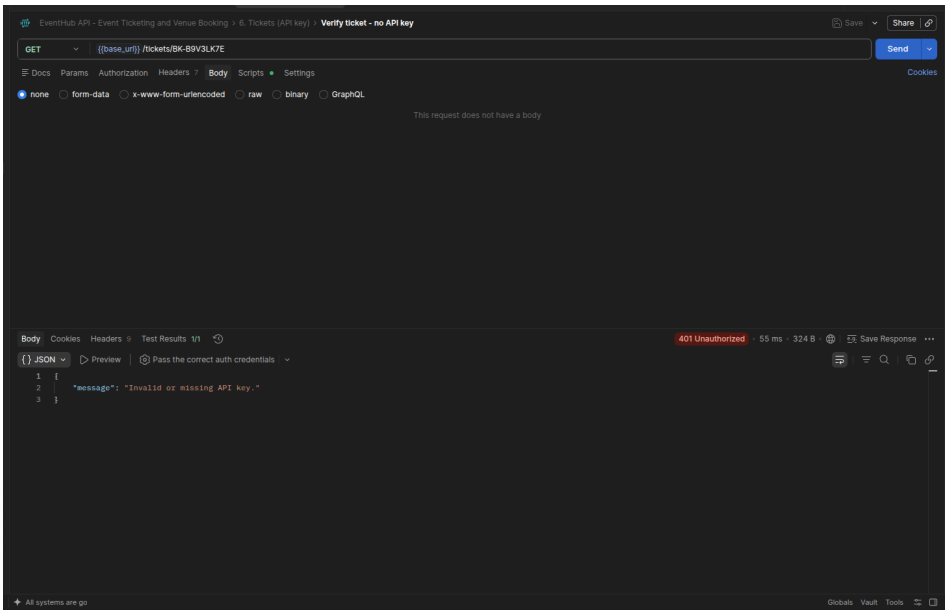


Figure 4.14: Verify Ticket - No API Key

The request GET /tickets/{reference} is intended for use by machines such as a door scanner and therefore does not require a user to log in. If it is sent without including the X-API-Key header, it will be refused with an error 401 Unauthorized and the message "Invalid or missing API key". When the same request is made with the correct key, however, it returns the ticket details together with information on whether the ticket is valid.

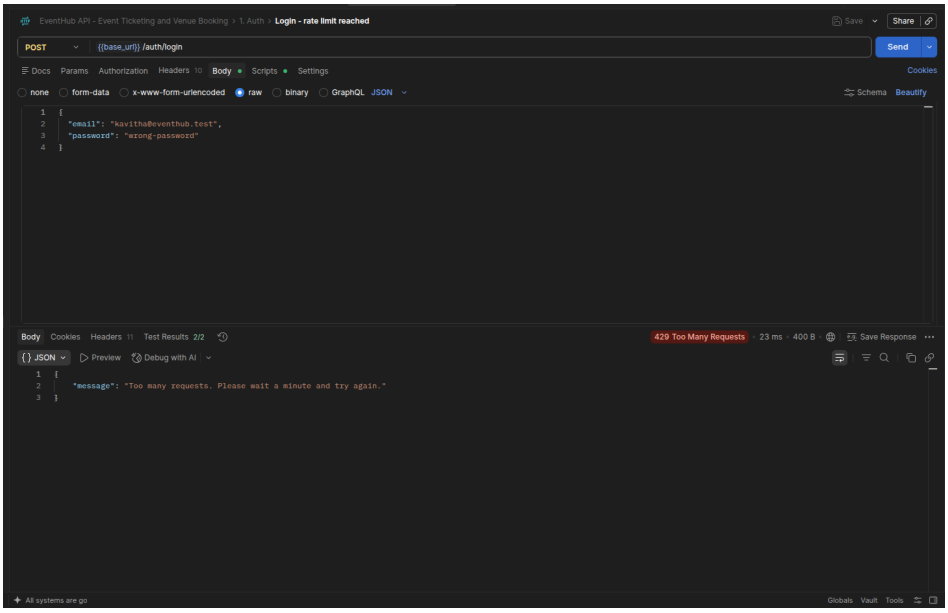


Figure 4.15: Login - Rate Limit Reached

The collection sends wrong passwords for the same email several times in a row. Once the limit is passed, the API stops checking the password at all and answers 429 Too Many Requests with "Too many requests. Please wait a minute and try again." This slows down anyone trying to guess a password, and both checks pass (2/2).

5.0 System Implementation

The setup consists of a single shared Laravel backend and a separate browser frontend for each group member, the two being connected solely via HTTP and JSON. The backend is responsible for all the data and all the rules: it handles login checks and role verification, validates the input, applies the business rules, interacts with the database and the QR code service. The frontends have no significant rules of their own concerning safety; their role is simply to make requests and display the responses that are returned, so even if a frontend has a bug it cannot book an event that is already sold out or change a price. Because the backend and frontend are kept separate in this way, each member is able to develop their frontend independently while using the same API. Section 5.1 explains the organisation of the backend, the path that a request takes through it and the business rules it enforces. Section 5.2 deals with each member's frontend and the endpoints it uses, and Section 5.3 illustrates how a frontend actually calls the API.

5.1 Shared Backend

The backend is located in the backend/ directory and adheres to Laravel's standard structure, which means that anyone familiar with Laravel should be able to navigate it easily. Each component has a single responsibility: the routes determine which piece of code will deal with a request, the middleware carries out the checks that every request to a route must go through, the controllers contain the logic for each resource, and the models manage the database. Settings that are cross-cutting in nature, such as the error handler, the rate limits and the API key, are each configured once in their own file rather than being repeated in every controller.

In most cases a 409 Conflict code is returned since the request is valid but conflicts with existing data, for example because an event is sold out or a venue is already booked. When the rules concern who is permitted to take an action, a 403 is returned; the rule regarding capacity gives a 422 response since it forms part of the input validation process, and the rules which are marked 'Not applicable' cannot be broken because the server carries out that task itself. The booking code listed below illustrates these rules in action: it verifies that the event is published, has not yet started and has sufficient tickets before saving the booking, sends out a clear 409 message if these conditions are not met, and operates within a database transaction with a row lock so that two customers cannot both book the last tickets.

```
69          // A transaction with a row lock: two customers booking the last
70          // tickets at the same moment cannot both succeed (no overselling).
71          $result = DB::transaction(function () use ($data, $request) {
72              $event = Event::lockForUpdate()->findOrFail($data['event_id']);
73
74              // Business rule: only published events can be booked.
75              if ($event->status !== 'published') {
76                  return 'This event is not open for booking.';
77              }
78
79              // Business rule: no bookings once the event has started.
80              if ($event->starts_at->isPast()) {
81                  return 'This event has already started.';
82              }
```

```
91              $booking = new Booking($data);
92              $booking->user_id = $request->user()->id;
93              $booking->booking_reference = $this->newReference();
94              $booking->total_price = $event->ticket_price * $data['quantity'];
95              $booking->status = 'confirmed';
96              $booking->save();
97
98              $event->decrement('available_tickets', $data['quantity']);
99
100             return $booking;
101         });
102
103         if (is_string($result)) {
104             return response()->json(['message' => $result], 409);
105         }
```

Figure 5.1: BookingController.php Code

5.2 Frontends

Every member has a frontend that is contained in its own folder and consists of its own code. All of these frontends make use of the same API. None of them access the database directly or share any code with one another; each one simply sends HTTP requests to the backend and then displays the JSON that it receives. It is because of this that four people are able to create four different interfaces using a single backend: so long as each frontend sends the correct requests together with the correct token, the backend handles them all in the same way and applies the same rules to each. The sections that follow explain the frontend of each member, the pages it contains, who is allowed to open them and the endpoints that each page uses. **ISHAK BOTAK QIUTi**

5.2.1 Member 1: Ishak Bin Ismail (frontend-ishak/)

The frontend is a single-page application encompassing all four roles. Once the user has logged in, the navigation bar displays only the pages that correspond to the pages the user's role is authorized to access. The application is developed using HTML, CSS, JavaScript modules and Bootstrap 5.3 (Bootstrap, n.d.), and does not involve any build process. It consists of a single index.html file, and when navigating between pages, only the section that comes after the # in the URL changes. A small router located in js/app.js monitors this change and then retrieves the appropriate view from the js/views/ directory, thus ensuring that the page is never reloaded. All requests made to the backend are directed through one file, js/api.js, and the token together with the user details obtained during login are stored in localStorage by js/session.js, which means the user remains logged in after the page is refreshed. Since there is no build step, the folder can be served by any basic web server and opened directly in the browser. The table below gives a list of each page, who is allowed to access it and the endpoints it uses.

Page	Access	Endpoints
Events	Everyone	GET /events
Event details	Everyone	GET /events/{id}, POST /bookings
Log in, Create account	Guests	POST /auth/login, POST /auth/register
Bookings	Logged-in users	GET, PUT, DELETE /bookings
Ticket and QR code	Logged-in users	GET /bookings/{id}, GET /bookings/{id}/qr, PUT /bookings/{id}
Manage, new and edit events	Admin, organizers	GET /venues, GET /users, GET, POST, PUT, DELETE /events
Venues	Admin, organizers	GET, POST, PUT, DELETE /venues
Users	Admin	GET, POST, PUT, DELETE /users

When the app starts it makes a GET request to /auth/me and when the user logs out it sends a POST request to /auth/logout. If a token has been saved from a previous visit, the request to /auth/me asks the API which user it belongs to and updates the user details that are stored; if the token has expired the API responds with a 401 and js/api.js then empties the session, in which case the user is displayed as being logged out. On logging out the app requests the API to blacklist the token so that it can no longer be used, and then clears the session on this device even if that request fails. The table below shows how the code is divided among the files and what function each one carries out.

File	Purpose
index.html	The page shell which consists of navigation bar, content area, message area and one shared pop-up
js/api.js	Sends every request, adds the token, reads the JSON and turns failures into one error type
js/session.js	Keeps the token and the user in localstorage
js/router.js, js/app.js	Match the address after # to a page, check the role and draw the navigation bar
js/ui.js	Shared helpers that consists of safe HTML, dates and prices, messages, pagination and form errors
js/views/*.js	One file per page

5.2.2 Member 2: Ahmad Naufal bin Ahmad Fuad

This front-end is a multi-page app including all four roles and guests. All features are not in one page, but each feature has its own HTML file and its own script, and when you click on one feature from the other, a new page appears in the browser. It is completely written in HTML, CSS, standard JavaScript and Bootstrap 5.3 (Bootstrap, n.d.), which is loaded from a CDN, and there is no build process required. The folder is served with PHP's built-in web server (php -S 127.0.0.1:5501 -t frontend-naufal) and talks to the backend at http://127.0.0.1:8000/api.

- 422: validation messages are shown below the corresponding input field.
- 409: business-rule conflicts, such as a sold-out event, a schedule clash or an event that still has bookings, appear as alerts.
- 403: displayed as a friendly permission message.
- 429: asks the user to wait before trying to log in again.
- 502: Service Unavailable.502: Service Unavailable (QR code service not available).
- 0: Indicate that API is unreachable.

After logging in, the token and user information are stored in localStorage in js/auth.js, allowing users to remain logged in after a page refresh. Once the user logs in, they are redirected to a home page relevant to their role, such as bookings for customers and staff; and event management for organisers and admins. The navigation bar is constructed based on the role stored, which means that only those pages that can be used by the role appear in the navigation bar to the user. A requireRole() call is made on every protected page prior to loading any data. The user is directed to the login screen, and those with an incorrect role are brought back to the event list with a warning. The rules are still the same in the backend and the backend still returns 403 for all requests that are not allowed, so the frontend check is only for usability.

The front-end doesn't request /auth/me on page loads. Rather than that, it uses the user that is stored, and if the API returns a 401, the session is terminated and

the user is returned to the login page with a “session expired” message. When it logs out it sends a post /auth/logout so that the key gets removed from the blacklist, but it also clears the local session even if this fails.

Multiple pages are shared between roles and not duplicated. The bookings page is for customers, staff, organisers and admins. The API already provides only the bookings which each role can access, and the page only updates its title and action buttons – customers can cancel and view QR tickets, staff and organisers can check customers in, and the admins can delete cancelled bookings. Customers can see the third-party API integration here. The QR code is loaded automatically next to each active booking – the backend already gets the QR code from the goQR.me API – and a ticket card with the QR code is generated right after the booking is made. All text that is returned by the API is escaped before being inserted into the page, to prevent HTML injection, and the X-API-Key sent by the ticket verification endpoint is not present in the browser.

The table below lists each page, who can open it and the endpoints it uses.

Page	Who can open	Endpoints
index.html	Everyone	GET /events (search, category, sort, pagination)
event.html	Everyone; booking form for customers	GET /events/{id}, POST /bookings, GET /bookings/{id}/qr
login.html	Guests	POST /auth/login
register.html	Guests	POST /auth/register
bookings,html	Customer, staff, organiser, admin	GET /bookings, PUT /bookings/{id}, DELETE /bookings/{id}, GET /bookings/{id}/qr
manage-event.html	Organiser, admin	GET /events, PUT /events/{id}, DELETE /events/{id}
event-form.html	Organiser, admin	GET /venues, GET /events/{id}, POST /events, PUT /events/{id}
venues.html	Organiser, admin	GET /venues, POST /venues, PUT /venues/{id}, DELETE /venues/{id} (admin only)
users.html	Admin	GET /users, POST /users, PUT /users/{id}, DELETE /users/{id}

The table below shows how the code is divided among the files.

File	Purpose
js/api.js	apiRequest(): sends every request, adds the JWT, checks response.ok, converts errors into one format and handles expired tokens
js/auth.js	Stores and clears the session, protects pages with requireRole(), builds the role-based navigation bar and footer, logs out
js/ui.js	Shared helpers: alerts, 422 field errors, loading buttons, HTML escaping, date and price formatting, pagination buttons
js/login.js, js/register.js	Login and customer registration forms
js/index.js, js/event.js	Public event list, event details, customer booking and QR ticket
js/bookings.js	Shared bookings page with role-dependent actions and automatic QR codes
js/manage-events.js, js/event-form.js	Event management and the create/edit form
js/venues.js, js/users.js	Venue and user management using pop-up forms
css/style.css	Custom theme layered on top of Bootstrap

5.2.3 Member 3: Muhammad Anas Bin Haddri

My frontend is a multi-page website. It works for guests and for all four roles. Each feature has its own HTML page and its own JavaScript file. When the user clicks to another feature, the browser opens a new page. I used plain HTML, CSS and JavaScript, with Bootstrap 5.3.3 from a CDN (Bootstrap, n.d.). There is no framework and nothing to build. I run the folder with PHP's built-in server (php -S 127.0.0.1:5501 -t frontend-anas). It sends all requests to the shared backend at http://127.0.0.1:8000/api.

The design is my own. I used a dark green colour for the navbar, buttons and login card. The colours are saved once as CSS variables in css/style.css. On the home page, an event card moves up a little when the mouse is over it, and the user can click anywhere on the card. My name and student ID are shown on every page, under the navbar.

Roles and navigation. After login, the token and the user details (including the role) are saved in localStorage. Because of this, the user stays logged in after a page refresh. js/nav.js builds the navbar from a list of links for each role, so users only see the pages they can use. Every protected page calls requireRole() before it loads any data:

• A guest is sent to the login page.

• A user with the wrong role sees a yellow box: "Access denied (403). Your role (customer) is not allowed to open this page.", with a link back to the events.

This check only makes the website easier to use. The backend still checks every request and returns 403 if the role is wrong.

One page for many roles. I did not make a separate page for each role. Instead, some pages change based on who is logged in. The bookings page is the best example. The API already decides which bookings each role can see, so the page only changes its title and buttons:

• Customer: the title is "My bookings". There is no customer column. Buttons are QR ticket and Cancel.

• Staff and organiser: a customer column with name and email. Buttons are Check in and Cancel.

• Admin: the same as staff, plus Delete for cancelled bookings.

The event management page works the same way. An organiser sees "My events" with only their own events, because the page sends organiser_id in the request. An admin sees "All events", and each row shows the organiser's name.

Page	Who can open it	Endpoints used
login.html	Everyone	POST /auth/login
register.html	Guests	POST /auth/register
index.html	Everyone	GET /events (search, sort, pages)
event.html?id=	Everyone. Only customers can book	GET /events/{id}, POST /bookings
bookings.html	Customer, staff, organiser, admin	GET /bookings, PUT /bookings/{id}, DELETE /bookings/{id}, GET /bookings/{id}/qr
manage-events.html	Organiser (own events), admin (all events)	GET /events, POST /events, PUT /events/{id}, DELETE /events/{id}, GET /venues
venues.html	Organiser, admin. Only admin can delete	GET /venues, POST /venues, PUT /venues/{id}, DELETE /venues/{id}
users.html	Admin	GET /users, POST /users, PUT /users/{id}, DELETE /users/{id}
Every page	Logged-in users	POST /auth/logout

My frontend uses 21 of the 27 API endpoints. It does not use these six:

• /auth/me and /auth/refresh, because the app uses the saved user and sends people back to login when the token expires.

• GET /users/{id}, GET /venues/{id} and GET /bookings/{id}, because the list already gives the full record.

• GET /tickets/{reference}, because it needs the API key and is made for a door scanner, not a browser.

File	What it does
js/api.js	apiRequest(method, path, body) sends every request, adds the token, checks if the reply is OK, and turns every error into the same format
js/auth.js	Saves and clears the login, protects pages with requireRole(), logs out
js/nav.js	Draws the navbar for each role, and my name and student ID
js/ui.js	Shared helpers: safe text, date and price format, alerts, field errors, loading buttons, simple error messages
js/login.js, js/register.js	Login form and customer sign-up form
js/index.js, js/event.js	Event list, event details and the booking form
js/bookings.js	Bookings page with buttons for each role, and the QR ticket pop-up
js/manage-events.js	Event list, plus the pop-up form to create and edit events
js/venues.js, js/users.js	Venue and user management with pop-up forms
css/style.css	Colours and small layout rules on top of Bootstrap

Things every page does

- Shows a loading spinner, a message when the list is empty, and the API's error if something fails.
- Checks the form in the browser first, using the same rules as the API. For example: required fields, email format, password of at least 8 characters, matching passwords, 1 to 10 tickets, end time after start time, and tickets not more than the venue capacity.
- Shows server errors (422) under the correct field. The other fields keep what the user typed.
- Asks "Are you sure?" before cancel, delete and publish.
- Shows a success message after every create, update and delete, using the API's own message.
- When editing, sends only the fields that changed. If nothing changed, it shows "Nothing has changed yet." and sends nothing.

5.2.4 Member 4: Muhammad Shihabuddin Al-Fikri Bin Anuar

a. Frontend Structure and Interface

The frontend files are stored in the frontend-al folder. The files are used to manage the website layout, design and functions. The interface is kept simple so users can find the features they need without too much difficulty. The navigation menu also changes based on the user's role.

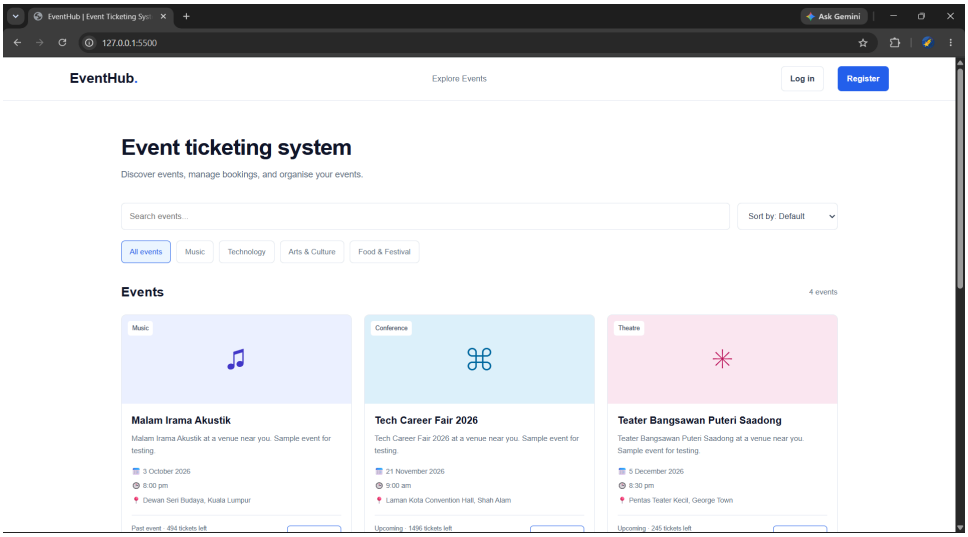


Figure 5.1: EventHub Main Page and Event Listing

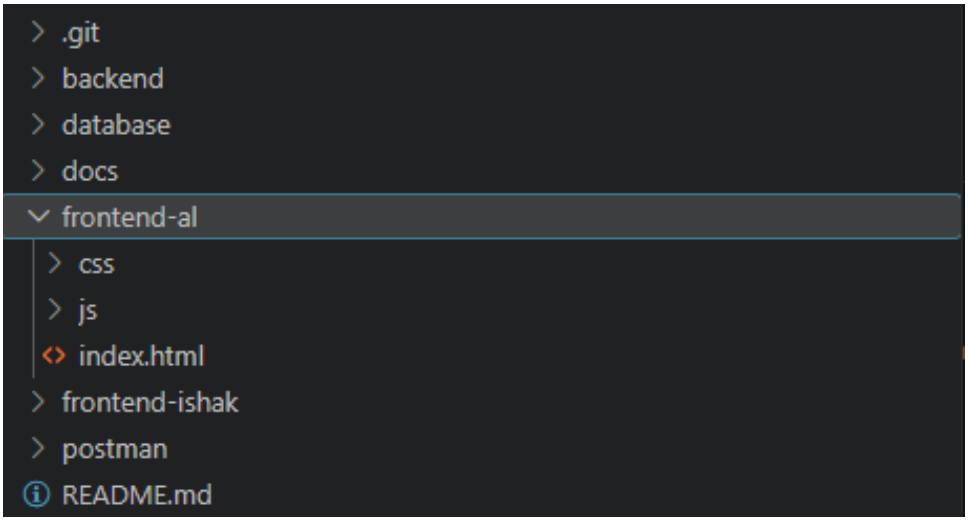


Figure 5.2: Frontend Project Structure

b. Frontend Features

The frontend provides different features for customers, organisers and administrators. Customers can browse events, search for events and choose the number of tickets they want to book. They can also check their booking history, view ticket QR codes and cancel bookings when allowed.

Organisers can manage events and venues through the available management pages. Administrators can manage users, events, venues and bookings. The features shown on the website depend on the user's role.

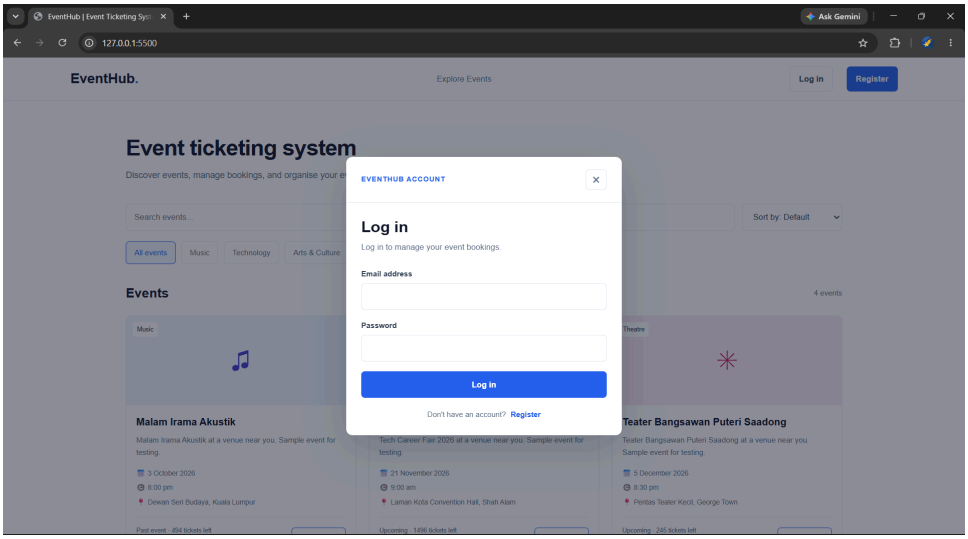


Figure 5.3: Customer Login and Registration Interface

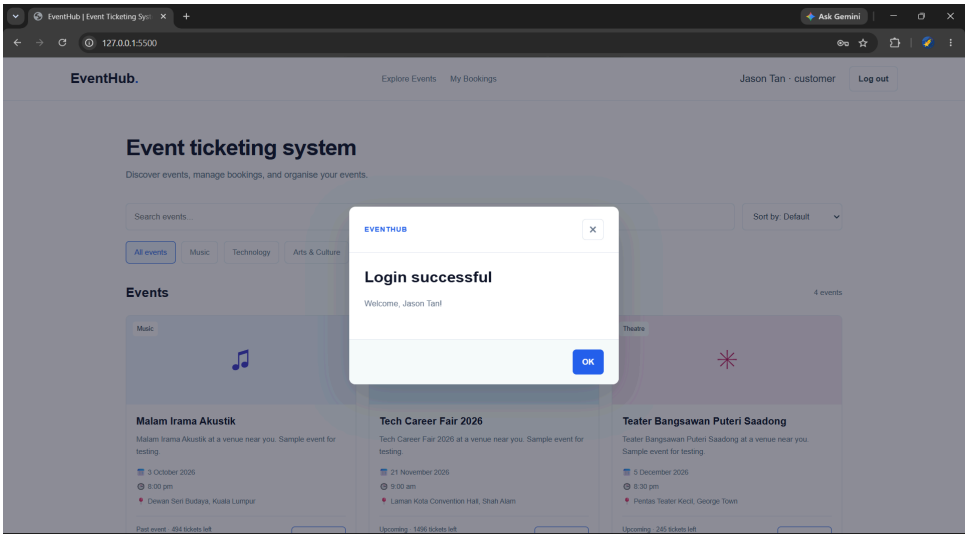


Figure 5.4: Login Successful

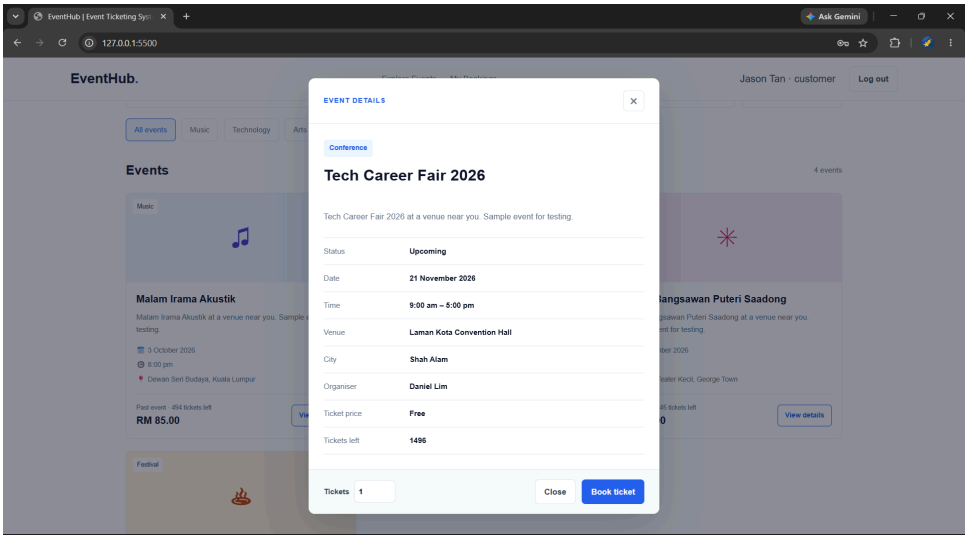


Figure 5.5: Ticket Booking Interface

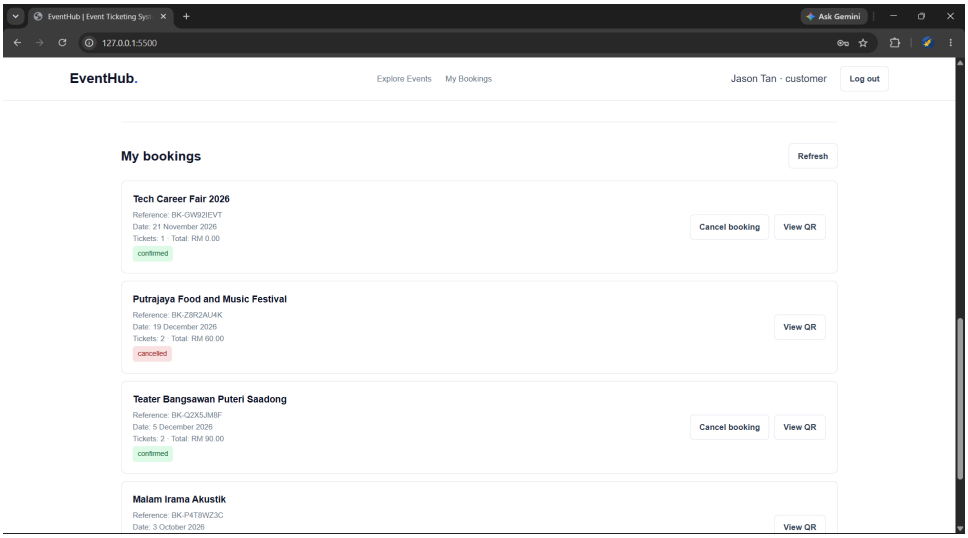


Figure 5.6: Customer Booking History and QR Code

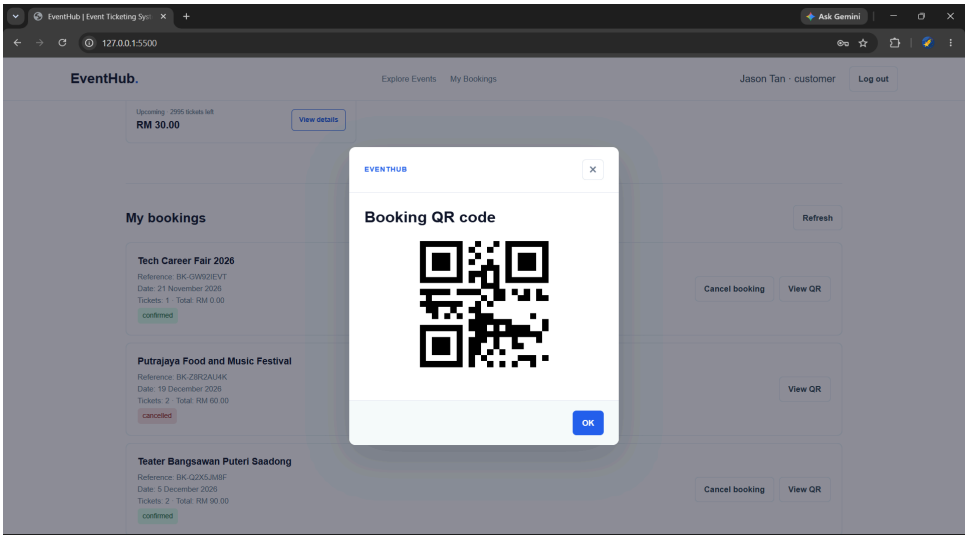


Figure 5.7: QR Code

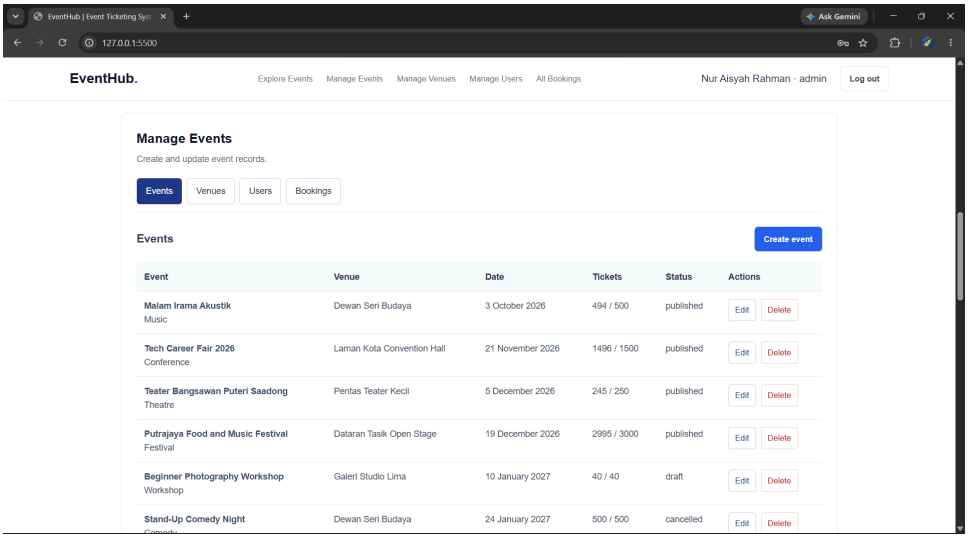


Figure 5.8: Event Management Interface

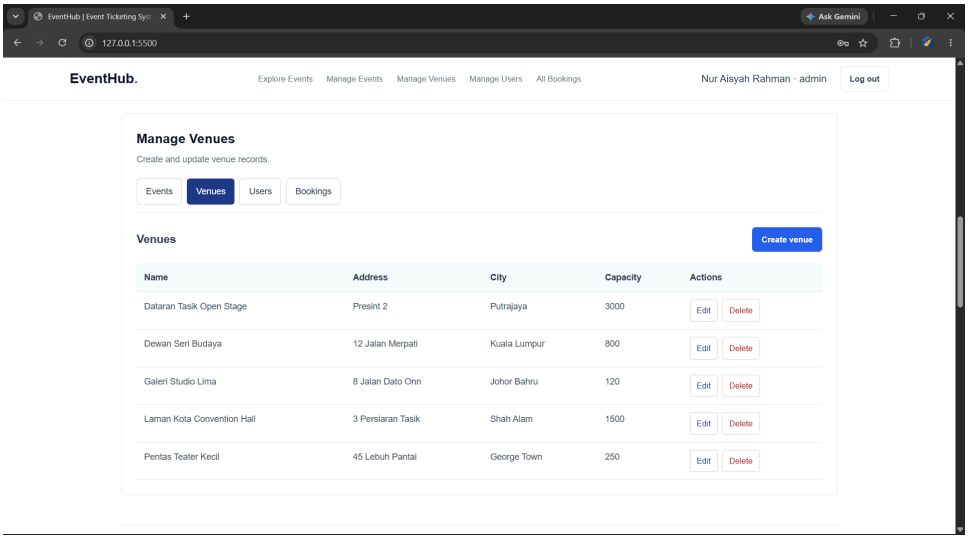


Figure 5.9: Event and Venue Management Interface

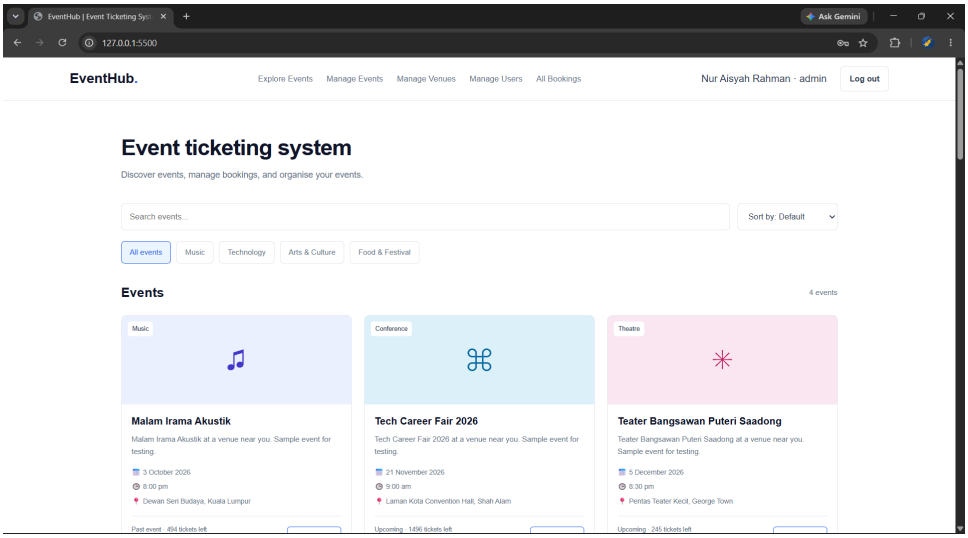


Figure 5.10: Administrator Management Dashboard

5.3 Frontend and API Communication

Each frontend transmits HTTP requests containing a JSON body and receives a JSON response. Regardless of the tools they have selected, the interaction with the API always follows this pattern: the request must be sent to the correct URL using the appropriate method, the JWT should be included in the Authorization header after the user has logged in, and then the status code should be read in order to determine what action to take next. If the status code is in the 2xx range, the data can be displayed, but if it is a 4xx status code there is a JSON message, and in the case of a 422 status code an errors object is provided which the page can display to the user. The various members then explain below how their particular frontend sends the requests, attaches the token and deals with errors.

5.3.1 Member 1: Ishak Bin Ismail (frontend-ishak/)

In this frontend, each request is processed by a single function which is based on the browser's Fetch API (MDN Web Docs, n.d.). This function includes the token obtained during login in every request and converts all failures into a uniform error. The function gradually constructs the complete URL from the API's base address, sets the Accept: application/json header, adds the Authorization: Bearer header whenever the user is logged in, and converts the body into JSON. Upon receiving the reply it parses the JSON, and if the status is a success it just returns the data; if the status is not successful it throws an ApiError which includes the status code, the message from the API and any field errors, so that all the pages can handle failures in the same manner. If a 401 error occurs while the user is logged in, this indicates that the token has expired and the function also clears the session at that time. Since all of this is managed in a single location, none of the pages have to look after the headers, the tokens or the error formats themselves.

```
export async function api(method, path, { query, body } = {}) {
    const url = new URL(API_BASE + path);

    for (const [key, value] of Object.entries(query || {})) {
        if (value !== '' && value !== null && value !== undefined && value !== false) {
            url.searchParams.set(key, value === true ? '1' : value);
        }
    }

    const headers = { Accept: 'application/json' };
    const token = getToken();
    if (token) {
        headers.Authorization = `Bearer ${token}`;
    }
    if (body !== undefined) {
        headers['Content-Type'] = 'application/json';
    }
```

```
    if (response.ok) {
        return data;
    }

    // 401 while logged in means the token expired or was logged out elsewhere.
    if (response.status === 401 && token) {
        clearSession();
        window.dispatchEvent(new CustomEvent('session-expired'));
    }

    const fallback = {
        401: 'Please log in to continue.',
        403: 'You are not allowed to do this.',
        404: 'This item could not be found.',
        429: 'Too many requests. Wait a minute and try again.',
    }[response.status] || 'Something went wrong on the server. Please try again.';

    throw new ApiError(response.status, (data && data.message) || fallback, (data && data.errors) ||
}
```

Figure 5. : frontend-ishak/js/app.js Code

The listing has been shortened; the complete file also deals with cases where the server cannot be reached and where the reply is not in JSON format. When the backend is not running, the error is stated in simple terms, and if the reply contains no message of its own, a helpful fallback message is displayed for errors 401, 403, 404 or 429, thus ensuring that the user never encounters a blank or technical error.

The frontend uses port 5500 while the API uses port 8000. Thanks to Laravel's built-in cross-origin (CORS) middleware, the browser is able to call the API which is running on the other port. Since a browser considers two different ports to be two separate origins, it normally prevents a page from reading the responses from a different origin. That is why the backend includes CORS headers in its API responses, informing the browser that the frontend may read them. If these headers were not included, all the requests from the frontend would fail in the browser even though the API itself functions correctly. For development purposes, the frontend is served using PHP's built-in server (php -S 127.0.0.1:5500 -t frontend-ishak), as stated in its README.

5.3.2 Member 2: Ahmad Naufal bin Ahmad Fuad (frontend-naufal/)

1. Request structure

The frontend only interacts with the backend via HTTP requests on the endpoint http://127.0.0.1:8000/api using JSON both in and out. All the pages build requests, and all read responses, in the same way, using the function apiRequest(method, path, body) in js/api.js .

Every request contains an Accept: application/json header, so the Laravel backend will always respond back to the request in JSON format even in the case of an error. If the request contains a body, like a login form, or a new event, the JavaScript object is converted to a JSON string using JSON.stringify() and sent with a Content-Type: application/json header. If the user is already logged in, the Authorization: Bearer <token> is included in the request, and the backend's authentication middleware retrieves the user and his/her role from the JWT.

2. Authentication flow

The token will be acquired from /auth/login or /auth/register. They both return the JWT in data.access_token, and the user (along with their role) in data.user. The Frontend stores both in localStorage and adds the token to subsequent requests.

The token will be valid for 60 minutes. Once it is expired, the following protected request will return 401. Then, apiRequest() removes the session and redirects to login with a message indicating that the session has expired. Logging out will send POST /auth/logout, which will blacklist the token on the server, and if the request is successful, the local session will be removed.

3. Reading responses

The back end is implemented with three different shapes of responses, each of which is read directly by the front end:

- Single resource: { "message": "...", "data": { ... } }. A page that displays or fills a form with data, and displays message as a success alert after a create, update or delete.

- Paginated list: the items are in data, next to current_page, last_page, total, from and to. These values are used to control the pagination buttons and the text “Showing 1–3 of 4 events”.
- Error: { "message": "..." }, and an errors object with validation errors.

Search, filtering, sorting and paging are not performed in the browser. User choices are saved to a tiny state object in List pages and then translated to query parameters using URLSearchParams.For instance, if you're looking for a list of festivals sorted by ticket price, you will send GET /events?page=1&per_page=3&sort=ticket_price&direction=asc&search=festival, and the API will only return the results on that specific page.

4. Error handling

Because fetch() only rejects when the network fails (MDN contributors, n.d.), apiRequest() checks response.ok and throws a single error object containing the status code, the API’s message and any field errors. Each page traps this object and reacts as per the status code as the table below.

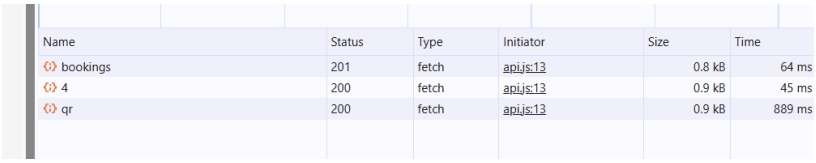
Status	Example from the API	Frontend behaviour
200 / 201	“Booking confirmed.”	Shows the result and a success message; refreshes the list
401	“Invalid email or password.” / expired token	On login: shows the message. Elsewhere: clears the session and returns to login
403	“Forbidden. Your role is not allowed to do this.”	Shows a friendly permission message
404	“Resource not found.”	Shows “This event does not exist or is not available.”
409	“This event has bookings. Cancel the event instead of deleting it.”	Shows the API’s business-rule message as an alert
422	errors.email: ["The email	Marks each field red and

	has already been taken."]	shows its message underneath
429	Too many login attempts	Asks the user to wait one minute
502	QR service did not respond	Shows that the QR code is unavailable without breaking the page
0 (no response)	Backend not running	Shows “Cannot reach the API”

Example: A customer booking.

The booking workflow illustrates how all these parts come together:

1. The event page redirects to GET /events/4 and shows the event, venue and the count of the tickets remaining.
2. The customer types in how many they need. The browser checks that it is between 1 and the lesser of 10 or the tickets remaining, and shows a preview of the total.
3. The page submits a POST /bookings request containing a body of { "event_id": 4, "quantity": 2 } and the customer's JWT. The server computes the actual total and subtracts the tickets available.
4. The API returns 201 with the new booking, including its booking_reference. The page returns a confirmation card, and resends GET /events/4 to get the other tickets.



Name	Status	Type	Initiator	Size	Time
bookings	201	fetch	api.js:13	0.8 kB	64 ms
4	200	fetch	api.js:13	0.9 kB	45 ms
qr	200	fetch	api.js:13	0.9 kB	889 ms

Figure 5: Booking fetch

Request URL	http://127.0.0.1:8000/api/bookings
Request Method	POST
Status Code	201 Created
Remote Address	127.0.0.1:8000
Referrer Policy	strict-origin-when-cross-origin

Figure 5: Header network

5. The page then returns a call to GET /bookings/{id}/qr.The page then returns a call for GET /bookings/{id}/qr. The frontend doesn't call the third party service itself, it simply asks the backend to get the QR code from the API of goQR.me which then caches it for 1 day and returns it as a base64 PNG in the qr_code field. This value is directly inserted into the src attribute of an <img> element by the frontend.

```
// QR code comes from the backend, which calls the goQR.me third-party API
async function showQr(id) {
  const body = document.getElementById('qrBody');
  body.innerHTML = '<div class="spinner-border my-4"></div>';
  qrModal.show();

  try {
    const { data } = await apiRequest('GET', `/bookings/${id}/qr`);
    body.innerHTML = `
      <img src="${data.qr_code}" alt="QR code" class="img-fluid mb-2" style="max-width: 220px;">
      <p class="mb-1"><code>${escapeHtml(data.booking_reference)}</code></p>
      <p class="small text-muted mb-0">Generated by ${escapeHtml(data.provider)}
        ${data.from_cache ? '(cached)' : ''}</p>`;
  } catch (err) {
    const msg = err.status === 502 ? 'The QR code service is not responding. Try again later.' : err.message;
    body.innerHTML = `<p class="text-danger my-3">${escapeHtml(msg)}</p>`;
  }
}
```

Figure 5: QR codes function


```
{
    "data": {
        "booking_id": 9,
        "booking_reference": "BK-YBHBV0UX",
        "status": "confirmed",
        "qr_code": "data:image\/png; base64,iVBORw0KGgoAAAANSUhEUgAAASwAAAE
        "provider": "goQR.me (api.qrserver.com)",
        "from_cache": false
    }
}
```


Figure 5: Response

6. If the event has sold out by then, step 3 returns 409, with “Only N tickets are left for this event.” and the message is displayed.

Security

Any text returned from the API (e.g., event titles and user names) are passed through escapeHtml() before being added to the page. This will not let HTML or scripts run in the browser from memory. The user's JWT is the only credential that is stored in the browser. The X-API-Key protecting /api/tickets/{reference} should only be used by machine to machine and never in the frontend code.

5.3.3 Member 3 : Muhammad Anas Bin Haddri (frontend-anas/)

1. One function for all requests

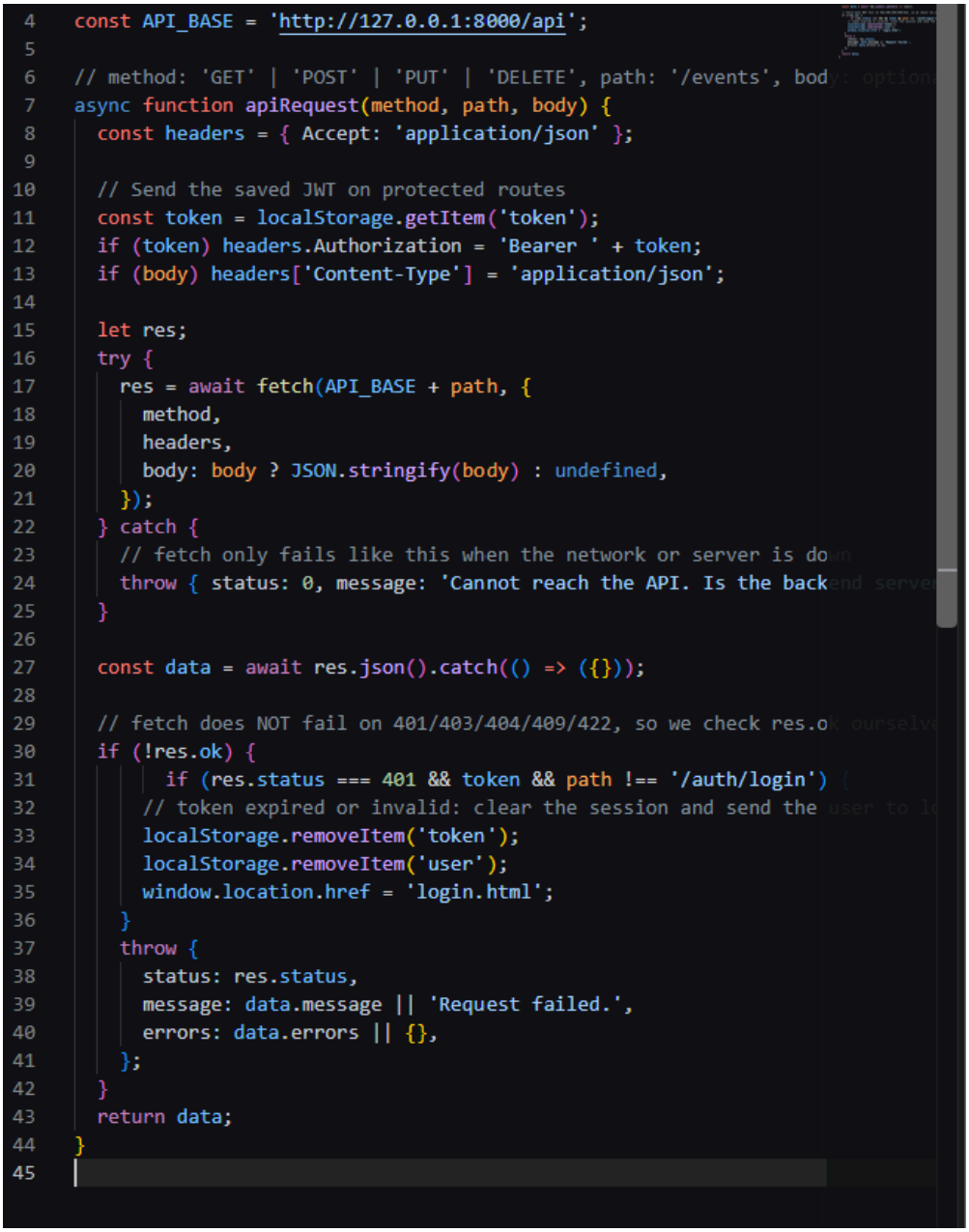
All requests to the backend go through one function, apiRequest(method, path, body), in js/api.js. It uses the browser's Fetch API (MDN Web Docs, n.d.). Because of this, no page needs to set headers or check status codes by itself. The function does four things:

• It adds Accept: application/json to every request, so Laravel always replies in JSON, even for errors.

• If there is a body, it changes it into a JSON string with JSON.stringify() and adds Content-Type: application/json.

• If the user is logged in, it adds the token as Authorization: Bearer <token>.

• If the reply is an error, it throws one error object in the same format every time.



```
const API_BASE = 'http://127.0.0.1:8000/api';

// method: 'GET' | 'POST' | 'PUT' | 'DELETE', path: '/events', bod
async function apiRequest(method, path, body) {
  const headers = { Accept: 'application/json' };

  // Send the saved JWT on protected routes
  const token = localStorage.getItem('token');
  if (token) headers.Authorization = 'Bearer ' + token;
  if (body) headers['Content-Type'] = 'application/json';

  let res;
  try {
    res = await fetch(API_BASE + path, {
      method,
      headers,
      body: body ? JSON.stringify(body) : undefined,
    });
  } catch {
    // fetch only fails like this when the network or server is do
    throw { status: 0, message: 'Cannot reach the API. Is the back
  }

  const data = await res.json().catch(() => ({}));

  // fetch does NOT fail on 401/403/404/409/422, so we check res.o
  if (!res.ok) {
      if (res.status === 401 && token && path !== '/auth/login')
    // token expired or invalid: clear the session and send the
    localStorage.removeItem('token');
    localStorage.removeItem('user');
    window.location.href = 'login.html';
    }
    throw {
      status: res.status,
      message: data.message || 'Request failed.',
      errors: data.errors || {},
    };
  }
  return data;
}
```

Figure 5.13. frontend-anas/js/api.js, the one function every page uses to call the API.

The most important part is the res.ok check. fetch() only fails when the network is down. A 403 or 409 reply still comes back as a normal reply (MDN Web Docs, n.d.). Without this check, a refused booking would look like a success. So the function throws one error object, { status, message, errors }, for every failure. Each page only needs to read these three values.

2. Login and staying logged in

POST /auth/login and POST /auth/register both send back the token in data.access_token and the user (with their role) in data.user (Jones et al., 2015). saveSession() in js/auth.js saves both in localStorage.

• The token lasts 60 minutes. After that, the next protected request returns 401. apiRequest() then clears the login and opens the login page.

• The login page does not follow this rule. On the login page, 401 just means a wrong password, so the user sees "Invalid email or password." instead.

• Logging out sends POST /auth/logout, so the server blocks the token. The browser clears the login even if this request fails.

3. Reading the replies

The API sends three kinds of replies:

• One item: { "message": "...", "data": { ... } }. The page reads data to fill a form or show details. It shows a message in a green box after a create, update or delete, such as "Booking confirmed.", "Customer checked in." or "Venue deleted."

• A list: the items are in data, with current_page, last_page and total. These control the Previous and Next buttons and the text "Page 1 of 2 (9 bookings)".

• An error: { "message": "..." }. For a 422 error, there is also an errors object.

Search, filter, sort and pages are all done by the server, not the browser. Each list page puts the user's choices in the URL with URLSearchParams. For example: GET /bookings?page=1&per_page=10&search=video1&status=confirmed. This way, the browser only downloads the rows it shows.

4. What the user sees for each status code

Status	Example from the API	What the frontend does
200 / 201	"Booking confirmed."	Shows the result and a

		success message, then reloads the list
401	"Invalid email or password.", or the token has expired	On the login page, it shows the message. On other pages, clears the login and opens the login page
403	"Forbidden. You can only manage your own events."	Shows "Your role is not allowed to do this." plus the API's message
404	"Resource not found."	Shows "That item no longer exists." On the event page, "Event not found."
409	"The venue is already booked for "Tech Career Fair 2026" at that time."	Shows the API's message as it is, in the form or above the list
422	errors.email: ["The email has already been taken."]	Makes each wrong field red and shows its message under it
429	Too many login tries	Shows "Too many requests. Wait a minute and try again."
502	QR service did not reply	Shows "The QR code service is not responding right now." in the QR pop-up. The rest of the page still works
0	Backend is not running	Shows "Cannot reach the API. Is the backend server running?"

These codes follow the normal meanings of HTTP status codes (Fielding et al., 2022). The short messages for 403, 404 and 429 are in one helper, friendlyError() in js/ui.js, so every page uses the same words.

5. Example: a customer books two tickets

1. event.html?id=3 calls GET /events/3. It shows the event, venue, organiser and "245 of 250" tickets left.

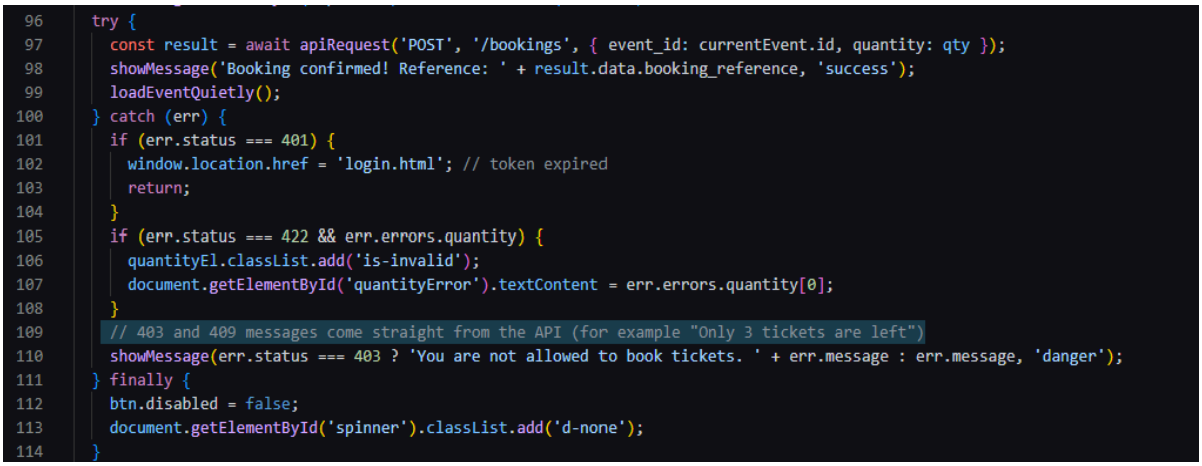
2. The customer types a number. The total changes as they type. This total is only a preview.

3. Before sending, the browser checks that the number is from 1 to 10. Then it asks "Book 2 ticket(s) for Teater Bangsawan Puteri Saadong?".

4. The page sends POST /bookings with { "event_id": 3, "quantity": 2 } and the customer's token. The server works out the real total and removes the tickets from the event.

5. The API replies 201 with the new booking. The page shows "Booking confirmed! Reference: BK-…". It then calls GET /events/3 again, so the number of tickets left goes down without reloading the page.

6. If someone else bought the last tickets first, step 4 returns 409 with "Only 1 tickets are left for this event.", and the page shows that message.



```
96  try {
97    const result = await apiRequest('POST', '/bookings', { event_id: currentEvent.id, quantity: qty });
98    showMessage('Booking confirmed! Reference: ' + result.data.booking_reference, 'success');
99    loadEventQuietly();
100 } catch (err) {
101   if (err.status === 401) {
102     window.location.href = 'login.html'; // token expired
103     return;
104   }
105   if (err.status === 422 && err.errors.quantity) {
106     quantityEl.classList.add('is-invalid');
107     document.getElementById('quantityError').textContent = err.errors.quantity[0];
108   }
109   // 403 and 409 messages come straight from the API (for example "Only 3 tickets are left")
110   showMessage(err.status === 403 ? 'You are not allowed to book tickets. ' + err.message : err.message, 'danger');
111 } finally {
112   btn.disabled = false;
113   document.getElementById('spinner').classList.add('d-none');
114 }
```

Figure 5.14. The booking request in frontend-anas/js/event.js, lines 96 to 114.

The QR ticket works in the same way. On the bookings page, the QR ticket button calls GET /bookings/{id}/qr. The browser never talks to goQR.me directly. The backend gets the image, saves it for a day and sends it back inside the JSON (goQR.me, n.d.). The page puts it in an <img> tag in a pop-up, with the booking reference under it.

6. Security in the browser

• Text from the API, like event titles and user names, goes through esc() or is set with textContent before it is shown. If a title has HTML or a script inside, it is shown as plain text and never runs.

• The token is the only login detail saved in the browser. The X-API-Key for /api/tickets/{reference} is only for machines and is not in this frontend at all.

• Hiding buttons by role is not real security. The "My events" filter only changes what an organiser sees. If they still sent a request to edit or delete another organiser's event, the server would reply 403.

The frontend runs on port 5501 and the API on port 8000. The browser treats these as two different origins. Laravel's built-in CORS middleware adds the headers that let my page read the API's replies.

5.3.4 Member 4: Muhammad Shihabuddin Al-Fikri Bin Anuar

5.4.1 Sending API Requests

The frontend uses the fetch() function to send HTTP requests to the backend. The apiRequest() function handles the request headers, sends the request and processes the response. It also checks for errors when a request fails.

```
async function apiRequest(path, options = {}) {
    const headers = {
        Accept: "application/json",
        ...(options.body ? { "Content-Type": "application/json" } : {}),
        ...(accessToken ? { Authorization: `Bearer ${accessToken}` } : {}),
        ...options.headers
    };

    const response = await fetch(`${API_URL}${path}`, {
        ...options,
        headers
    });

    const text = await response.text();
    let result = {};

    if (text) {
        try {
            result = JSON.parse(text);
        } catch {
            result = { message: text };
        }
    }

    if (!response.ok) {
        const error = new Error(
            result.message || `Request failed (${response.status})`
        );
        error.status = response.status;
        error.data = result;
        throw error;
    }

    return result;
}
```

Figure 5.8: JavaScript API Request and Response Handling

5.4.2 User Authentication

When a user logs in, the frontend sends the account details to the /auth/login endpoint. After a successful login, the backend returns an access token and user information. The frontend stores the token and includes it in the Authorization header when sending requests that require authentication.

```
async function handleAuthSubmit(event) {
    event.preventDefault();

    const registering = authMode === "register";
    const email = $("#auth-email").value.trim();
    const password = passwordInput.value;
    const payload = { email, password };

    if (registering) {···
    }

    authSubmit.disabled = true;
    authSubmit.textContent = registering ? "Creating account..." : "Logging in...";
    authMessage.textContent = "";

    try {···
    } catch (error) {
        authMessage.textContent = errorMessage(error);
    } finally {
        authSubmit.disabled = false;
        authSubmit.textContent = registering ? "Create account" : "Log in";
    }
}
```

Figure 5.9: Login and JWT Authentication Code

5.4.3 Booking API Integration

When a customer books a ticket, the frontend sends a POST request to the /bookings endpoint. The request includes the event ID and the number of tickets selected. After the booking is processed, the frontend refreshes the event information and booking list.

The frontend also sends a request to retrieve the QR code for a booking. The returned QR code is displayed on the website so customers can access their ticket information.



```
async function createBooking() {
    if (!selectedEvent || isSubmittingBooking) return;

    if (!currentUser || !accessToken) {···
    }

    if (currentRole() !== "customer") {···
    }

    const quantity = Number(quantityInput.value);
    const available = Number(selectedEvent.available_tickets ?? 0);

    if (!Number.isInteger(quantity) || quantity < 1 || quantity > 10) {···
    }

    if (quantity > available) {···
    }

    isSubmittingBooking = true;
    modalBookButton.disabled = true;
    modalBookButton.textContent = "Processing...";

    try {···
    } catch (error) {
        if (error.status === 401) {
            accessToken = null;
            currentUser = null;
            updateNavigation();
            closeEventModal();
            openAuthModal("login");
            authMessage.textContent = "Your session expired. Please log in again.";
        } else {
            showMessage("Booking unsuccessful", errorMessage(error));
            await loadEvents();
        }
    } finally {···
    }
}
```

Figure 5.10: Ticket Booking API Integration

```
async function showBookingQR(id) {
    try {
        const result = await apiRequest(`/bookings/${id}/qr`);
        const data = result.data || {};

        const source = data.image ||
            data.qr_code ||
            (data.base64 ? `${data.base64}` : "");

        if (!source) {
            showMessage("QR code", "The API did not return a recognised QR image field.");
            return;
        }

        $("#message-title").textContent = "Booking QR code";

        const text = $("#message-text");
        text.replaceChildren();

        const image = document.createElement("img");
        image.src = source;
        image.alt = "Booking QR code";
        image.style.display = "block";
        image.style.width = "220px";
        image.style.maxWidth = "100%";
        image.style.margin = "12px auto";

        text.appendChild(image);
        showModal(messageModal);
    } catch (error) {
        showMessage("QR unavailable", errorMessage(error));
    }
}
```

Figure 5.11: QR Code API Integration

5.4.4 Error Handling

The frontend checks the API response and handles unsuccessful requests. When an error occurs, the system processes the response and displays an appropriate message to the user. The backend also validates submitted data and returns error details when the request does not meet the required rules.

```
if (!response.ok) {
    const error = new Error(
        result.message || `Request failed (${response.status})`
    );
    error.status = response.status;
    error.data = result;
    throw error;
}
```

Figure 5.12: API Error Handling Code

6.0 System Demonstration

The screenshots presented in this section illustrate the system as it operates within the browser, focusing on one role at a time and identifying the API call associated with each screen. They are organised by member, allowing each member's frontend to be viewed separately. For each member's section, the screens are arranged in the same order that a real user would encounter them: first a guest carrying out browsing and signing up, then a customer booking and viewing a ticket, followed by the staff, organiser and admin pages. The screenshots also include error situations, for example those involving a duplicate email or a conflicting event, in order to demonstrate that the frontend displays the API's own error messages rather than a general failure message. Each caption specifies the endpoint that the screen calls, so that the screens can be referenced against the API documentation in Section 3.

6.1 Member 1: Ishak Bin Ismail (frontend-ishak/)

6.1.1 Guest

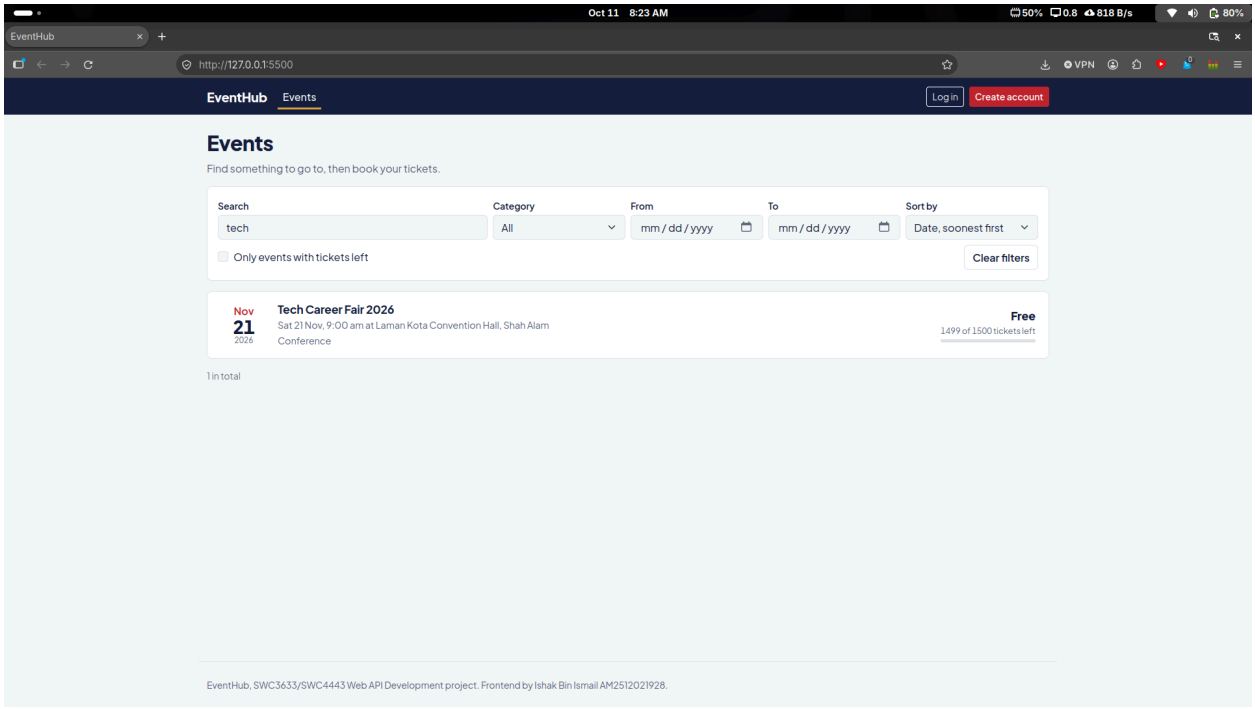


Figure 6.1: Guest Dashboard

Since the guest is not logged in, the navigation bar displays only the Events, Log in and Create account options. The filter bar forwards to the API the search box input, the category, the date range, the 'Only events with tickets left' option and the sort order; the API then carries out the filtering on the server and so the browser does not load any events that it doesn't need. For each result the date, venue, category, price and the number of tickets remaining are shown, and the figure under the list indicating the total ('1 in total') is obtained from the pagination information in the API's response.

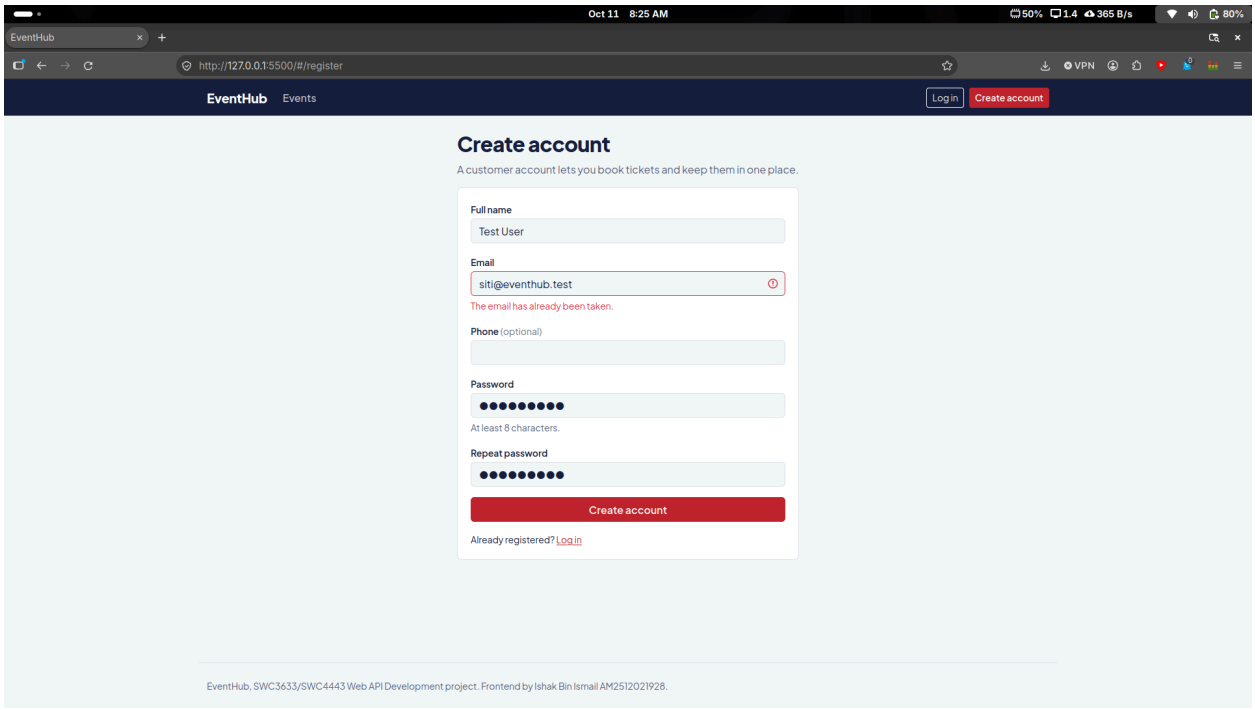


Figure 6.2: Register Page

The form requires the full name, an email address, an optional phone number and a password which must be at least eight characters long and must be entered twice. Since the email address siti@eventhub.test is already associated with a seeded customer, the API's validation rejects it. The frontend takes the errors object from the 422 response and displays the exact message from the server by highlighting the Email field in red, the other fields retaining the text that the user had typed.

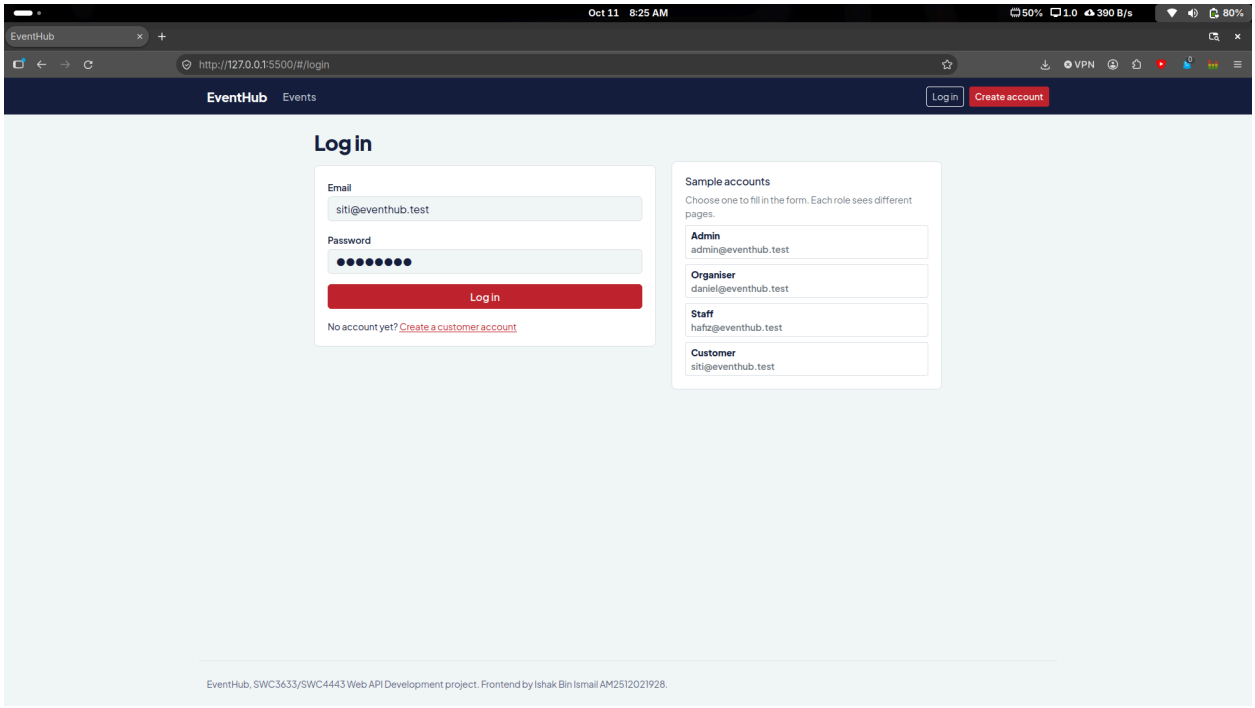


Figure 6.3: Login Page

On the right-hand side, the Accounts panel shows one sample account for each role, and when you click on one it fills in the form, which means it's easy to test what each role can see. Once you have logged in successfully, the token and user details are stored in localStorage and the navigation bar is then altered to correspond to the user's role.

6.1.2 Customer

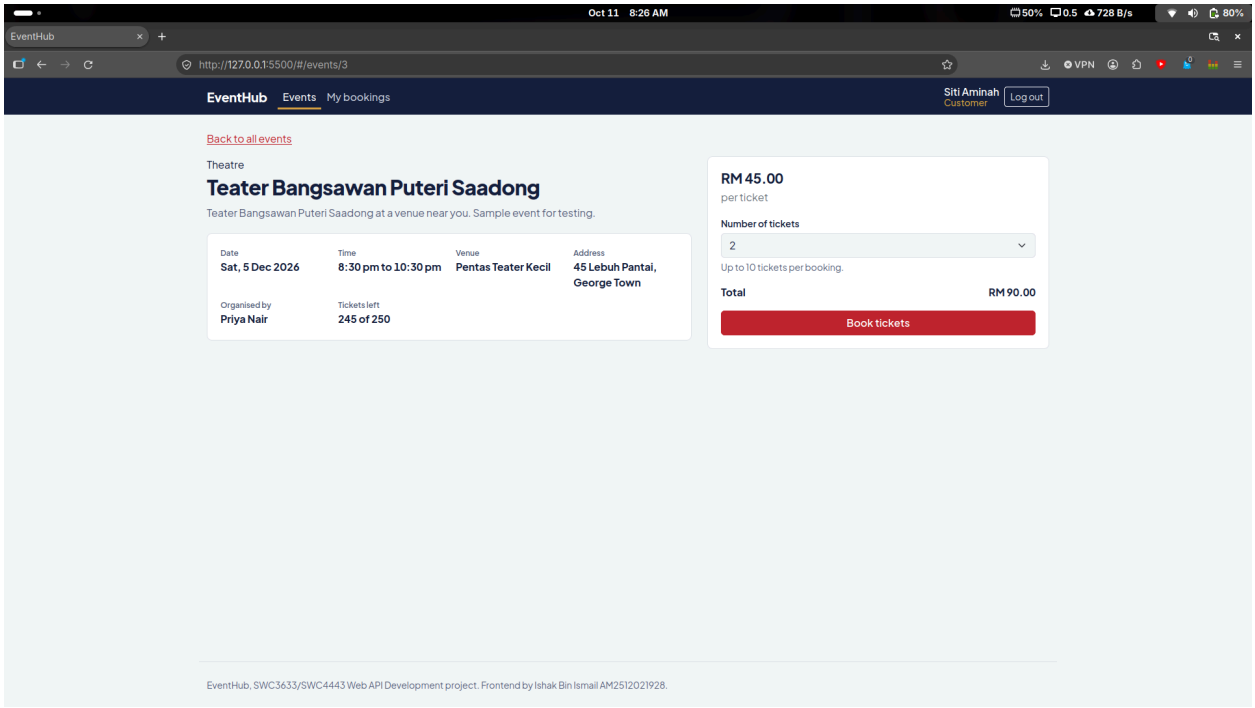


Figure 6.4: Customer Dashboard

The page displays the information obtained from the GET /events/{id} request, including the date and time, the venue and its address, the organiser, and the number of tickets remaining (245 out of 250). Siti, who is the logged-in customer, selects 2 tickets at a price of RM 45.00 and the page then shows a total amount of RM 90.00. However, this total is merely a preview; the server calculates the actual total when the booking is saved, which means that the price cannot be altered from within the browser. The list of quantities is cut off at 10, consistent with the API's limit of 10 tickets per booking.

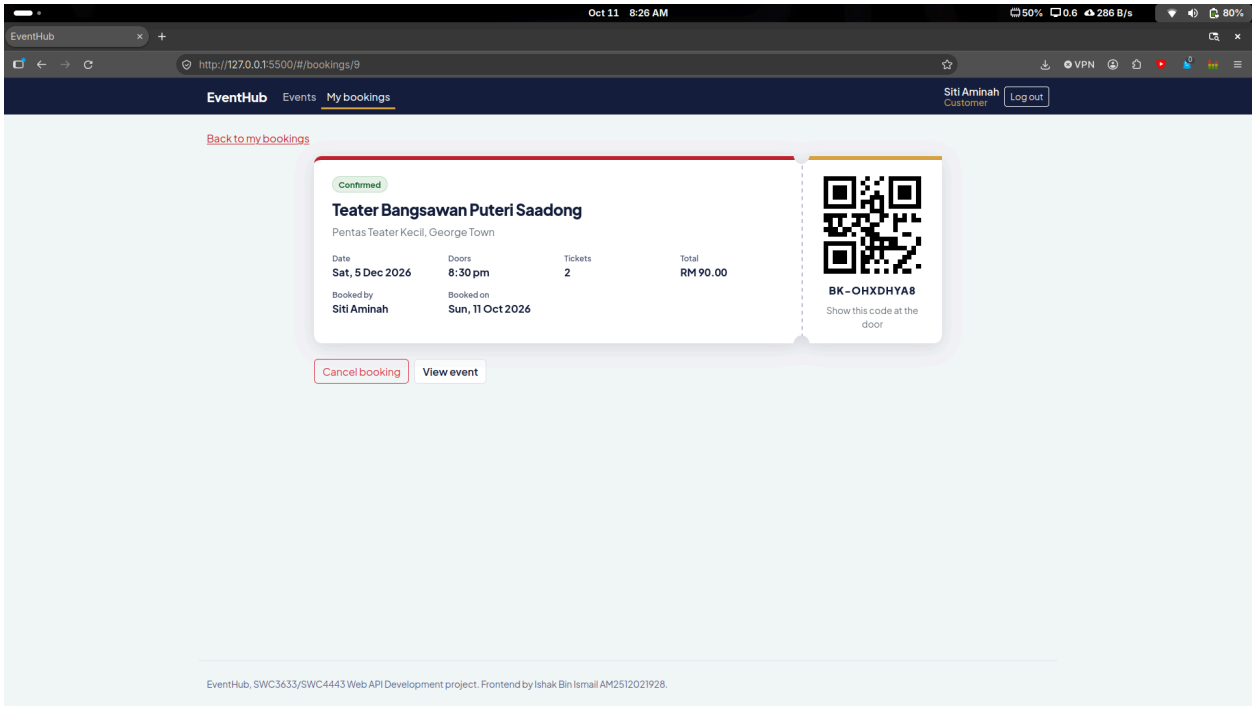


Figure 6.5: QR Code Ticket

As soon as the booking has been made, the customer is directed straight to the ticket, which displays the event, the venue, the date, the door time, the number of tickets, the total amount and the date of the booking, together with a Confirmed badge. The QR code on the right side of the ticket contains the booking reference BK-OHXDHYA8, which the door staff can scan. Under the ticket there are two options: one allows the customer to cancel the booking as long as the event hasn't yet started, and the other one enables the customer to access the event page again.

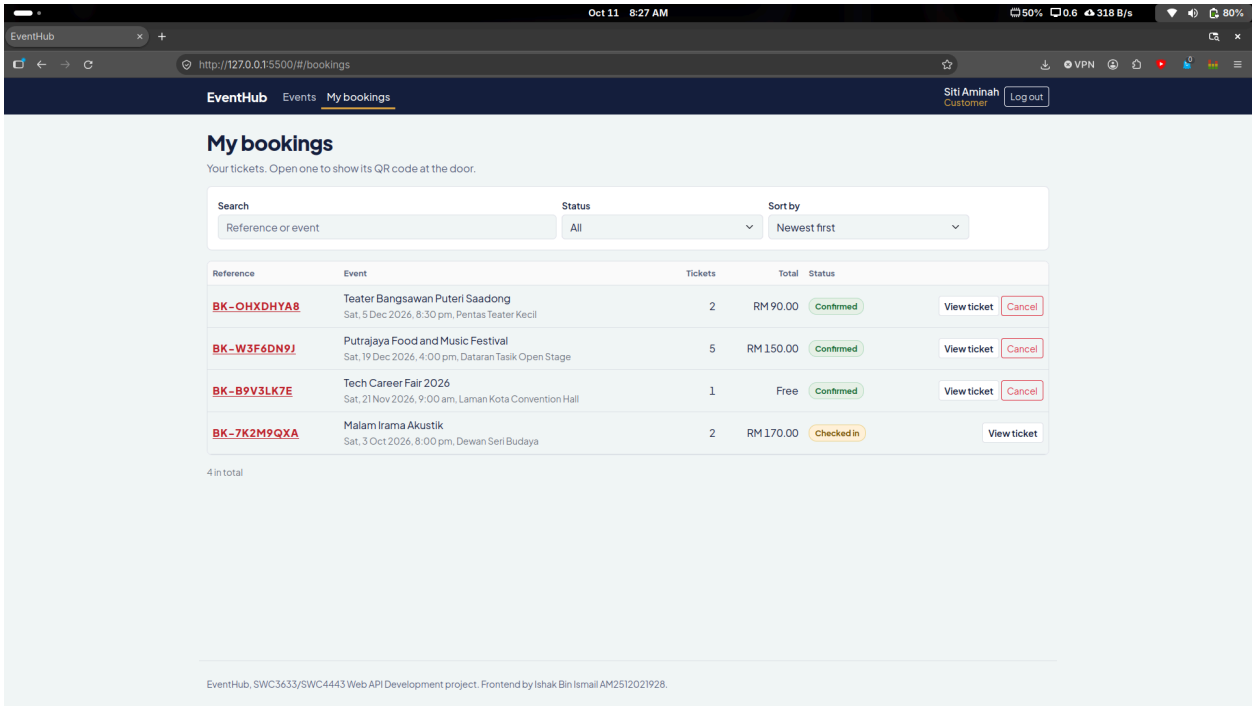


Figure 6.6: Customer's My Bookings Page

Siti can see only her four bookings since the API restricts the list to the current user and thus she is unable to view anyone else's. Each row displays the reference, the event, the number of tickets, the total amount and a coloured status badge; the booking for the Free Tech Career Fair shows "Free" rather than RM 0.00, and the booking for Malam Irama Akustik which has already been used does not have a Cancel button.

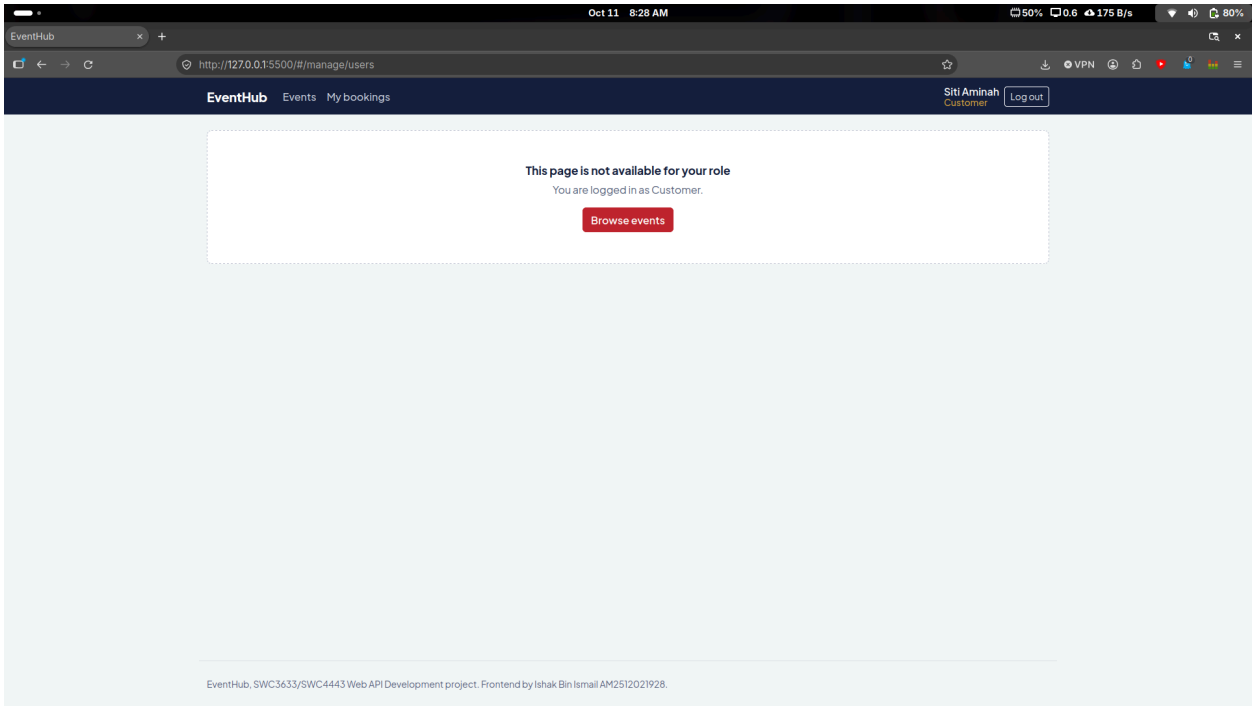


Figure 6.7: Role Handling in The Frontend

The address bar displays #/manage/users, an address which a customer could type in by hand. Rather than showing an error page, the frontend shows a clear message that identifies the user's role together with a button that takes you back to the events list. Although this check is intended to improve the user experience, the actual protection is provided by the server, which returns a 403 response regardless of what the frontend does.

6.1.3 Staff

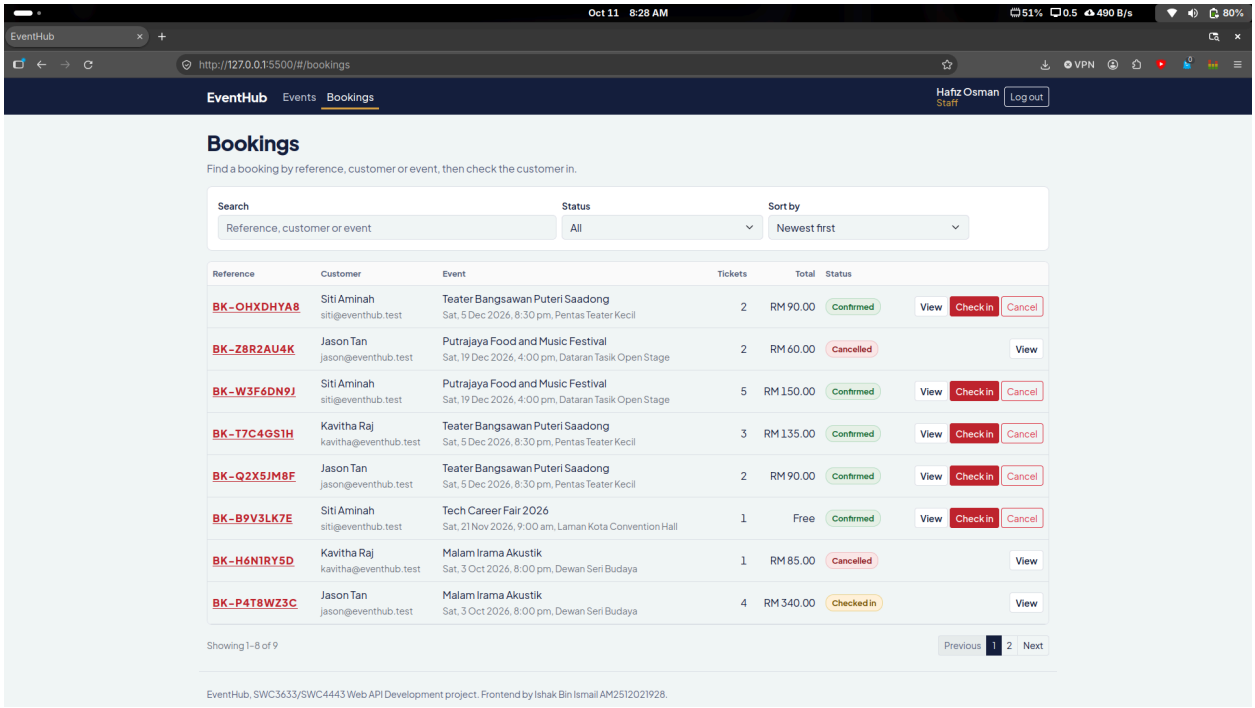


Figure 6.8: Staff Dashboard

Since Hafiz is logged in under the staff account, the navigation bar displays 'Bookings' rather than 'My bookings' and the table includes a Customer column showing each customer's name and email address. Confirmed bookings have the options 'View', 'Check in' and 'Cancel', whereas cancelled and checked-in bookings show only 'View'. The 'Previous', '1', '2' and 'Next' buttons at the bottom allow movement between the different pages of results from the API.

6.1.4 Organizer

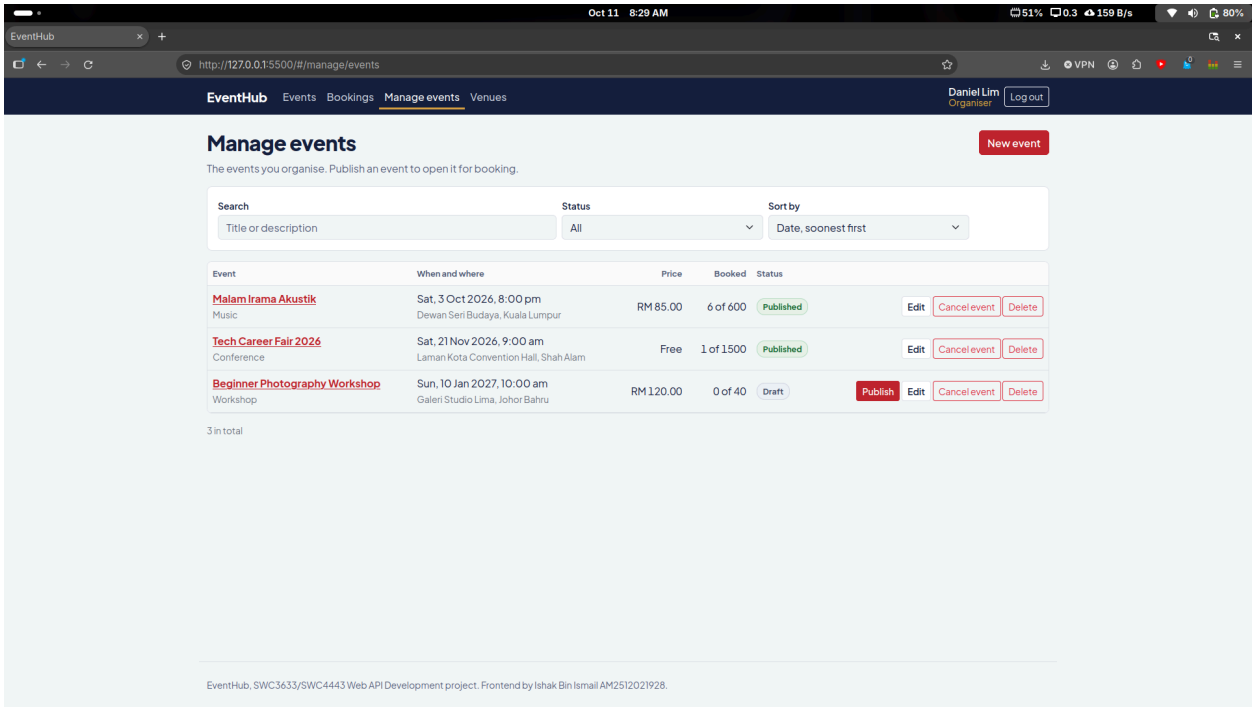


Figure 6.9: Organizer Dashboard

Daniel, who acts as an organiser, can see only the three events that he is responsible for, as they are listed in the response to the GET /events request. For each row displayed, the information shown includes the date, the venue, the price, the number of tickets that have been booked out of the total number available, and the current status. The draft version of the Beginner Photography Workshop has an additional 'Publish' button; when this button is clicked, a PUT request is sent to /events/{id} in order to set the status to 'published' so that customers can book it. For each row, both 'Cancel event' and 'Delete' buttons are provided, although the API still verifies these actions and, for instance, will not allow an event to be deleted if it has bookings.

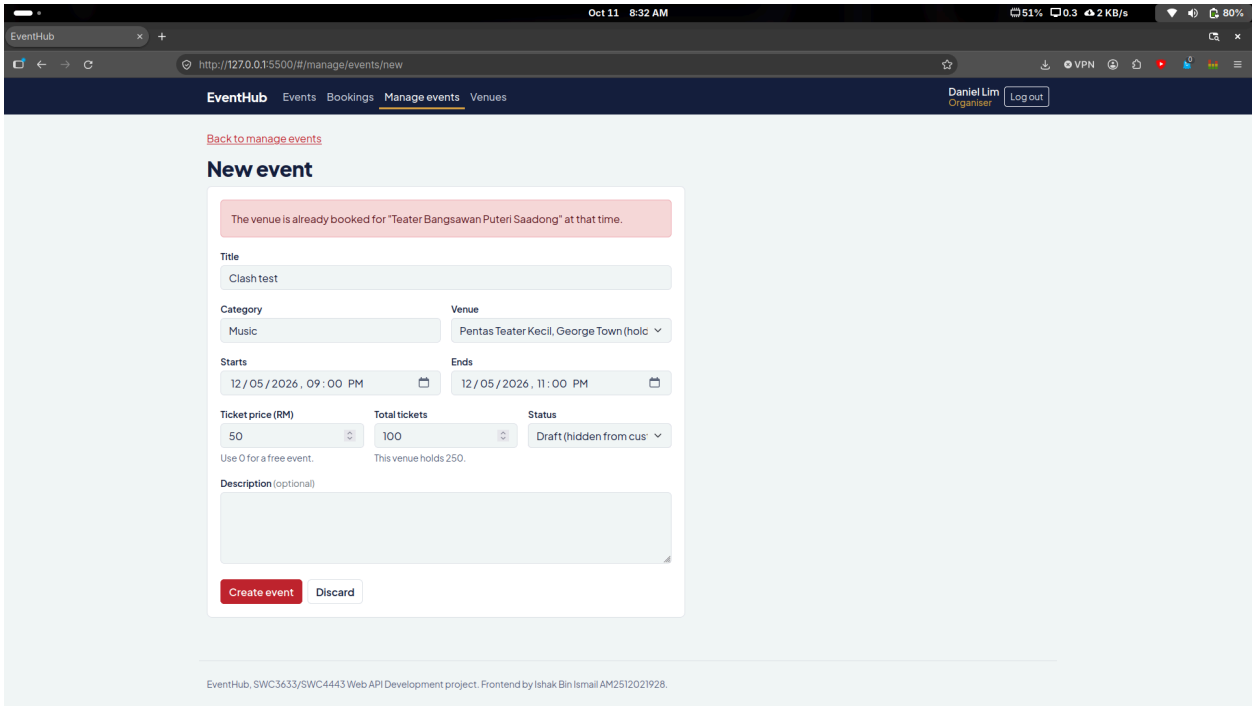


Figure 6.10: Create Event Page

The person arranging the "Clash test" at Pentas Teater Kecil on 5 December 2026 from 9:00 pm to 11:00 pm finds that this clashes with Teater Bangsawan Puteri Saadong which is already booked from 8:30 pm to 10:30 pm. Although the form is valid, the API responds with a 409 Conflict message and displays this message in a red box at the top of the form, indicating the event that is occupying the time slot. The hint under 'Total tickets' ('This venue holds 250') is derived from the venue's data and thus allows the organiser to see the capacity limit before the form is saved.

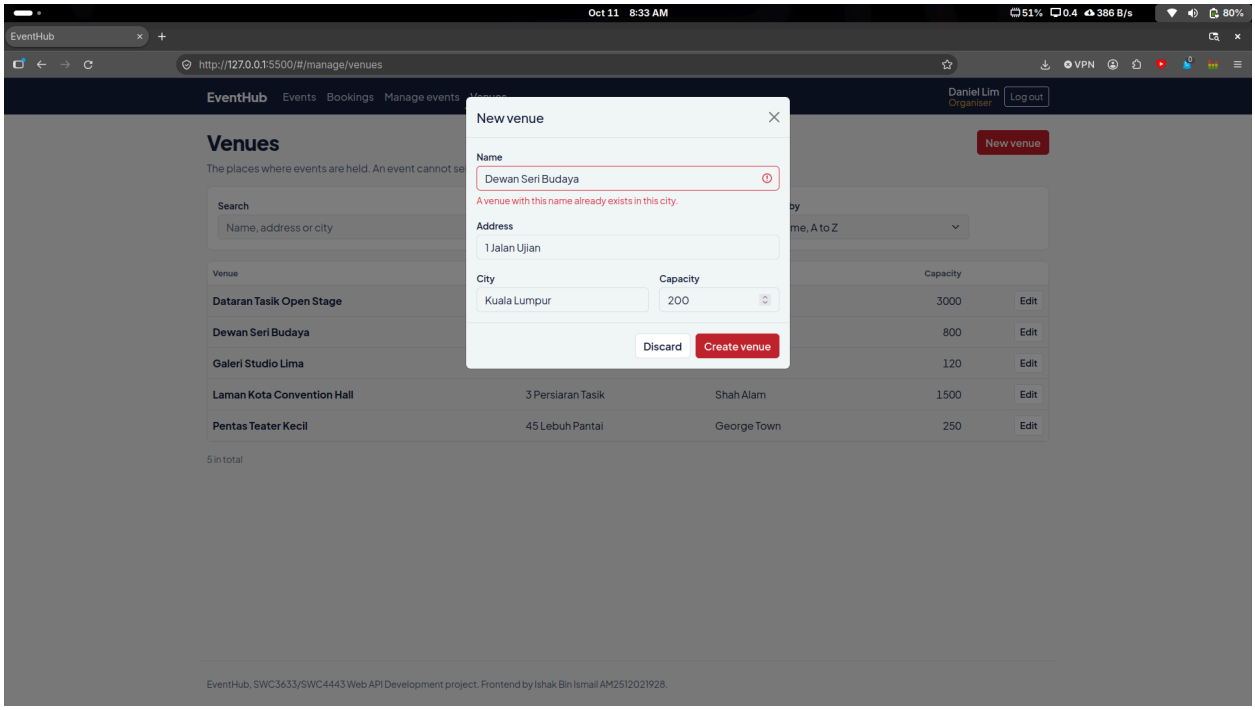


Figure 6.11: Create Venues Page

When the dialog for the new venue is displayed it appears above the list of venues, this list showing the five seeded venues together with their capacities. Since Dewan Seri Budaya is already present in Kuala Lumpur the API's rule that requires a unique name rejects the new venue, and as a result the message appears next to the Name field while the dialog remains open, allowing the organiser to correct the name without having to retype the other details.

6.1.5 Admin

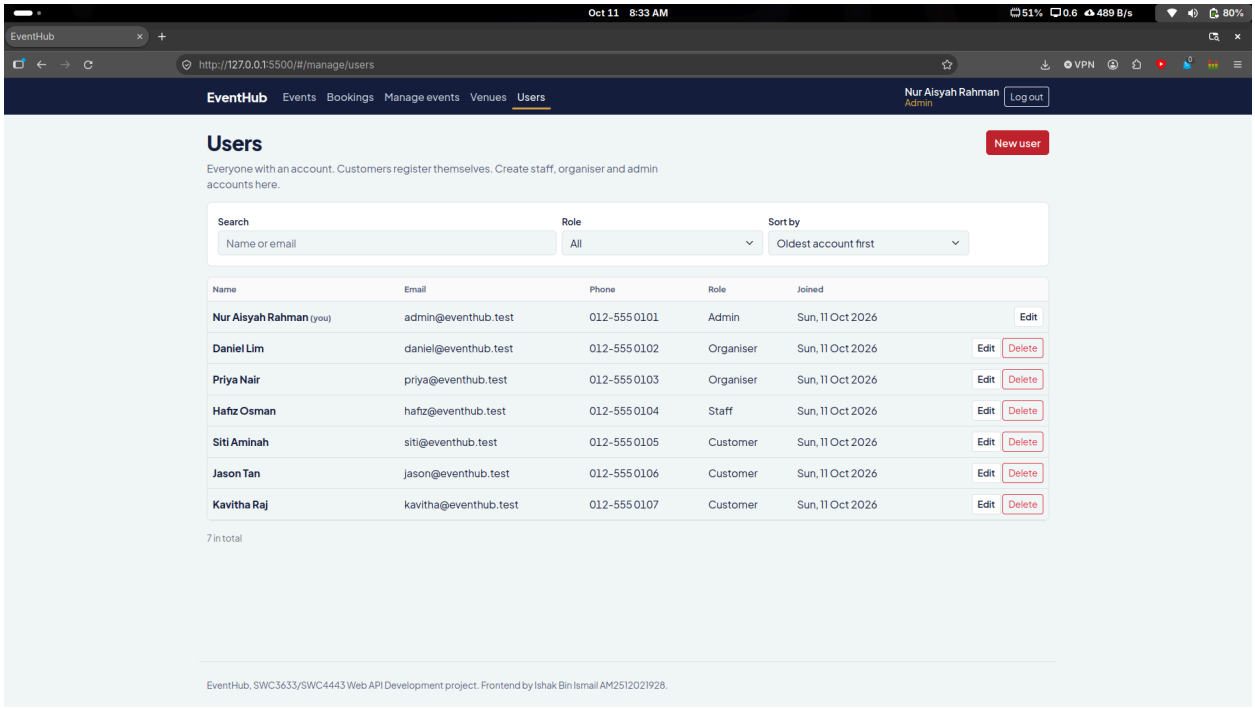


Figure 6.12: Admin Dashboard

Since Nur Aisyah is logged in with administrative privileges, the navigation bar displays all the pages, one of which is the Users page. The table contains all seven of the seeded accounts, showing for each account the name, email address, phone number, role and the date they joined; the row corresponding to the admin herself is indicated by "(you)". The search function, the role filter and the sort order are sent to the URL GET /users as query parameters, and the 'New user' option enables the admin to create staff, organiser and admin accounts, something that customers cannot achieve via the registration process.

6.1.6 Frontend and Backend Interaction

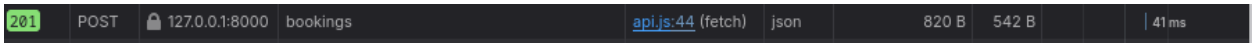


Figure 6.13: Frontend and Backend Interaction

The evidence that the frontend and backend are separate programs communicating over HTTP is shown in this screenshot: it displays the status (201), the method (POST), the address and path of the API, the piece of code which initiated the request, the JSON response type and the time taken, all of which correspond to the booking request.

6.2 Member: Ahmad Naufal bin Ahmad Fuad (frontend-naufal)

6.2.1 Guest

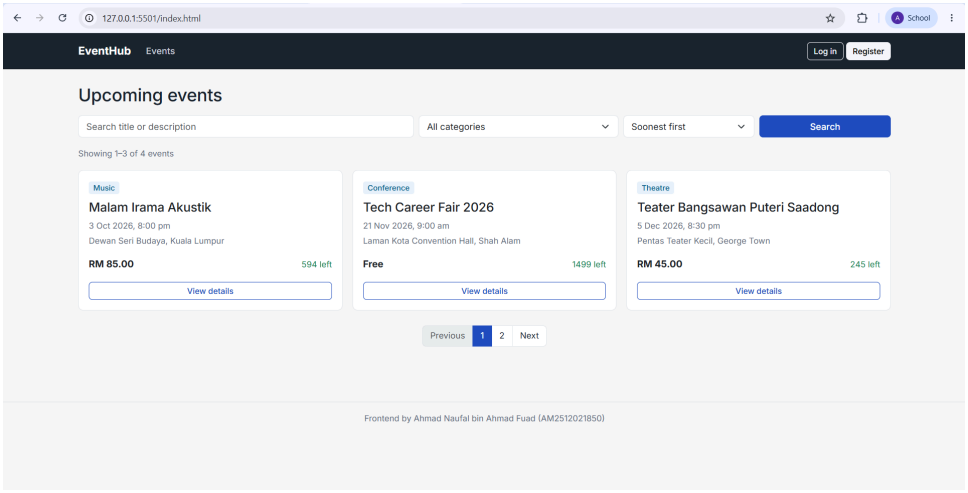


Figure 6.14: Guest Dashboard

The event list is available without logging in. The search box, category filter/sort option are passed to GET /events as query parameters and the API provides a single page of events that match the search. The paginator buttons are created from the current_page and last_page of the response.

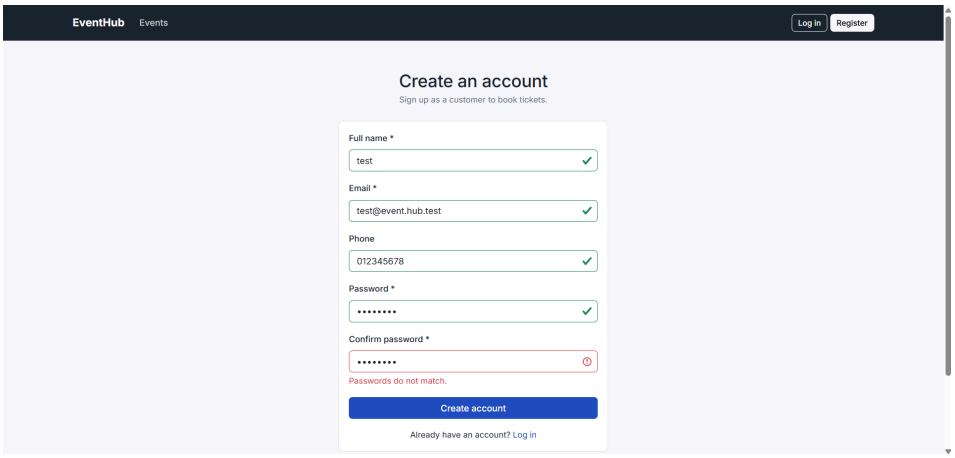


Figure 6.15: Register form

Before any request is sent, the browser checks required fields, the email format, a minimum password length of 8 characters and that both passwords match. All data submitted to POST /auth/register is valid and will create a customer account and return a JWT to the user to log them in on the spot.

6.2.2 Customer

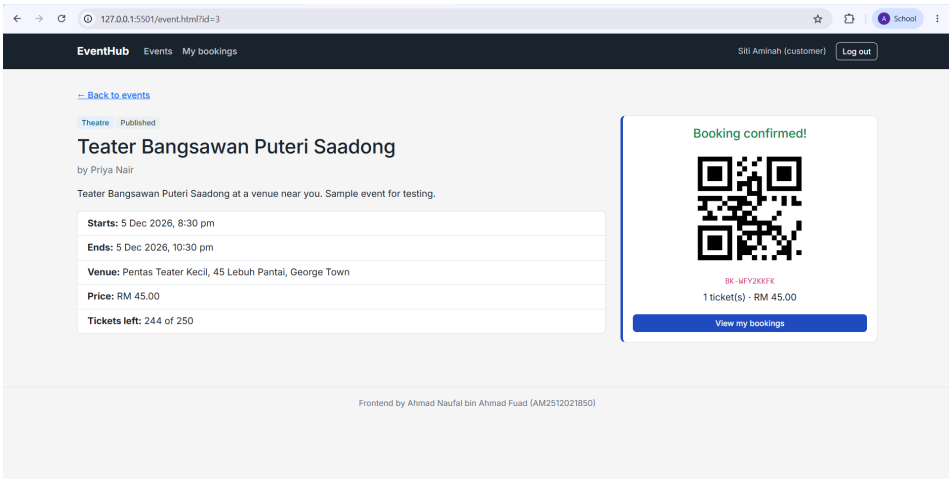


Figure 6.16: Booking confirmation with QR

The customer selects a quantity and the page posts /bookings. If the API returns 201, the form is replaced with a confirmation card with the booking reference and the total price calculated by the server. The QR code is then downloaded from GET /bookings/{id}/qr from the backend that fetches the QR code from the goQR.me API.

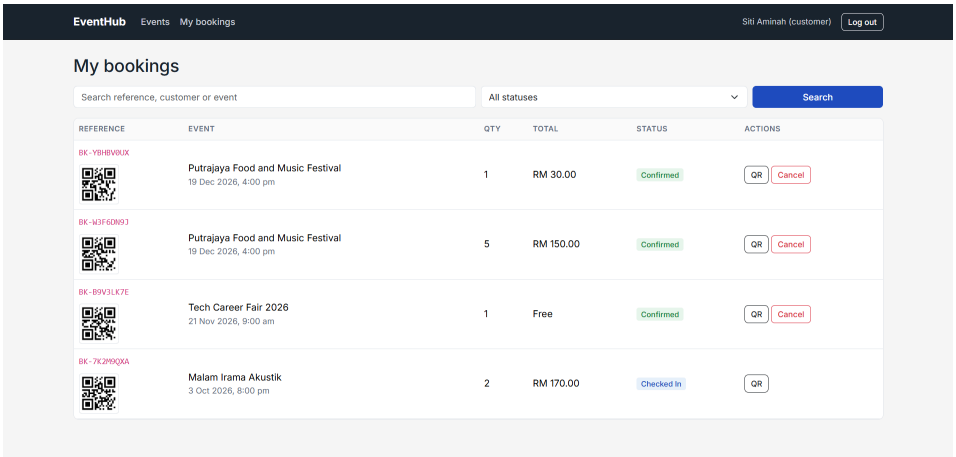


Figure 6.17: Booking confirmation with QR

The API filters results by the user and only shows a user's bookings to them. Each active booking is displayed immediately as a QR code, which when clicked opens a larger version of it. Tickets can be cancelled before the event which will return them to the event.

6.2.3 Staff

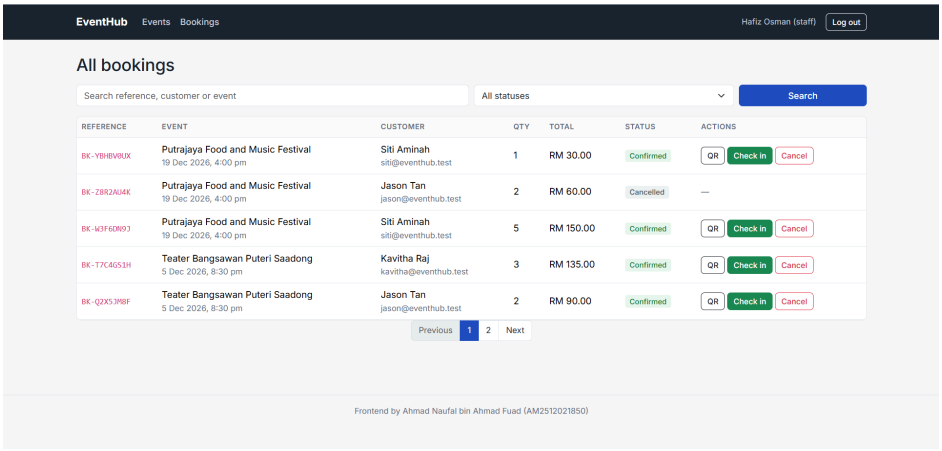


Figure 6.18: Staff checking customer

Staff are using the same bookings page, but the API returns all bookings and the page displays the customer column and Check in button. Once confirmed, the page posts PUT /bookings/{id} with {"status": "checked_in"} to the page. The success message comes from the API's response, and the list reloads with the new status.

6.2.4 Organiser

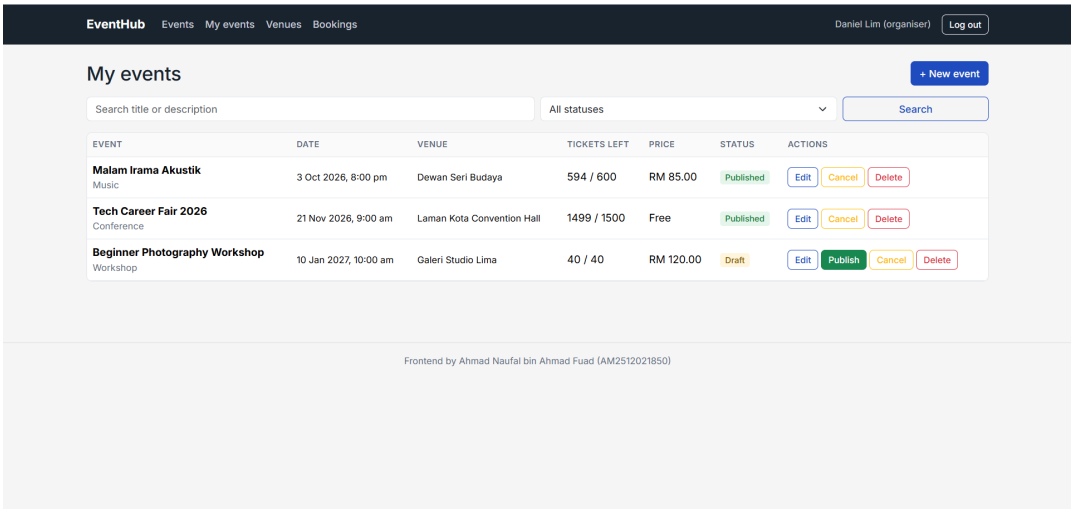


Figure 6.19: Organiser's event management

A new event appears in the event list only when it is sent to the server with a PUT /events/{id} request containing a JSON body with a "published" status.

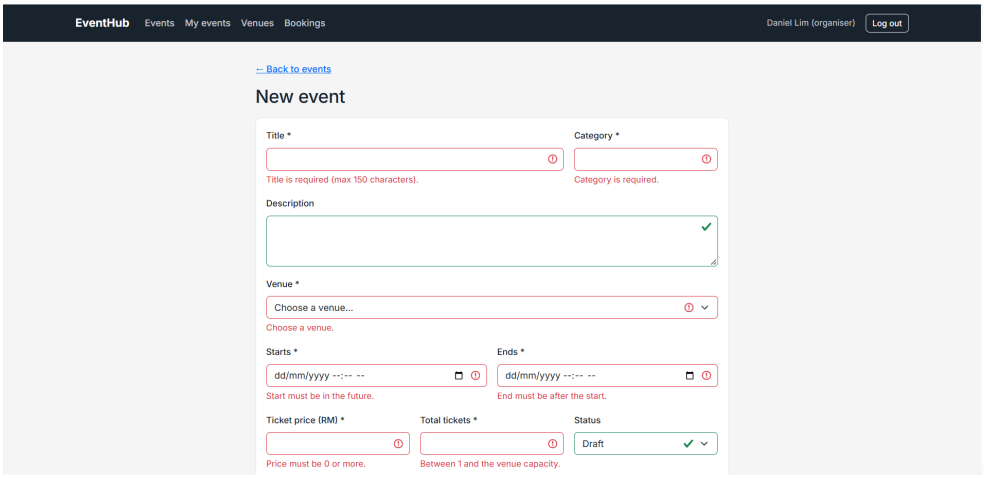


Figure 6.20: Validation event form

The event form requires all the required fields to be completed, the end time is after the start time, and that the ticket total is not over the selected venue's capacity. If the API still rejects the data, its 422 field errors are shown under the matching inputs.

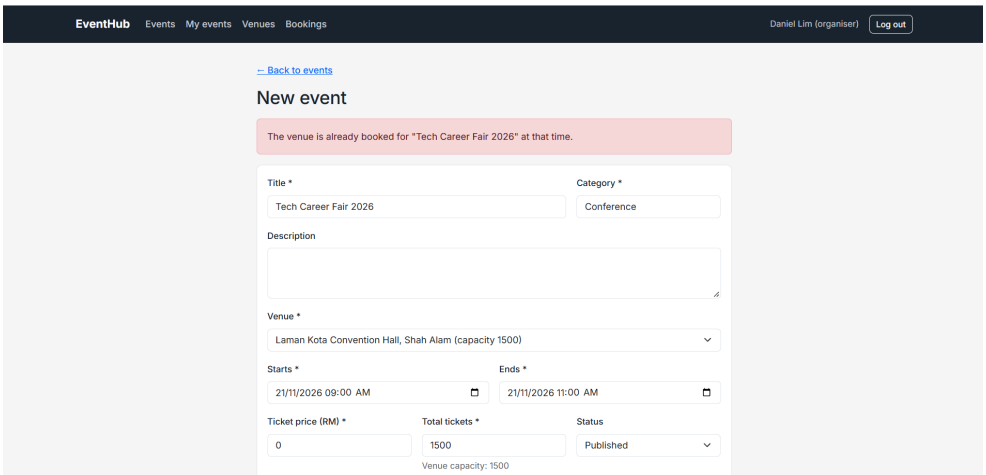


Figure 6.21: Ruled Conflict

If an organiser attempts to delete an event that has bookings, the API will return 409 and the frontend will show its message. This is a booking history rule: events must be cancelled, and the booking is also cancelled, returning the tickets.

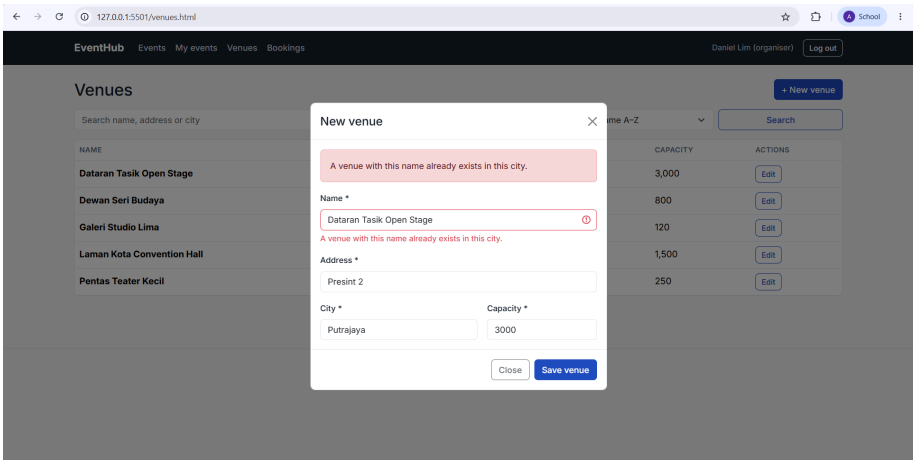


Figure 6.22: Venue form

Venues are created and edited in a pop-up form that sends POST /venues or PUT /venues/{id}. Errors (such as duplicate venue name in same city - 422, capacity less than existing event's ticket total 409) are displayed within the popup to allow the user to fix the error without losing the input.

6.2.5 Admin and access control

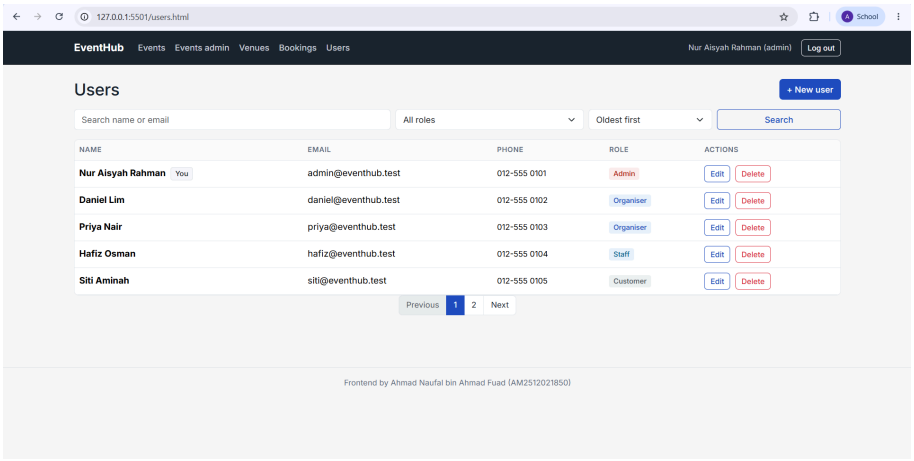


Figure 6.23: Admin user management

Only admins can open the Users page, which supports searching, filtering by role, and creating, editing and deleting users through /users. The API also has safety rules, e.g. that an admin cannot set his/her own role to something else, and the frontend presents these safety rules in the form.

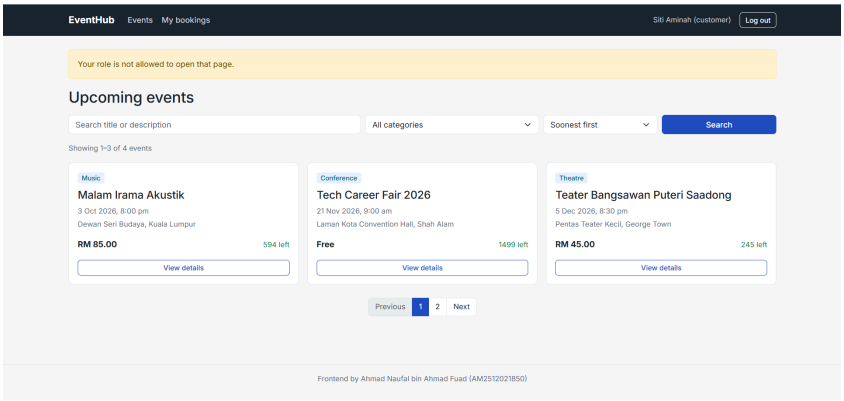


Figure 6.24: Role guard

All protected pages include requireRole() prior to data loading. If an organiser attempts to open the Users page (which is only available to administrators), it redirects the user to the list of events with a warning message. Navigation bar also only displays links for pages that user's role can open. The same restriction is applied on the backend and would return 403 status for the request.

6.3 Member: Muhammad Anas Bin Haddri (frontend-anas/)

The screenshots below show one user going through every role, in the same order as my demo video. Each line in square brackets is where a screenshot goes. Replace it with the picture and keep the caption under it.

6.3.1 Guest

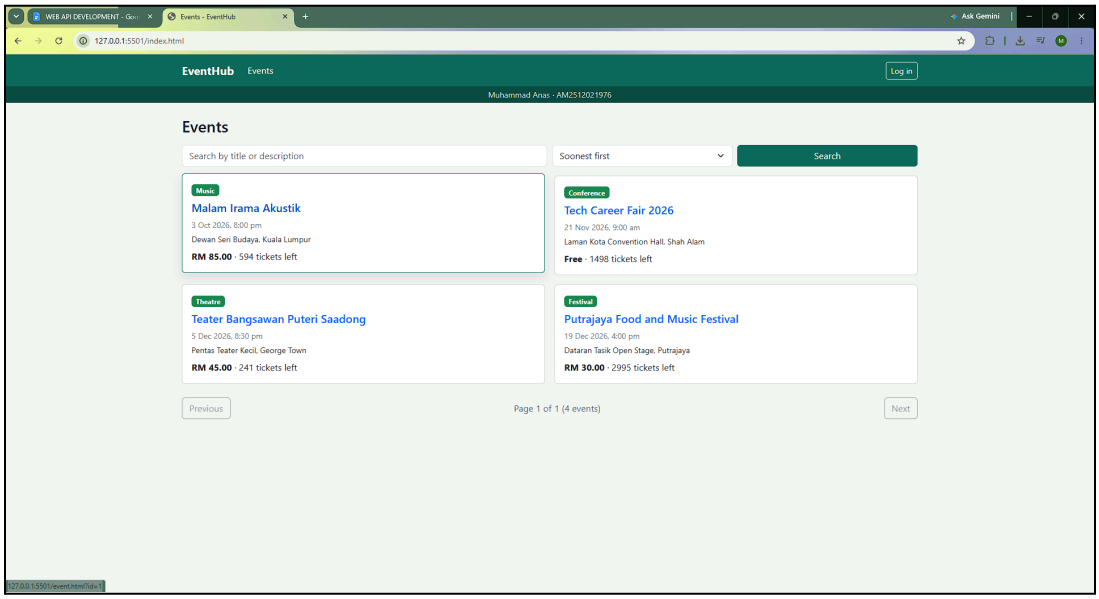


Figure 6.25: Event list for a guest

Guests can look at events without an account, so the navbar only shows Events and Log in. The search box and sort choice are sent to GET /events in the URL, and the API sends back six published events per page. Each card shows the category, date, venue, price and tickets left. The user can click anywhere on the card, and it moves up when the mouse is over it. The text under the list, like "Page 1 of 1 (4 events)", comes from the page details in the API's reply.

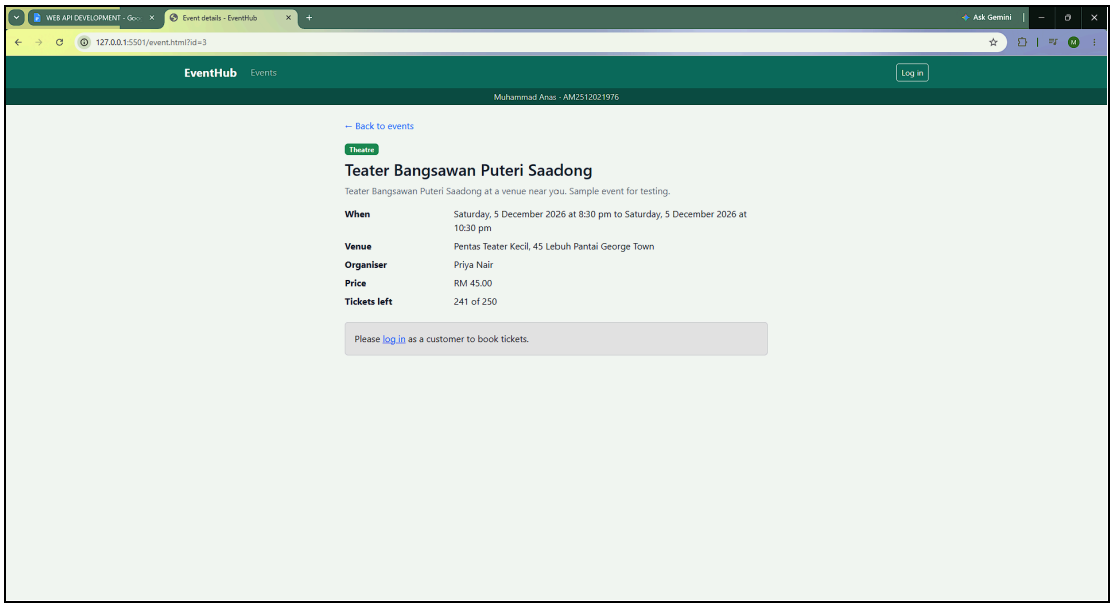


Figure 6.26: Event details for a guest

The page gets the event with GET /events/{id}. A guest can see all the details but cannot book. Instead of the booking form, a box asks them to log in as a customer. Staff, organisers and admins see a box saying only customers can book.

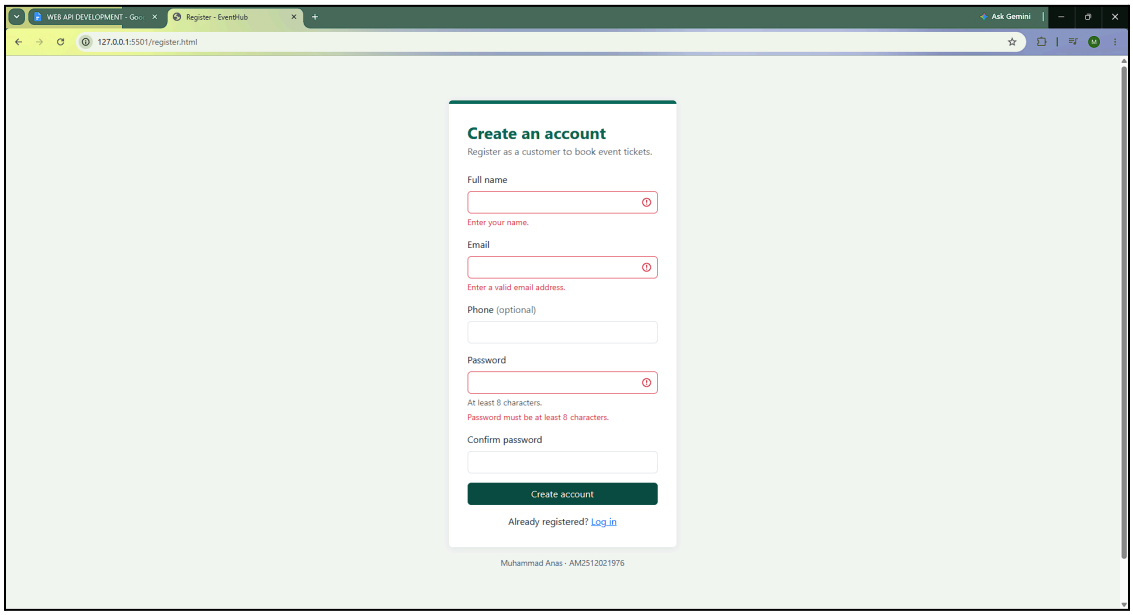


Figure 6.27: Sign-up form checked in the browser

The form checks the name, the email format, a password of at least 8 characters and that both passwords match, before sending anything. If everything is correct, POST /auth/register creates a customer account and sends back a token. The new user is logged in straight away and taken to the events. If the email is already used, the API's 422 message "The email has already been taken." shows under the Email field.

6.3.2 Customer

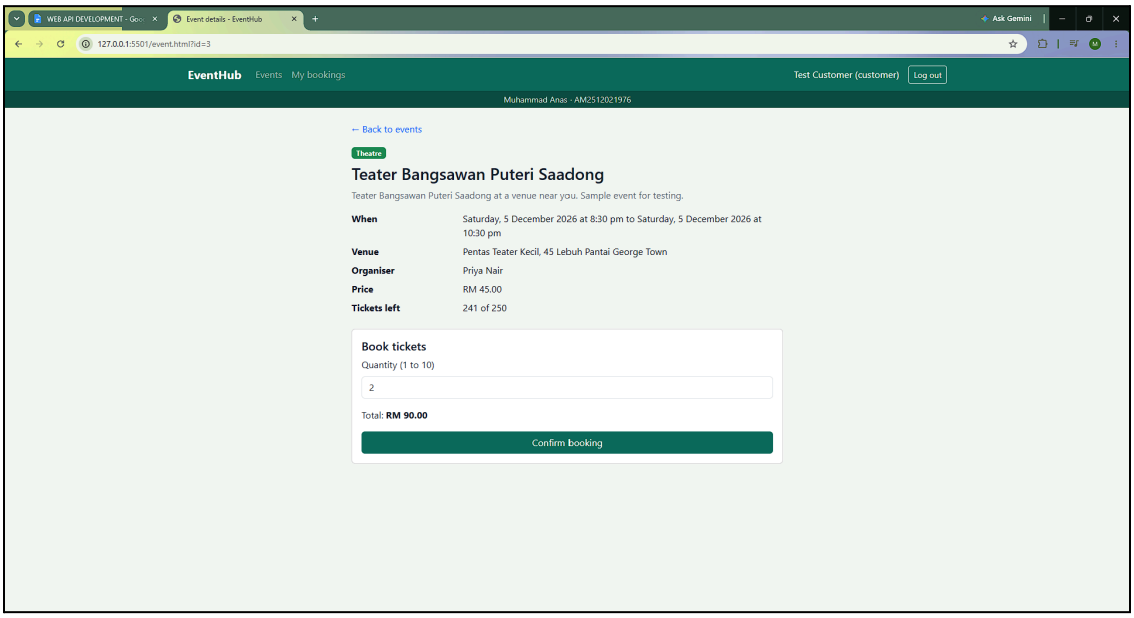


Figure 6.28: Booking form with a live total

The customer chooses how many tickets they want, and the total changes as they type. This total is only a preview, because the server works out the real total. When they click Confirm booking, the page asks "Book 2 ticket(s) for Teater Bangsawan Puteri Saadong?" and then sends POST /bookings.

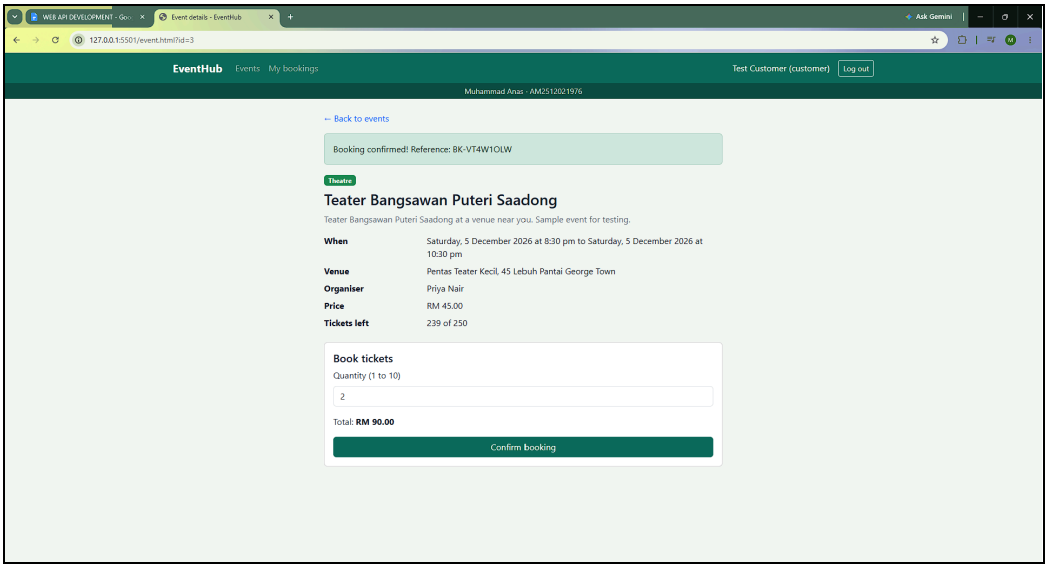


Figure 6.29: Booking confirmed

The API replies 201 with the new booking, and the page shows the booking reference. It then calls GET /events/{id} again in the background, so the tickets left go down from 245 to 243 without reloading the page.

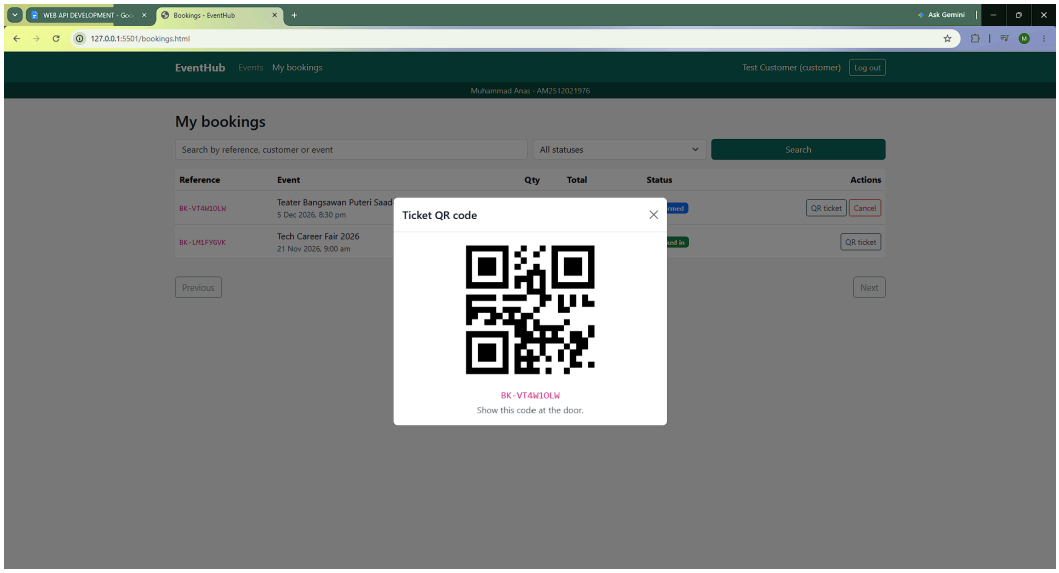


Figure 6.30: My bookings and the QR ticket

The page title is "My bookings". It only shows this customer's bookings, because the API only returns the bookings of the logged-in user. The QR ticket button calls GET /bookings/{id}/qr. The backend gets the picture from goQR.me, and the pop-up shows it with the booking reference under it. Confirmed bookings also have a Cancel button, which sends PUT /bookings/{id} with the status cancelled.

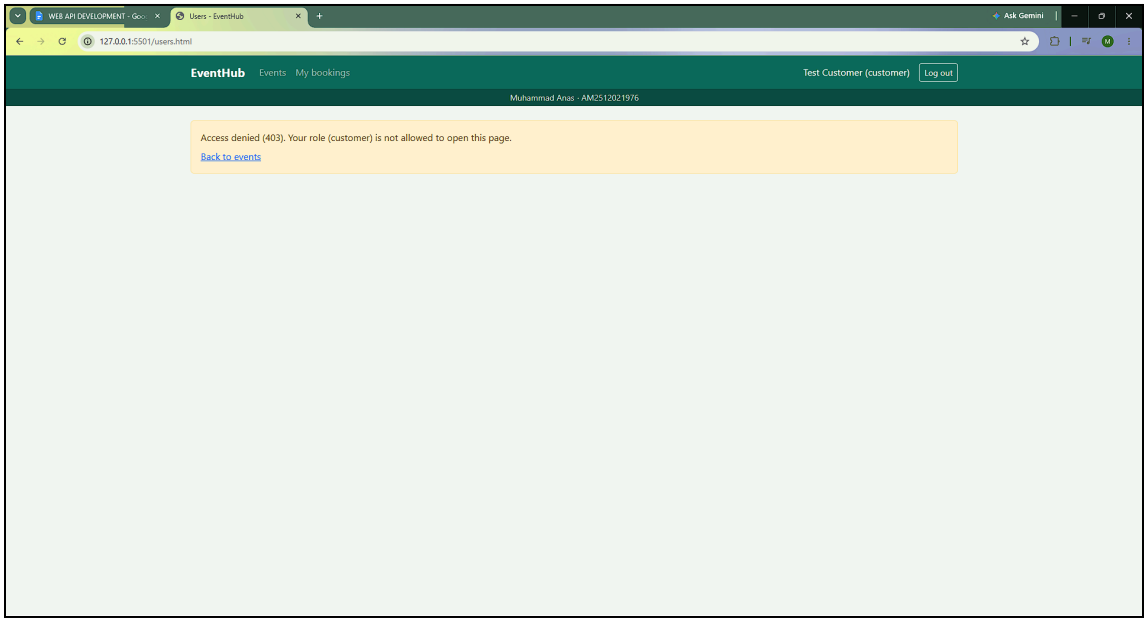


Figure 6.31: Role check on an admin page

requireRole(['admin']) runs before the page loads any data. The customer does not see a blank page. They see their role and a link back to the events. The API would also refuse the request with 403, so this check is to help the user, not to protect the data.

6.3.3 Staff

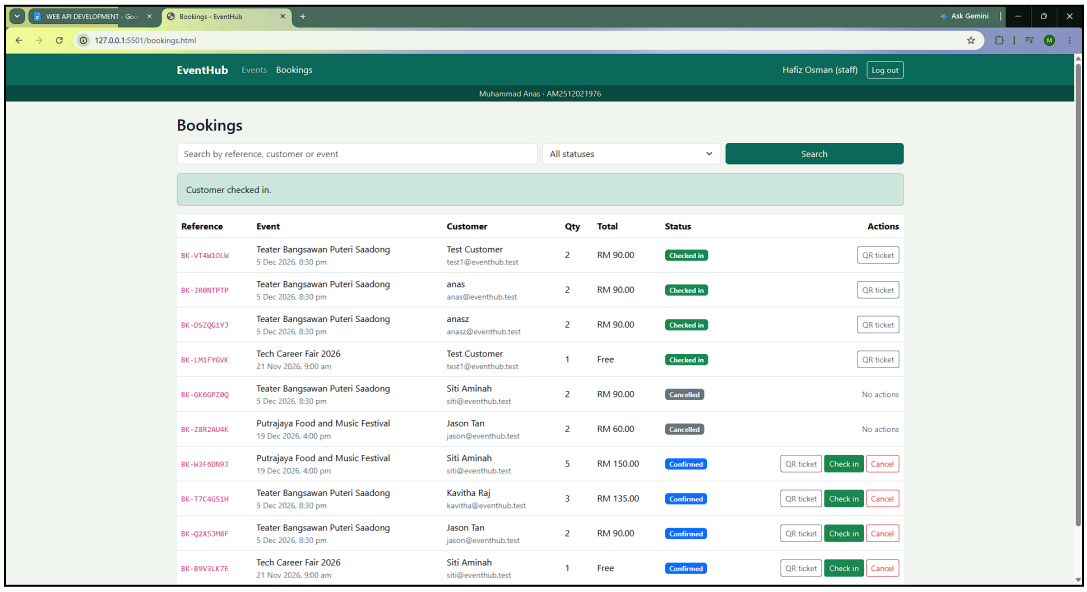


Figure 6.32: Staff checking a customer in

Staff use the same bookings page. The API sends back every booking, and the page adds a Customer column with name and email. Staff can search by reference, customer or event. Check in sends PUT /bookings/{id} with {"status": "checked_in"}. The success message comes from the API. The list then reloads with the new badge, and the Check in button is gone.

6.3.4 Organiser

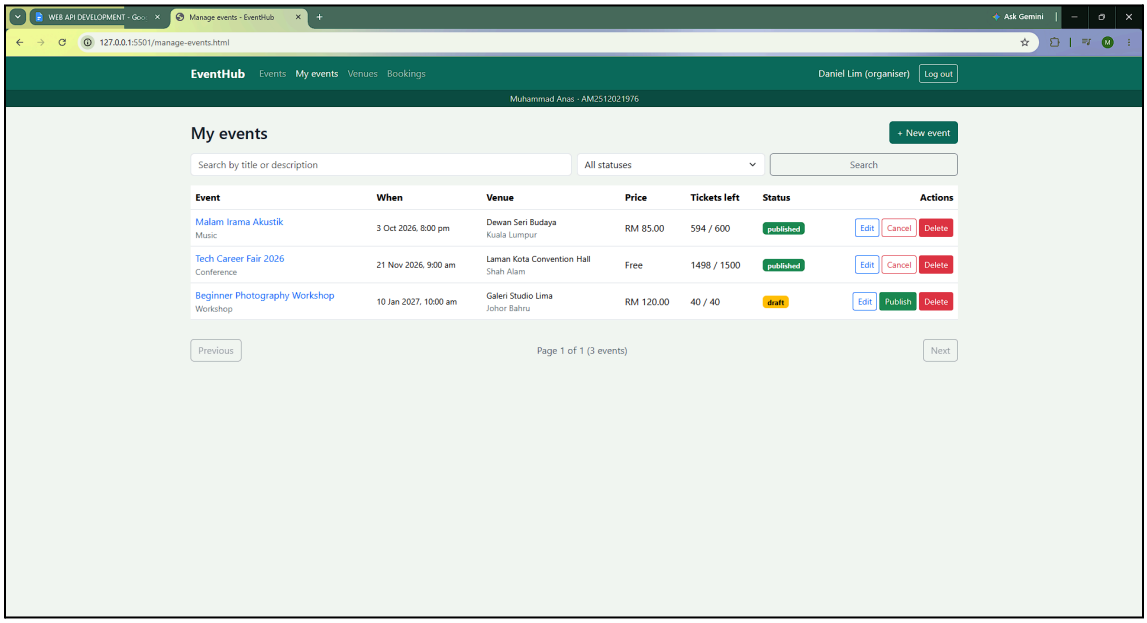


Figure 6.33: Organiser's My events page

Daniel is logged in as an organiser, so the navbar shows Events, My events, Venues and Bookings. The page is called "My events" and only shows his three events, because the page sends organiser_id with GET /events. Each row shows the date, venue, price, tickets left out of the total, and a coloured status badge. The buttons change with the status:

• Published events (Malam Irama Akustik and Tech Career Fair 2026) have Edit, Cancel and Delete.

• The draft event, Beginner Photography Workshop, has Publish instead of Cancel. Publish sends PUT /events/{id} with {"status": "published"}, so customers can then see and book it.

The search box and status filter are sent to the API, and the text "Page 1 of 1 (3 events)" comes from the page details in the reply. The API still checks every action. For example, it will not delete an event that has bookings.

6.3.5 Admin

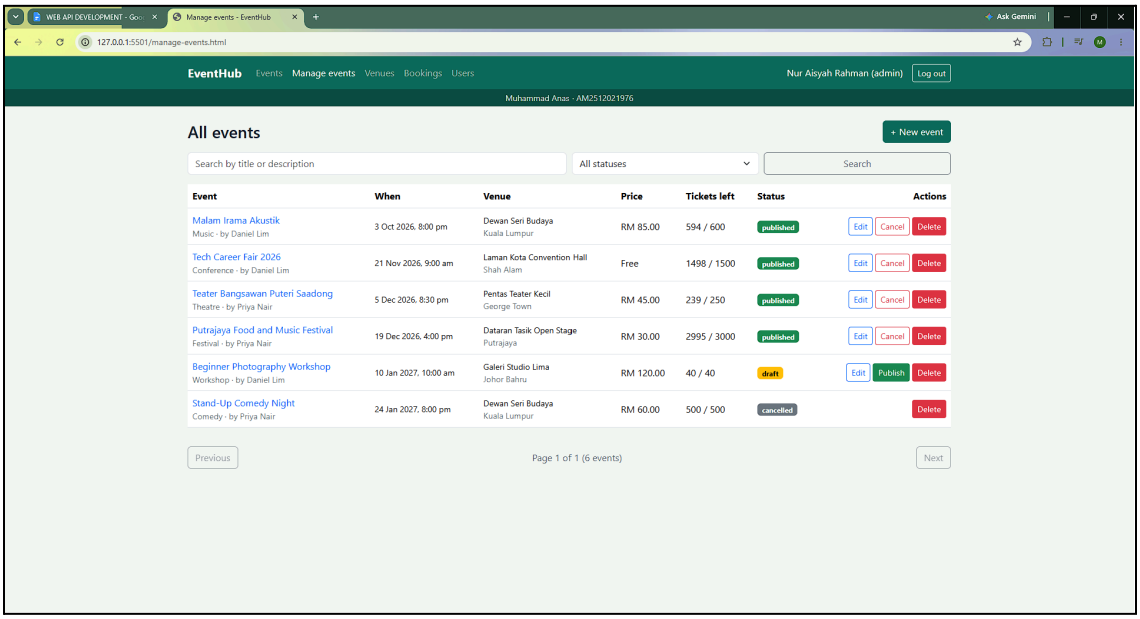


Figure 6.34: Admin sees all organisers' events

The admin's navbar shows every page. On Manage events, the title is "All events", and each row shows which organiser owns it. The admin can edit, publish, cancel and delete any event.

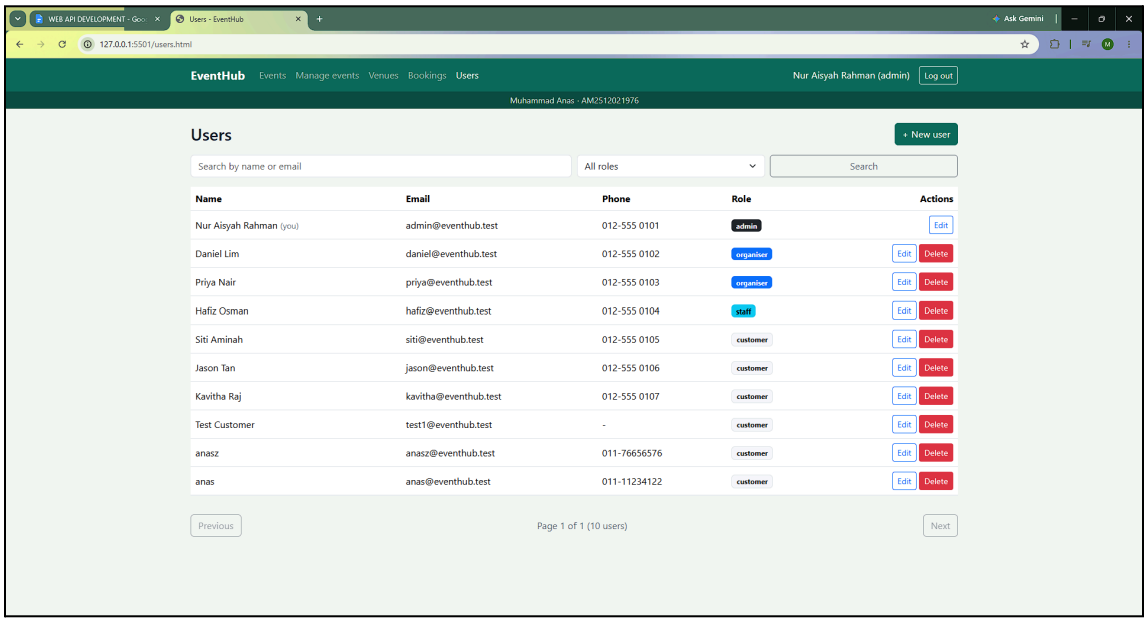


Figure 6.35: Managing users

The Users page lists accounts from GET /users, with search and a role filter. New user and Edit open a pop-up that sends POST /users or PUT /users/{id}. The admin can give any role, which customers cannot do when they sign up. The admin's own row has no Delete button, and the API also refuses with "You cannot delete your own account."

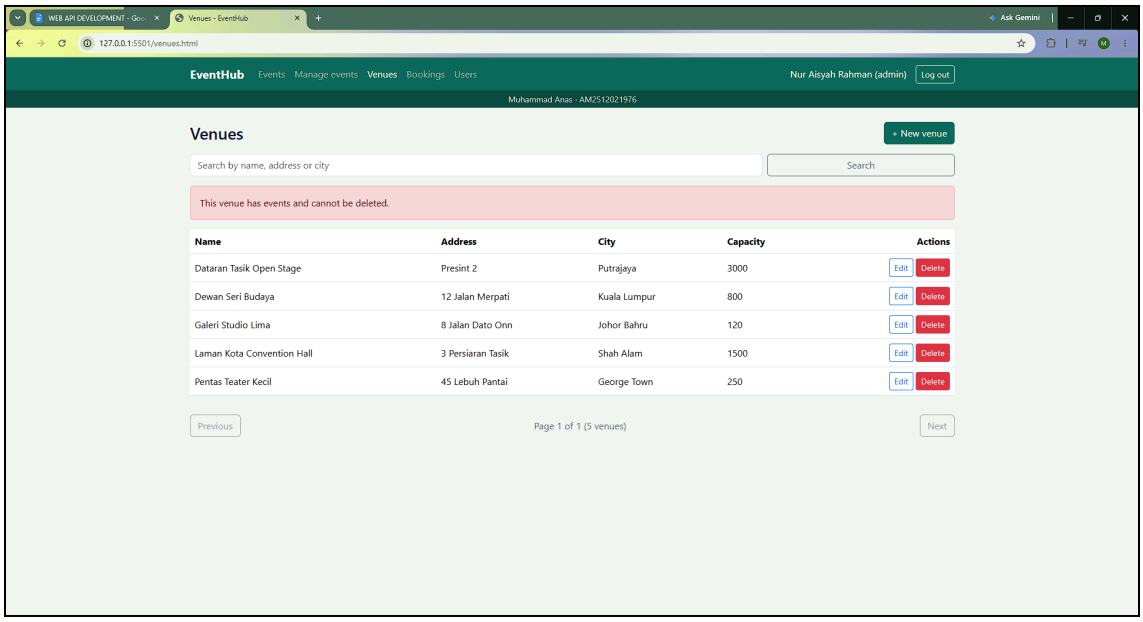


Figure 6.36: Rule for deleting a venue

Only admins see the Delete button for venues. After "Are you sure?", DELETE /venues/{id} returns 409: "This venue has events and cannot be deleted." The page shows this message above the list.

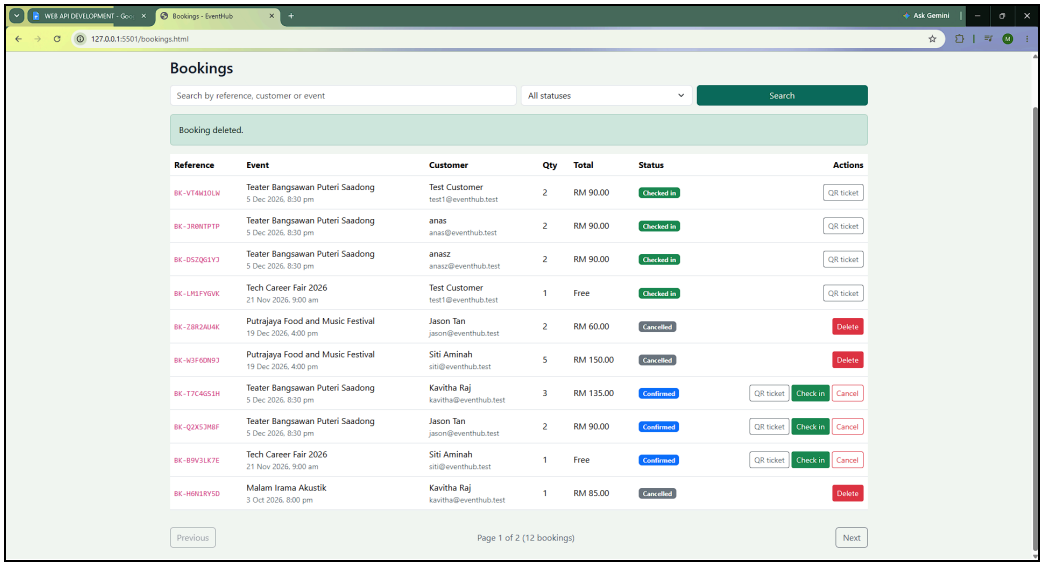


Figure 6.37: Deleting a cancelled booking

Only the admin has a Delete button, and only for cancelled bookings. After the admin confirms, DELETE /bookings/{id} removes the booking, and the success message comes from the API.

6.3.6 Frontend and backend working together

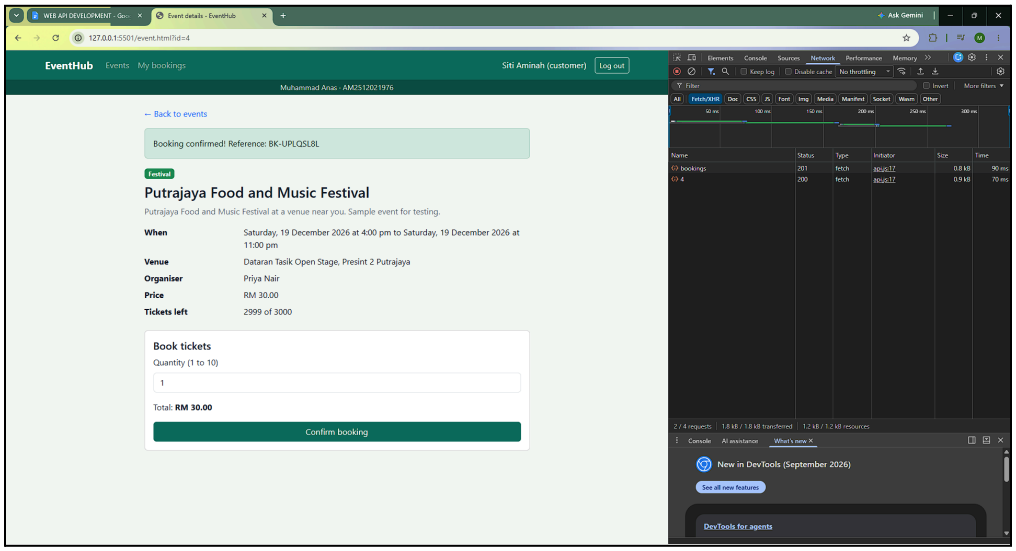


Figure 6.38: The booking request in the Network tab

This shows that the frontend and the API are two separate programs that talk over HTTP. Siti books 1 ticket for Putrajaya Food and Music Festival. The Network tab shows two requests:

• bookings: the page sent POST /bookings to the API on port 8000. The API replied 201 Created in 90 ms. The Initiator column shows api.js:17, which is the fetch() line inside apiRequest() (Figure 5.13).

• 4: right after the booking, the page called GET /events/4 to get the new number of tickets. The API replied 200 OK.

On the left, the page shows the green "Booking confirmed! Reference: BK-UPLQSL8L" message, and the tickets left have gone down to 2999 of 3000 without reloading the page.

6.4 Member: Muhammad Shihabuddin Al-Fikri Bin Anuar (frontend-AI)

This section shows the main features of the EventHub system through screenshots of the running frontend. The screenshots demonstrate how users interact with the system and use its features.

6.4.1 Main Page and Event Listing

The main page displays the available events with details such as the event name, date, venue and ticket price. Users can browse the events and use the search and filter functions to find events they are interested in.

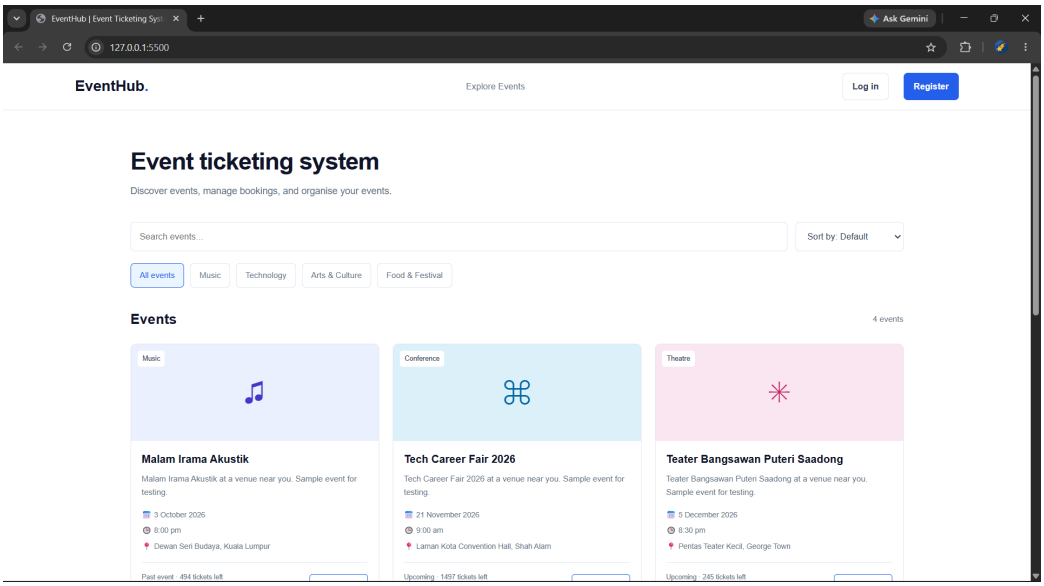


Figure 6.1: EventHub Main Page and Event Listing

6.4.2 Login

Users can log in to their accounts to access the features available for their roles. After a successful login, the navigation menu changes based on the user's role.

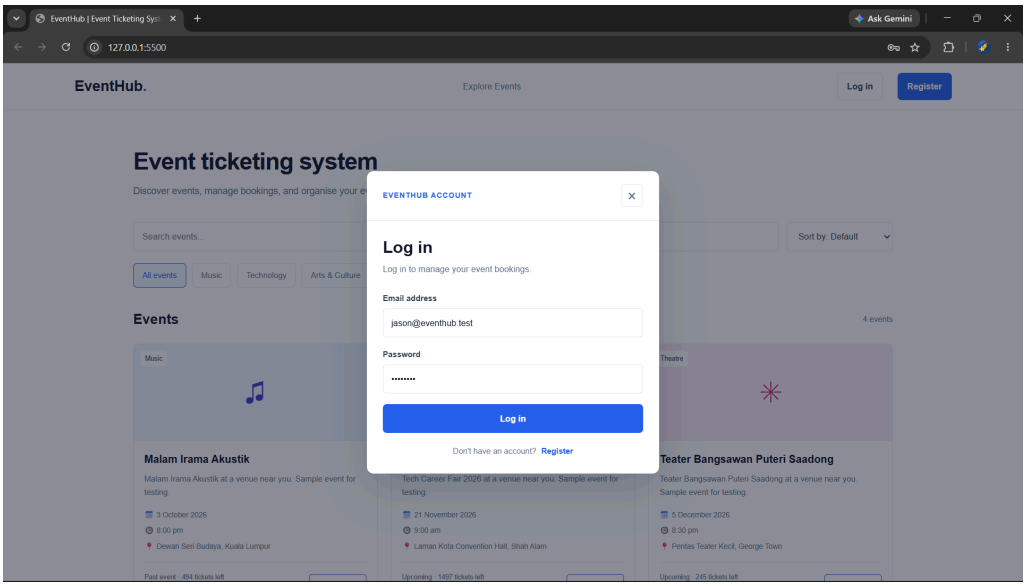


Figure 6.2: Customer Login and Role-Based Navigation

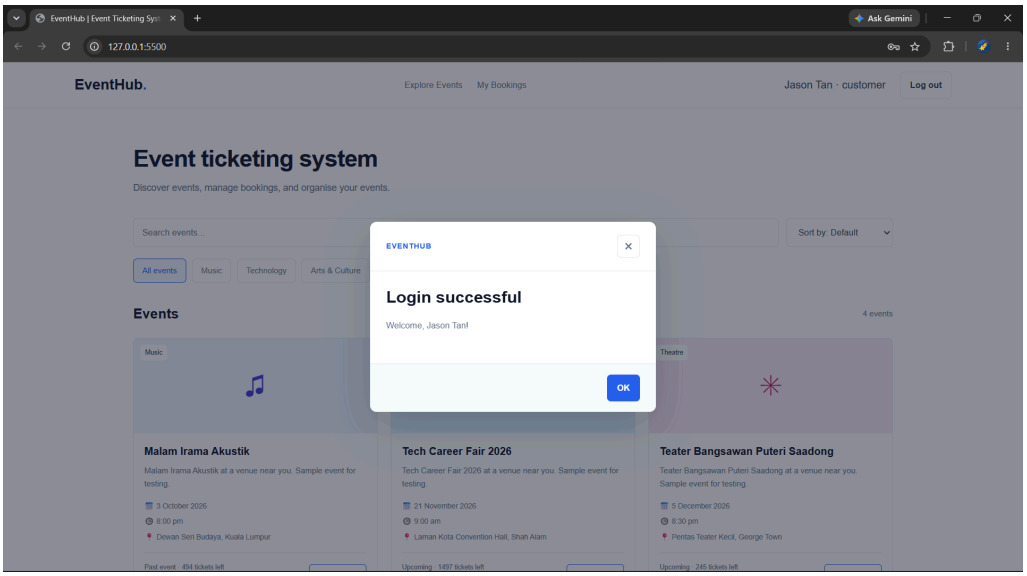


Figure 6.3: Successful Ticket Booking Confirmation

6.4.3 Event Booking and QR Code

Customers can select an event, choose the number of tickets and submit a booking. After the booking is successful, the system displays the booking details. Customers can also view the QR code for their tickets.

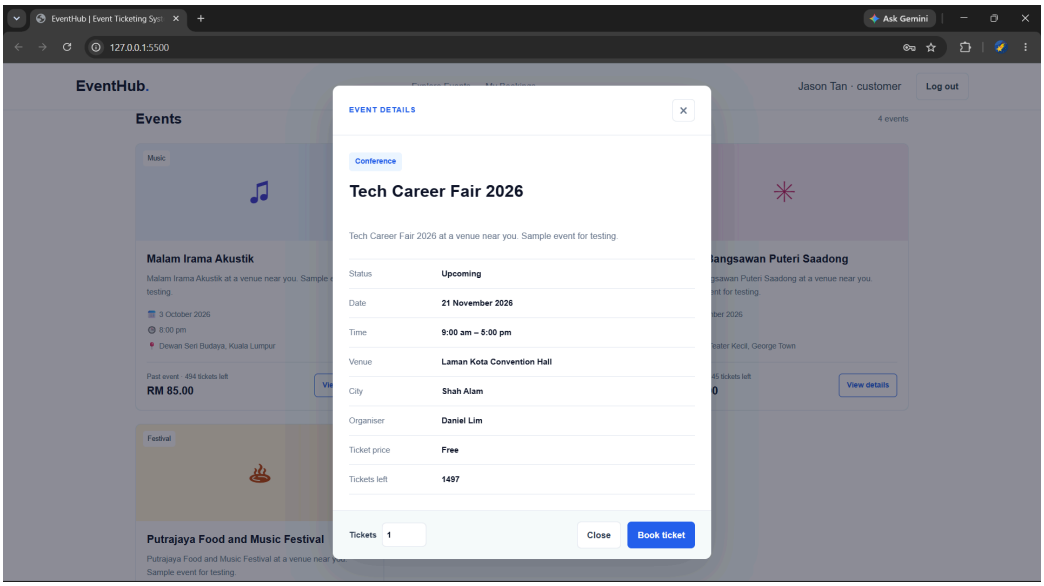


Figure 6.4: Book Ticket

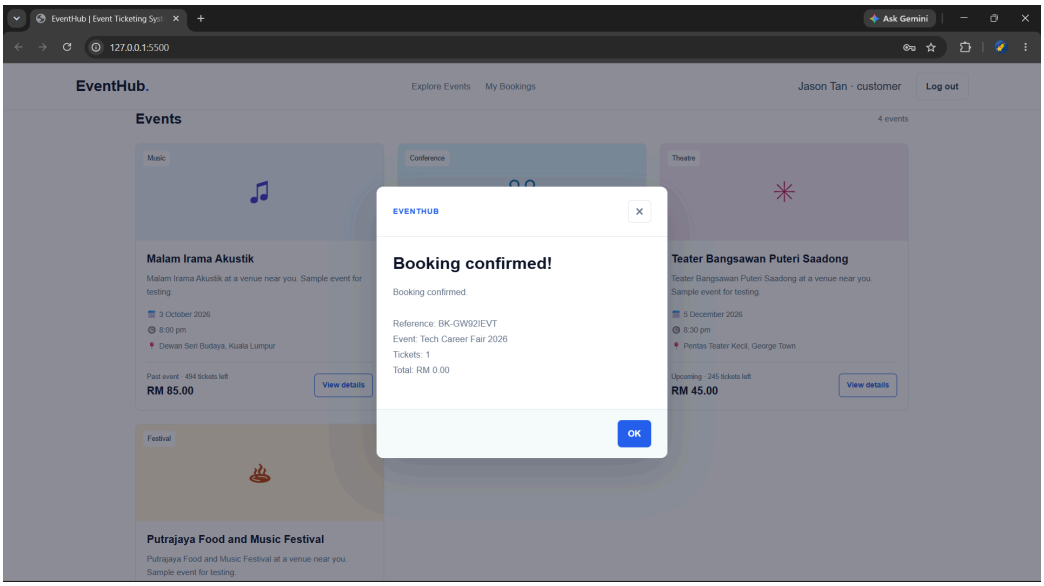


Figure 6.5: Booking Confirmation

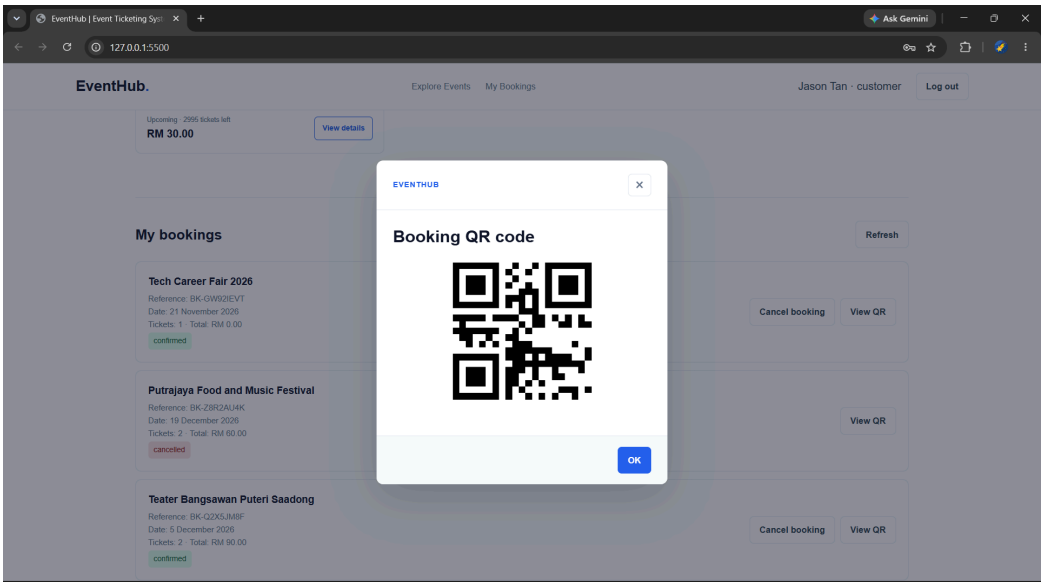


Figure 6.6: Booking Ticket QR Code

6.4.4 Management Functions

The frontend also provides management features for organisers and administrators. Organisers can manage events and venues, while administrators can manage users, events, venues and bookings.

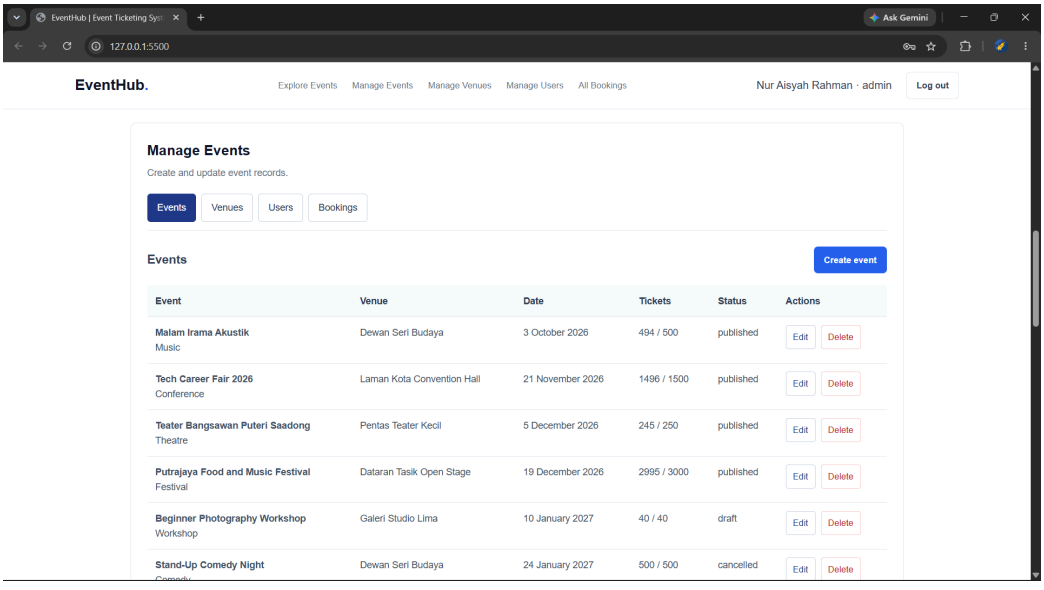


Figure 6.7: Event Management Interface

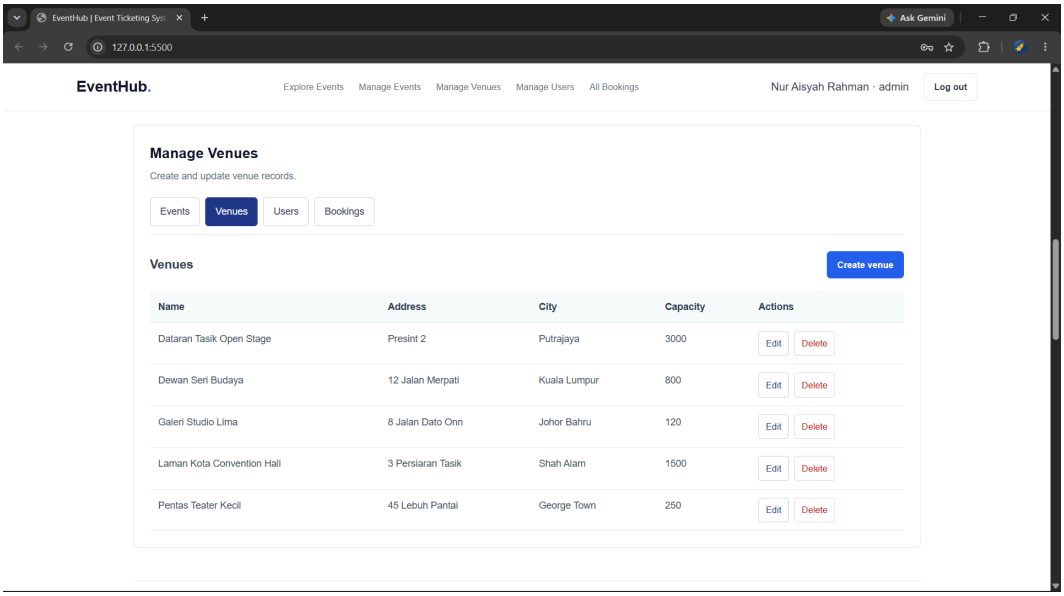


Figure 6.8: Venue Management Interface

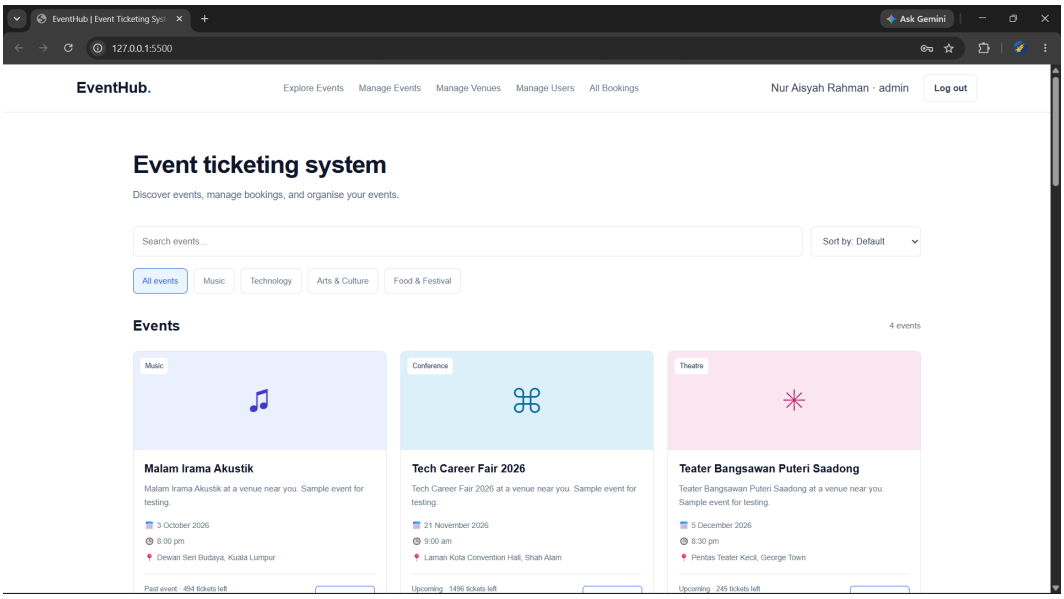


Figure 6.9: Administrator Management Dashboard

6.4.5 My Bookings

Customers can view their booking records, check booking details and cancel eligible bookings through the My Bookings page. The page also provides access to their ticket QR codes.

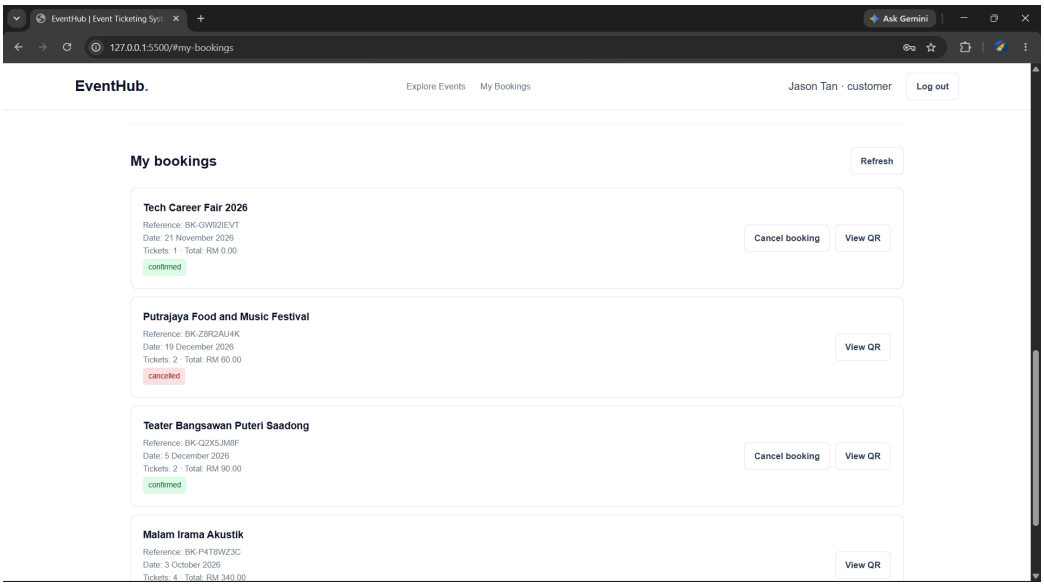


Figure 6.10: Customer Booking History and Management

7.0 Project Repository

The source code together with all the other supporting files is available at github.com/ishakismail7905/event-ticketing, which is a public repository. Since the entire project is contained in this one repository, the backend, the database scripts, the Postman collection, the documentation and each member's frontend can all be cloned and run together. The top level is divided into clear folders: backend/ for the shared Laravel API, database/ for the ERD and the SQL scripts, postman/ for the test collection, docs/ for the documentation, and one folder called frontend-<name> for each member. By having each frontend in its own folder, the members were able to work in parallel without altering each other's code, while all of them using the same backend. The table below indicates where each required item can be found.

Required Item	Location
Project report in PDF	docs/report.pdf
Entity Relationship Diagram	database/erd.png and database/erd.md
DDL and DML scripts	database/01_ddl.sql and database/02_dml.sql
Frontend source codes	frontend-ishak/, frontend-anas/, frontend-naufal/ and frontend-al/
Shared backend source code	backend/
Exported Postman collection	postman/EventHub.postman_collection.json
API documentation	docs/API.md
Setup and run instructions	README.md
Demonstration video	https://youtu.be/_c6sfGLXplU, also placed in README.md

7.1 Setup in Brief

The README lists all the steps in full, with the key ones summarised below. Once you have cloned the repository, you should run composer install in the backend/ directory in order to download Laravel and the other PHP packages. Then, copy .env.example to .env since this file contains the local settings and use the following commands to generate the secrets that the application requires: php artisan key:generate produces the application key and php artisan jwt:secret --force creates the key used for signing login tokens. Additionally, the EVENTHUB_API_KEY value, which is expected by the ticket check endpoint, must be set in that same file.

Now that the settings have been set up, set up a new MySQL database named eventhub and execute php artisan migrate:fresh --seed, thereby creating all the tables and filling them with the sample data. Lastly, start the API using php artisan serve and, in another terminal window, serve the frontend folder for example with php -S 127.0.0.1:5500 -t frontend-ishak, then open that address in the browser.

There is no actual secret kept in the repository; the sample accounts use a public test password; the app key, the token secret and the API key are located in backend/.env, a file that is ignored by Git.

7.2 Collaboration

The project was divided in such a way that each person had responsibility for one definite section of the repository. Ishak was in charge of the shared backend, the database scripts, the Postman collection and the documentation, while each member developed their own frontend in a separate frontend-<name>/ folder using the same API. Since each frontend was in its own folder, the members were able to work simultaneously without overwriting one another's code.

The frontends were introduced via GitHub pull requests rather than being pushed directly to main; the members worked on their individual branches (frontend-anas, frontend-naufal and svprianto) and then submitted a pull request to merge each one, thus ensuring that each change could be traced back to its author. From 11th to 14th October 2026 the repository had 11 pull requests merged and 35 regular commits, making a total of 47 commits. In the case where two branches both modified README.md, the conflict was settled in a separate merge commit before the pull request was merged.

Member	GitHub account	Area of work
Ishak Bin Ismail	ishakismail7905	backend/, database/, postman/, frontend-ishak/, README.md
Muhammad Anas Bin Haddri	mcZenz	frontend-anas/, README.md
Ahmad Naufal Bin Ahmad Fuad	opale0108	frontend-naufal/, README.md
Muhammad Shihabuddin Al-Fikri Bin Anuar	svprianto	frontend-al/, README.md

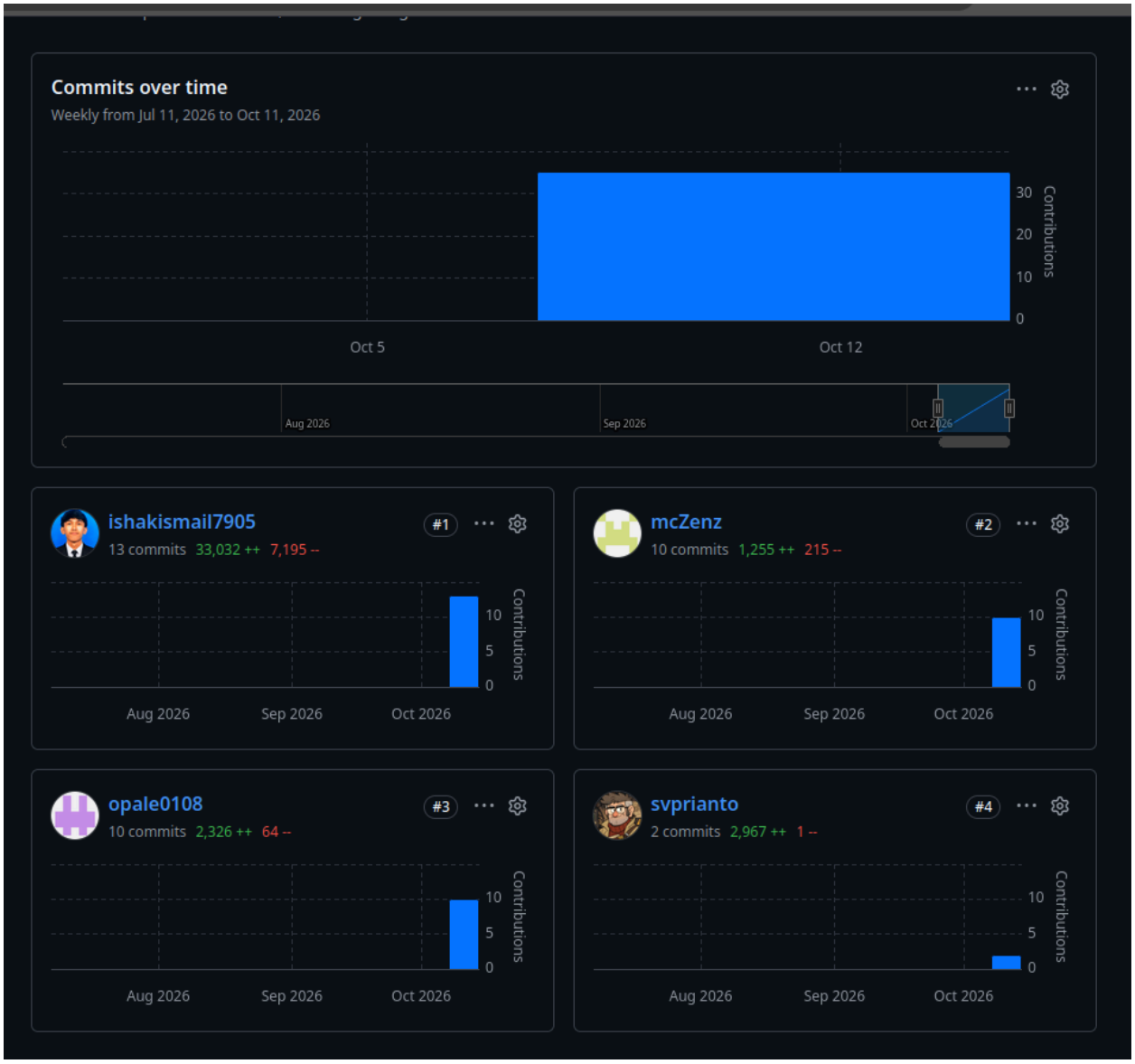


Figure 7.1: Contributors and Commits

8.0 Additional and Advanced Features

All the necessary security, middleware and API management features have been implemented, including one real third-party API. Security is provided by means of JWT login and an API key for machines to control who can call the API at all, and by means of roles and ownership checks to determine what each user may do once they have accessed it. The middleware and the central error handling ensure that every request is validated, logged and responded to in the same JSON format, even in the case of an error. Features of API management such as rate limiting and pagination are used to keep the API fast and to protect it against overuse, and the integration with goQR.me demonstrates the backend making a call to an external service on the client's behalf. The table below gives a list of each requirement, how it has been implemented and where evidence of its working can be found, together with the rest of this section explaining each one with the relevant code.

8.1 Security

8.1.1 JWT Authentication

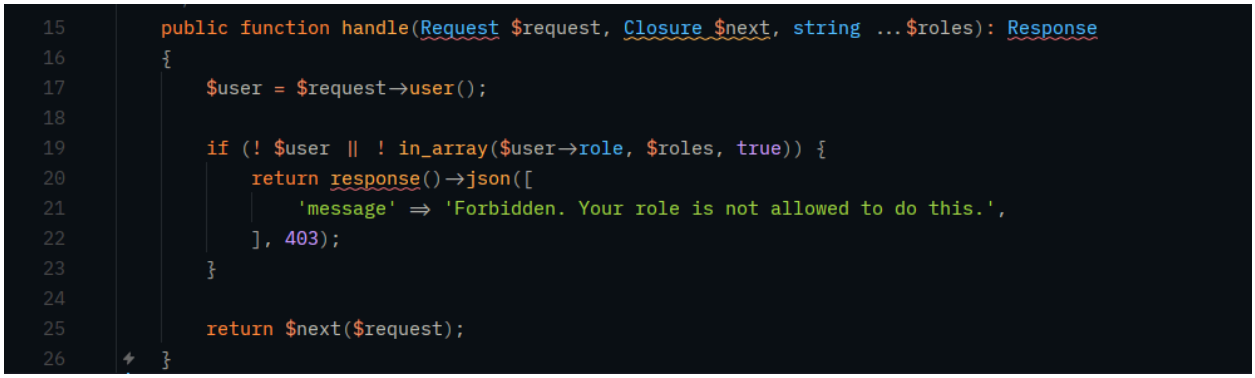
On login, a signed JSON Web Token is returned (Jones et al., 2015) and includes the user's ID and role. The client includes the token in the Authorization header and the server does not maintain any session. The php-open-source-saver/jwt-auth package is used both for signing and for checking the tokens (PHP-Open-Source-Saver, n.d.). The token is signed using HS256 with the secret found in JWT_SECRET, so if any part of its content is altered—for example, if the user changes their own role—the signature is broken and the server rejects the request with a 401 error. Since the role is contained within the token, the server knows who the caller is and what actions they are allowed to take without having to look up the session details. Each token is valid for 60 minutes, which limits the potential damage if a token is ever stolen, and the client can call the refresh endpoint in order to obtain a new token without having to log in again. When a user logs out or requests a token to be refreshed, the old token is added to a blacklist and thus ceases to function immediately.

8.1.2 Password and Roles

The passwords are stored as bcrypt hashes and never included in any response. Since the role column cannot be set during registration, every new account is automatically assigned the customer role and it is only possible for an admin to create or modify other roles. The User model has the password cast as hashed, which means that Laravel will hash it automatically each time it is saved (using 12 bcrypt rounds, as specified in the .env file), and the password is also listed as hidden so that it is omitted from all JSON replies. As a bcrypt hash cannot be converted back into the original password, even somebody who has access to the database will not be able to work out the actual passwords. Registration also requires the user to enter their password twice and mandates a minimum length of 8 characters. The role field is not part of the model's fillable list, so even if a request includes "role": "admin", the register method will ignore it and set the role to customer by itself.

8.1.3 Role-Based Access Control

The routes specify the roles that they accept, for instance ->middleware('role:admin,organiser'). The middleware is registered in the file bootstrap/app.php using the short name role, which means that any route or group of routes can make use of it with just one line. In routes/api.php, user management is restricted to users with the role:admin, creating and editing venues and events is allowed only for users with the role:admin,organiser, and booking tickets is available to users with the role:customer. Since the check takes place before the controller action, a user with the incorrect role is denied access with a 403 response before any data is read or modified. The middleware is short:



```
15      public function handle(Request $request, Closure $next, string ...$roles): Response
16      {
17          $user = $request->user();
18
19          if (! $user || ! in_array($user->role, $roles, true)) {
20              return response()->json([
21                  'message' => 'Forbidden. Your role is not allowed to do this.',
22              ], 403);
23          }
24
25          return $next($request);
26      }
```

Figure 8.1: EnsureUserHasRole.php Snippet Code

Just having roles is not sufficient when it comes to records that belong to someone, and therefore controllers also verify ownership. An organiser is allowed to modify only their own events, and a customer can see only their own bookings. Before modifying an event, the controller checks that the organiser of the event matches the currently logged-in user and sends back a 403 response if there is no match, provided the user is an admin. The bookings list is likewise filtered on the server, so that an organiser only sees the bookings for their own events and no one can view other people's records by simply changing the ID in the URL.

8.1.4 API Key

The ticket check is designed for use by a machine, for example a scanner at the entrance door, and therefore does not require a user to log in. A key is used and is compared using the hash_equals function, which takes the same amount of time regardless of whether the key is almost correct or entirely wrong, thus preventing an attacker from working out the key character by character by timing the responses. The actual key is never included in the code: it is obtained from the EVENTHUB_API_KEY entry in the .env file via config/eventhub.php, and the .env file is not committed to GitHub. If the key is empty or doesn't match the value in the X-API-Key header, the middleware returns a 401 status code with the message "Invalid or missing API key" and the ticket controller is never executed.

```php
$expected = (string) config('eventhub.api_key');
$given = (string) $request->header('X-API-Key');

// hash_equals compares in constant time, which avoids timing attacks.
if ($expected === '' || ! hash_equals($expected, $given)) {
    return response()->json([
        'message' => 'Invalid or missing API key.',
    ], 401);
}
```

Figure 8.2: EnsureValidApiKey.php Snippet Code

8.2 Middleware and Error Handling

8.2.1 Validation

Before carrying out any action, each controller action calls $request->validate() together with its rules. If a rule is not met, Laravel sends back a 422 status code accompanied by a message for each field (Laravel, n.d.-e). The rules verify more than simply whether a field is present; for a new event, venue_id must exist in the venues table, starts_at must be in the future, ends_at must come after starts_at and ticket_price must not be negative. With regard to a booking, the quantity must be a whole number between 1 and 10. The endpoints in the list also validate their query parameters so that a sort value which is not on the allowed list is rejected rather than being passed on to the database. Since validation takes place first, invalid input never gets to the business rules or the database.

8.2.2 Central Error Handler

All errors apart from those occurring under /api are converted into the same JSON format, which means that clients never receive an HTML error page or a stack trace (Laravel, n.d.-b). The relevant function is located in bootstrap/app.php and only applies to requests made under /api; on all other pages Laravel continues to function as it normally does. Validation errors are passed on without any change, so they retain their 422 status code and the individual field messages. In all other cases a status code and a brief, fixed message are provided: 401 if the token is missing or invalid, 404 if a record or URL does not exist, 405 if the wrong HTTP method is used, 429 when the rate limit has been reached and 500 for any other unexpected failure. In the case of a 500 error, the actual cause is not sent to the client, preventing database details or file paths from being disclosed, and for a 429 error the Retry-After header is maintained so that the client knows when to make the request again. This is essentially how it works:

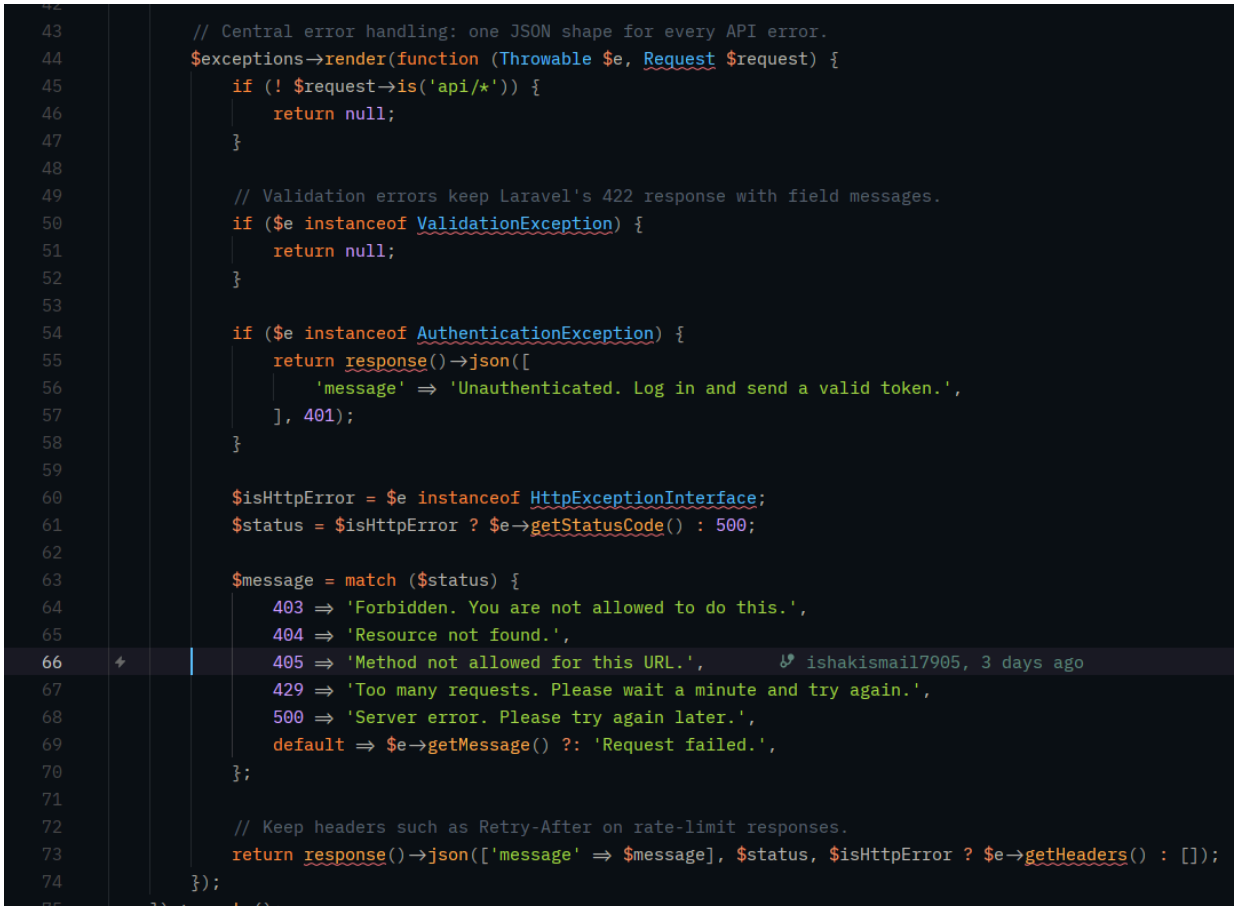


Figure 8.3: Central Error Handling Snippet Code

8.2.3 Logging

The logging middleware keeps a record of the method, the path, the status code, the time taken, the user and the IP address for each API request. Since it does not log the request bodies, passwords are not included in the logs. The middleware is included globally in the file bootstrap/app.php, which means that no route can avoid it, and it only adds an entry for paths that are under /api. It begins timing before the request is processed and then writes the entry after the response has been prepared, ensuring that the status code and the time recorded are the actual ones that the client receives, including errors such as 401, 403 and 429. The entries are directed to their own file, storage/logs/api.log, rather than to Laravel's main log, which makes it easy to see who made which call and to identify slow or failing requests when debugging.

8.3 API Management

8.3.1 Rate Limiting

Laravel's rate limiter includes two types of limiter. The first of these covers the entire API and permits 120 requests per minute, the number being calculated either per logged-in user or, in the case of guests, per IP address, and it is applied to all API routes through the throttleApi() function in bootstrap/app.php. The second type is designed to reduce the likelihood of password guessing. It is attached solely to the POST request to /auth/login and allows five attempts per minute for each combination of email and IP address, which means that an attacker cannot continuously try passwords for a single account, although other users on the same network can still log in. Whenever either of these limits is reached, the API responds with a 429 status code together with a Retry-After header that informs the client how long it should wait. The two limiters are defined in app/Providers/AppServiceProvider.php:

```
26          RateLimiter::for('api', function (Request $request) {
27              return Limit::perMinute(120)->by(auth('api')->id() ?: $request->ip());
28          });
29
30          // Login: 5 attempts per minute for each email and IP address,
31          // which slows down password guessing.
32          RateLimiter::for('login', function (Request $request) {
33              return Limit::perMinute(5)->by(strtolower((string) $request->input('email')).'|'.$request->ip());
34          });
```

Figure 8.4: AppServiceProvider.php Snippet Code

8.3.2 Pagination, Search, Filtering and Sorting

The endpoint entries for the four items on the list—users, venues, events and bookings—can all take these as query parameters. The default number of results returned is 10 per page and the per_page value can be set between 1 and 100, which means that a client will never have to request the entire table in one go. The search function matches against text fields such as names and titles, the filter parameters reduce the number of results based on values like category, status or date, and the sort option with direction determines the order. Since all the processing is carried out by the database query on the server side, the browser only gets the rows that it will display together with the paging information such as the current page and the total number of items. These different options can all be included in a single request.

8.4 Third-Party API: goQR.me

A QR code is obtained from the goQR.me web API, an external service accessed via HTTPS (goQR.me, n.d.), and the library is not installed locally. Upon a client requesting a QR code for a booking, the backend first verifies that the person making the request is authorised to view that booking, then passes the booking reference on to goQR.me and in return receives a PNG image. This image is converted into base64 and sent back as part of the usual JSON response, enabling the frontend to display it without having to make a direct contact with goQR.me. No account or key is required for this function, which ensures that secrets are not included in the request, and the address of the service is stored in config/eventhub.php so that it can be modified without having to alter the controller. The table below gives the details of the call.

The call employs Laravel's HTTP client with a timeout period of 10 seconds (Laravel, n.d.-c):

```php
$image = Cache::remember($cacheKey, now()->addDay(), function () use ($booking) {
    // Third-party API call: GET https://api.qrserver.com/v1/create-qr-code/
    $response = Http::timeout(10)->get(config('eventhub.qr_api_url'), [
        'data' => $booking->booking_reference,
        'size' => '300x300',
        'format' => 'png',
        'margin' => 10,
    ]);

    // throw() turns a 4xx or 5xx reply into an exception.
    $response->throw();

    return base64_encode($response->body());
});
```

Figure 8.5: BookingController.php Snippet Code

When goQR.me is slow or unavailable, the backend picks up the error and responds with a 502 status code together with a clear message. The frontend will still display the ticket and its reference number, including a 'Try again' button for the QR code.

```javascript
const { data } = await get(`/bookings/${booking.id}/qr`);
// data.qr_code is a PNG as a "..." address, so it works directly as an image source.
box.innerHTML = `<img src="${data.qr_code}" alt="QR code for booking ${esc(booking.booking_reference)}">`;
```

Figure 8.6: ticket.js Snippet Code

The booking reference is included in the QR code, allowing a scanner to use the API key to make a GET request to /api/tickets/{reference} in order to verify that the ticket is valid. The response from this request indicates whether the ticket is valid, and this is true only in the case of a confirmed booking, as well as providing the booking status, the event, the venue, the start time and the number of tickets, so that the door staff can immediately see whether or not to admit the holder. If the booking has been cancelled or if it has already been checked in, the response says that the ticket is not valid and thus the same code cannot be used twice. This completes the process: the external API converts the reference into a code that the customer can display, and the EventHub API checks that code at the door. Figure 4.12 illustrates the actual request and response.

9.0 Debugging and Performance Optimization

Nine important issues were identified and resolved during development, and caching the QR code reduced that endpoint's response time from 765 ms to 42 ms. The problems arose in every aspect of the project: in the Laravel setup and authentication, in the database migrations, in the PHP environment on Fedora, in the Postman scripts and in the frontend. For each issue the root cause was determined rather than just suppressed, and where appropriate the fix was supported by a Postman test or a note in the README so that the problem would not reoccur. This section first gives a list of the issues together with their causes and solutions, then goes on to explain how errors are handled at run time, and finally describes the performance improvements and the way in which they were measured.

9.1 Exception Handling

Errors that are expected are detected at the point where they occur and are converted into a clear response; if a booking violates a rule it returns a 409 status code together with the reason given. If the QR service fails it returns a 502 status code and the error is logged. In the case of any other unexpected error the central handler is invoked, it returns a 500 status code with a general message and never includes a stack trace. The QR call has a timeout of 10 seconds so that a slow external service cannot cause a request to hang, and before the 502 reply is sent the failure is recorded using report(). Since bookings are saved within a database transaction, if an error occurs part way through the entire change is rolled back and thus tickets are never deducted without a booking having first been created. On the client side the frontend converts each of these responses into a message that the user can understand, including a specific message when the server cannot be reached at all.

9.2 Performance Improvements

The third-party call is cached (measurement carried out). Since the QR image for a booking reference never changes, the backend stores it for one day using Laravel's cache (Laravel, n.d.-a). Each image is saved under a key that is constructed from the booking reference (booking-qr:<reference>) in base64 format. The initial request for a booking triggers a call to goQR.me via Laravel's HTTP client with a 10-second timeout, and the result is then stored. All subsequent requests for the same booking within that day are served directly from the cache, without requiring the server to respond. The response contains a from_cache field, enabling the tester to see which route was used. This also reduces the number of calls made to the free public service, which is important in the event that a large number of attendees open their tickets at the same time just before the event.

During the tests, the first request (as shown in Figure 4.12) had from_cache: false and required 765 ms, whereas the second request for the same booking (as shown in Figure 9.1) gave from_cache: true and took only 42 ms. The cached response is roughly 18 times quicker and remains functional even if goQR.me is not available.

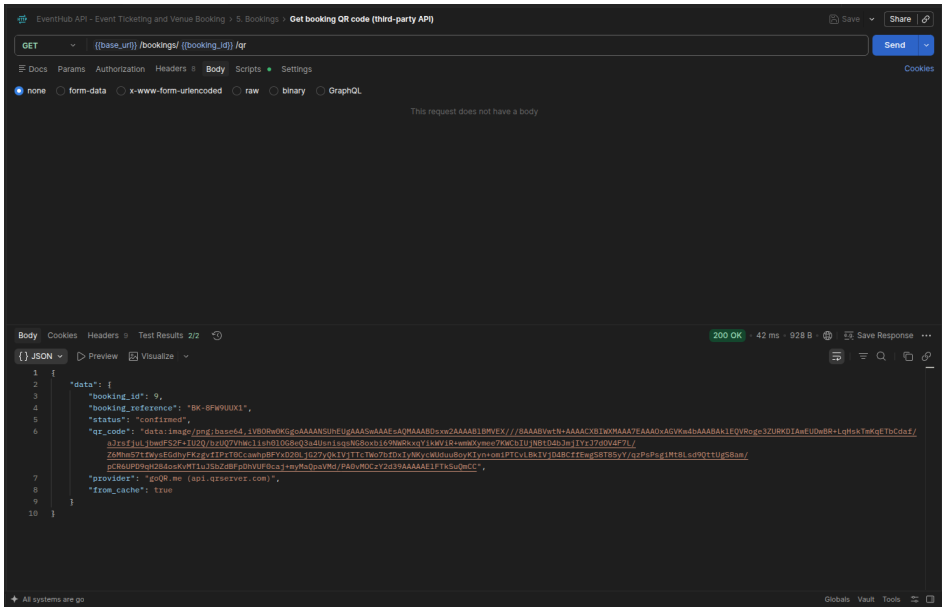


Figure 9.1: Get Booking QR Code (Third-Party API)

9.2.1 Fewer Database Queries by Design

Retrieve the load-related records for the endpoints by using eager loading and include only the columns that are actually needed. While the event list obtains the associated venues and organisers through two additional queries for the entire page, loading these details one event at a time would require 20 extra queries for a page containing 10 events. This situation is referred to as the N+1 problem, and the number of extra queries would continue to increase as the number of items per page rises. Similarly, the booking list and the ticket check perform the same operation, loading the event, venue and user associated with each booking with a fixed number of queries. For each relationship, only the columns that are required are named (for instance, venue:id,name,city), so that the database returns less data and the resulting JSON response remains small.



```
42          $events = Event::query()
43              ->with(['venue:id,name,city', 'organiser:id,name'])
44              ->when(! $canSeeAll, fn ($query) => $query->where('status', 'published'))
45              ->when($canSeeAll && isset($filters['status']), fn ($query) => $query->where('status', $filters['status']))
46              ->when($canSeeAll && isset($filters['organiser_id']), fn ($query) => $query->where('organiser_id', $filters['organiser_id']))
47              ->when($filters['search'] ?? null, function ($query, $search) {
48                  $query->where(function ($q) use ($search) {
49                      $q->where('title', 'like', "%{$search}%")
50                          ->orWhere('description', 'like', "%{$search}%");
51                  });
52              })
53              ->when($filters['category'] ?? null, fn ($query, $category) => $query->where('category', $category))
54              ->when($filters['venue_id'] ?? null, fn ($query, $venueId) => $query->where('venue_id', $venueId))
55              ->when($filters['date_from'] ?? null, fn ($query, $from) => $query->where('starts_at', '>=', $from))
56              ->when($filters['date_to'] ?? null, fn ($query, $to) => $query->where('starts_at', '<=', $to.' 23:59:59'))
57              ->when($request->boolean('available_only'), fn ($query) => $query->where('available_tickets', '>', 0))
58              ->orderBy($filters['sort'] ?? 'starts_at', $filters['direction'] ?? 'asc')
59              ->paginate($filters['per_page'] ?? 10);
60
61          return response()->json($events);
```

Figure 9.2: EventController.php Snippet Code

9.2.2 Indexes by Design

The event list available to the public is filtered according to status and is sorted by start time, which means that there is an index on (status, starts_at). When carrying out ticket checks, the system uses the unique index on booking_reference. If the index on the events table were not in place, MySQL would have to scan every row in the table in order to locate the published events and then sort them. With the index however, MySQL can go directly to the published rows, and these are already in order of start time. Since the status column is the first one in the index, this single index serves both the filtering and sorting needs. The unique index on booking_reference has two functions: it allows the organiser to carry out the ticket check with a single quick lookup and it ensures that two bookings can never have the same reference. There is also a unique index on email in the users table, which speeds up the login query, and an index on role. All of these indexes are defined in the Laravel migrations (for example, on line 27 of the events migration) and are also listed in database/01_ddl.sql.

9.2.3 Bounded Responses by Design

The lists are paginated, showing at most 100 items per page, which means that the size of the response does not increase as the table grows. This applies to all four of the list endpoints: users, venues, events, and bookings. Each of these returns 10 items by default, and a client can request more using the per_page parameter, but the validation rule max:100 rejects any amount exceeding 100 with a 422 error. If that limit were not in place, a single request for a table containing thousands of rows could cause all of them to be loaded into memory and result in a very large response, thereby slowing down both the server and the browser. Furthermore, Laravel's paginator includes in the response the total number of items, the number of the last page, and links to the pages, so the frontend can find out how many pages there are without having to make a further request.

10.0 Individual Reflection and Contribution Record

10.1 Member 1: Ishak Bin Ismail (AM2512021928)

I was responsible for most of the shared systems which are the Laravel backend, the MySQL database and its scripts, the Postman collection and the API documentation. I also built my own frontend, frontend-ishak, a single-page app in HTML, JavaScript and Bootstrap that covers all four roles. Two problems taught me the most. First, calling a protected endpoint without a token returned 500 with "Route [login] not defined" instead of 401 since Laravel tried to send the guest to a web login page. I set redirectGuestsTo(null), made the central error handler return 401 as JSON and added a Postman test for it. Second, seeding failed with "Unknown column 'role'" since I had run the migrations while the files were still empty so I completed them and rebuilt the database with migrate:fresh --seed. I learned that error responses matter as much as successful ones and that hiding a page by role in the frontend is only a convenience and the API must enforce the rule.

My contribution is recorded under the GitHub account ishakismail7905. I made 13 commits on 11 October 2026 from the Laravel setup (1136385) to a README update (62e4c3f), covering the database, authentication, all endpoints, the QR integration, security features, the Postman collection, documentation and my frontend. On 13 October 2026, I also merged pull requests #1 and #2 from other members. Evidence is in the backend, database, postman and frontend-ishak folders and in Figures 4.1 to 4.14, 6.1 to 6.13 and 9.1.

With more time, I would add automated PHPUnit feature tests and commit in smaller steps over several days so the history shows how the work developed.

10.2 Member 2: Ahmad Naufal bin Ahmad Fuad(AM2512021850)

My main responsibility was building my own frontend, frontend-naufal, which serves guests, customers, staff, organisers and admins through the shared EventHub API. Wrote nine pages, a shared request helper (api.js), a reusable UI helpers and a role guard (auth.js). On the backend side, I helped Ishak set up the MySQL database in XAMPP, working on its scripts database/01_ddl.sql (tables, keys and constraints) and database/02_dml.sql (sample data), locally ran the backend and tested its endpoints by using my frontend, and observed any errors in the Laravel log.

The most challenging part was grasping the concept of using Laravel and JWT authentication, which I hadn't yet encountered. I read docs/API.md and played with endpoints through the browser until I got the feel for how the token gets issued when I log in, and sent when I request an endpoint but gets rejected by 401 when it expires. This influenced my design: All requests pass through a single helper which manages the token and takes care of expired sessions.

Another issue was the installation of PHP on Windows. I created QR tickets and they were failing with a 502 and Laravel log reported that there is an issue with my manually installed PHP, due to the lack of a CA certificate bundle. It was solved by adding the following to the php.ini: loaded configuration file "cacert.pem". I also solved a merge conflict in the common README, now I have both members' rows.

I learned that fetch() will not fail to return when it returns a 4xx or 5xx response. I learned that the requirement to check access to the backend resource is still enforced even though the frontend role check is also done to improve user experience. I learned that the smaller the commit the easier it will be to review the code.

I have made my contribution using pull request #6 and the commits made in the frontend-naufal branch. I would change to renewing tokens before expiration using /auth/refresh in the future, add automated tests on the frontend and work on accessibility.

10.3 Member 3: Muhammad Shihabuddin Al-Fikri Bin Anuar (AM2512021974)

For this project, my main contribution was developing the frontend for the EventHub system using HTML, CSS and JavaScript. I worked on the user interface and connected it to the backend API so users could interact with the system. The features I worked on included event listing, search and filtering, login and registration, ticket booking, My Bookings, QR codes and management pages for different user roles.

During the development, I learned how a frontend actually communicates with a backend through API requests and responses. I also gained more experience when using JavaScript to send requests, handle JSON data, manage login tokens and display information from the API. Testing the features really helped me understand how different parts of the system work together.

One of the challenges was making sure the frontend handled API responses correctly, especially when a request failed or a user did not have permission to perform an action. I had to check the request methods, endpoint URLs and response data to make sure the features worked as expected. Testing the login, booking, cancellation and QR code features also helped me find and fix problems during development.

Overall, this project gave me a better understanding of web API development and frontend integration. I also learned that when building a working system requires more than just creating a good-looking interface and the frontend also must communicate properly with the backend, handle errors and provide clear feedback to users. This experience has helped me improve my practical skills and understand how the different parts of a web application work together.

10.4 Member 4: Muhammad Anas Bin Haddri (AM2512021976)

My primary duty was creating my own frontend, frontend-anas, which uses the shared EventHub API to serve visitors, clients, employees, organizers, and administrators. I created eight pages: a role-based navbar (nav.js), a shared API helper (api.js), reusable UI helpers (ui.js), and a role guard (auth.js). I tested every site using the example account for every role, including error scenarios like 403, 409, and 422, while running the backend locally.

Correctly addressing errors was the most difficult part. Because get() does not fail when the API returns a 403 or 409, a rejected booking initially appeared to be successful on my page. I resolved this by passing all requests through a single function called apiRequest(), which verifies the result.When the token expires with a 401, it displays the actual message from the API and returns the user to the login page.

The Windows QR ticket was another problem. The Laravel log displayed cURL error 60 since my PHP did not have a CA certificate bundle, and the QR pop-up repeatedly stated that the service was not responding. Adding the path to cacert.pem to php.ini resolved the issue. To prevent my peers from experiencing the same issue, I included these procedures in my README.

I discovered that while the backend must still verify each request, the frontend role check only enhances the user experience. I discovered that while editing, sending just the modified fields keeps requests tidy. Small commits also make the job easier to understand and review, I discovered.

Using 17 commits in the frontend-anas branch and pull requests #3, #4, #5, and #7 to #18 under the GitHub username mcZenz, I have contributed. In the future, I would share common helpers with the frontends of the other members and add automated tests for the frontend.

References

Bootstrap. (n.d.). *Get started with Bootstrap* (Version 5.3). Retrieved October 14, 2026, from https://getbootstrap.com/docs/5.3/getting-started/introduction/

Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022). *HTTP semantics* (RFC 9110). RFC Editor. https://doi.org/10.17487/RFC9110

goQR.me. (n.d.). *QR code API: Command "create-qr-code"*. Retrieved October 14, 2026, from https://goqr.me/api/doc/create-qr-code/

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). RFC Editor. https://doi.org/10.17487/RFC7519

Laravel. (n.d.-a). *Cache*. Laravel 13.x documentation. Retrieved October 14, 2026, from https://laravel.com/docs/13.x/cache

Laravel. (n.d.-b). *Error handling*. Laravel 13.x documentation. Retrieved October 14, 2026, from https://laravel.com/docs/13.x/errors

Laravel. (n.d.-c). *HTTP client*. Laravel 13.x documentation. Retrieved October 14, 2026, from https://laravel.com/docs/13.x/http-client

Laravel. (n.d.-d). *Release notes*. Laravel 13.x documentation. Retrieved October 14, 2026, from https://laravel.com/docs/13.x/releases

Laravel. (n.d.-e). *Routing*. Laravel 13.x documentation. Retrieved October 14, 2026, from https://laravel.com/docs/13.x/routing

Laravel. (n.d.-f). *Validation*. Laravel 13.x documentation. Retrieved October 14, 2026, from https://laravel.com/docs/13.x/validation

MDN contributors. (2025, August 20). *Using the Fetch API*. MDN Web Docs. Mozilla. Retrieved October 14, 2026, from https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

PHP-Open-Source-Saver. (n.d.). *Laravel JWT Auth*. Retrieved October 14, 2026, from https://laravel-jwt-auth.readthedocs.io/